

DeribitVerdictEngine — User Manual

Introduction

DeribitVerdictEngine is a VB.NET / .NET 8 Windows Forms desktop application on BTC-PERPETUAL that runs a multi-tier technical indicator pipeline on live market data and emits a directional verdict with supporting diagnostics. It is an analysis and decision-support tool, not an execution system — it does not send orders. Since v42 it runs live on a WebSocket feed (`network.transport = "ws"`) with per-run REST fallback; REST polling was the sole transport pre-v38.

This manual is a field-by-field reference for every variable and display block the engine renders — the **card layout** in the live UI, kept in parity with the **plaintext snapshot** (`BuildPlaintextSnapshot`) used for the output dump and quoted in this manual's example blocks. It assumes familiarity with BTC perpetual scalping, order flow, volume profile, and the trader profile in `docs/trader-profile.md`. For pipeline architecture and scoring steps, see `docs/architecture.md` and `docs/DeribitIndicatorProject.md`.

How to read the output (top to bottom):

1. **Verdict block** — the headline call, context qualifier, confidence tier, and raw/effective scores vs the regime-adjusted ceiling.
2. **ATR entry levels and Kelly sizing** — advisory reference frame for volatility context and directional conviction. See Kelly note below.
3. **Indicator sections** — each primitive's raw values and classification, ordered by tier (core → structural → microstructure → gates).
4. **Signal breakdown table** — the itemised scoring ledger. Reconciles every [L] / [S] hit and penalty against the TOTAL row, which matches the header's SCORE.

Important on Kelly sizing. The Kelly block is **advisory only** and uses an **ATR-basis** payoff ratio (`target_mult / stop_mult` — $1.75/1.6 \approx 1.09$ since v51; was 1.67 pre-v51). Per the trader profile, real execution uses **structural** stops and targets (previous swing low/high), not ATR multiples — so the displayed Kelly fraction does not correspond to your actual R:R. Treat Kelly as a directional-bias sanity check, not a position-sizing prescription. The advisory label is rendered inline under the KELLY SIZING header to reinforce this.

Source of truth. When this manual and the code disagree, the code wins. Primary source files: `UI/MainForm_Render_Cards.vb` (the live card layer — the P5b reskin retired the old `MainForm_Render_Header.vb / MainForm_Render_Sections.vb` RTF renderers), `UI/MainForm_PlaintextSnapshot.vb` (`BuildPlaintextSnapshot` — the canonical text surface kept in parity with the cards; this manual's field-by-field reference is anchored on it), `Core/ScoringEngine*.vb`, `Core/Indicators*.vb`, `analysis/*.vb`, `tools/AutoTweaker/*.vb`, and `settings.json` (version at line 1 of the file is the source of truth — v51 at this writing; auto-bumped by AutoTweaker after applied tweaks).

Display surface note. The app's output is rendered two ways that stay in parity: a **card layout** (`MainForm_Render_Cards.vb`) in the live UI, and a **plaintext snapshot** (`BuildPlaintextSnapshot`) used for the output dump, CSV-adjacent text records, and as the reference this manual quotes example blocks from. A few live elements — the WS-health status line, the EXIT GUARD strip, the ON-CLOSE countdown, and the TAPE microstructure strip — are status-bar elements outside this card/snapshot parity (by design, per the engine display-string parity rule in `CLAUDE.md`); they're documented separately in §22–§25.

Table of Contents

1. Verdict Block
2. ATR Entry Levels
3. Kelly Sizing
4. Dynamic Norms
5. Regime
6. Core Signals (1m)

7. VWAP
 8. BBW / TTM Squeeze
 9. EMA Ribbon
 10. Market Structure
 11. Open Interest
 12. Order Flow
 13. Liquidations
 14. MTF Gate
 15. Funding
 16. Signal Breakdown Table
 17. CSV Logging — analysis_log.csv
 18. Analysis Report Viewer
 19. Tweak Settings & Auto-Tweaker
 20. Settings Snapshot History
 21. Live Performance Display
 22. WebSocket Health Status Line
 23. Realtime Exit Guard
 24. On-Close Trigger Mode
 25. Live Microstructure Strip (TAPE)
 26. What-If Replay (Backtesting)
 27. MIN NET MOVE % Row (SETTINGS & TOOLS)
 28. BacktestRunner (offline CLI)
-

1. Verdict Block

The header panel at the top of each analysis run. Always present.

```
=====
VERDICT:      WEAK LONG
CONTEXT:      MOMENTUM_FADING
CONFIDENCE:   LOW
SCORE:        Long 8/15 (eff.7) | Short 2/15 (eff.1) | TRANSITIONAL penalty: -1
TIME:         2026-04-22 00:39:42 UTC+8
=====
LAST TRANSACTED PRICE:  76038.0
HOLD / EXIT: EXIT -- momentum break (ROC crossed below 0)
```

VERDICT

What: Final directional call from the scoring pipeline.

Calculation: In `ScoringEngine_Calculate_Verdict.vb` Step 5. After all scoring passes, funding modifiers, and veto gates produce `effectiveLS` and `effectiveSS`, these are compared against percentage thresholds of the regime-adjusted `MaxScore`:

- `tStrong` = `ceil(MaxScore × verdict_strong_pct)` — default 0.70
- `tMed` = `ceil(MaxScore × verdict_med_pct)` — default 0.53
- `tWeak` = `ceil(MaxScore × verdict_weak_pct)` — default 0.35

Long side is evaluated first; if long passes no tier, short is evaluated.

Possible values:

Verdict (displayed)	Stored / wire string	Trigger
STRONG LONG	STRONG LONG	$\text{effectiveLS} \geq \text{tStrong}$
MEDIUM LONG	LONG	$\text{tMed} \leq \text{effectiveLS} < \text{tStrong}$
WEAK LONG	WEAK LONG	$\text{tWeak} \leq \text{effectiveLS} < \text{tMed}$
STRONG SHORT	STRONG SHORT	$\text{effectiveSS} \geq \text{tStrong}$ AND short wins
MEDIUM SHORT	SHORT	$\text{tMed} \leq \text{effectiveSS} < \text{tStrong}$ AND short wins
WEAK SHORT	WEAK SHORT	$\text{tWeak} \leq \text{effectiveSS} < \text{tMed}$ AND short wins
NO TRADE	NO TRADE	Neither side clears tWeak
NO TRADE [WEAK LONG] / NO TRADE [WEAK SHORT]	(same, verbatim)	Regime veto (Step 4) or MTF Gate block (Step 4b) triggered, but the underlying score still has a weak lean. The bracketed tag is the suppressed direction.

Display ↔ stored mapping (v55, 2026-07-21). The screen (VERDICT line, verdict card) renders the middle band as MEDIUM LONG / MEDIUM SHORT so the on-screen ladder reads STRONG / MEDIUM / WEAK explicitly. Every other surface still carries the bare LONG / SHORT stored form — the CSV Verdict column, the bridge payload verdict field, the eval cache, and every string-matching site (AnalysisLogger, SignalEmitter.DeriveDirection, LivePerformanceTracker) are UNTOUCHED. The frozen bridge contract (schema v1) routes actionability through direction + confidence, which are unchanged. Anyone joining the CSV to the on-screen record joins on (Timestamp, Verdict) with the bare middle-band strings, exactly as before.

Interpretation: Trader-profile rule is act on MEDIUM / HIGH tiers only. WEAK readings are informational — they mean a partial setup exists but the cross-confirmation density is insufficient. NO TRADE [WEAK X] means “the scoring said X but a hard gate killed it” — the lean is real but you’re fighting either regime or the 15m timeframe. Stand down.

CONTEXT

What: Quality qualifier on the verdict. **Conditionally displayed** — only shown when VerdictContext != "CONFIRMED". If absent, the read is clean.

Calculation: In ScoringEngine_Calculate_Scoring.vb::CalcVerdictContext. Classifies the dominant side using three audits:

1. **Fading count.** Scores hits from: MicroCVDSignal = BULL_DECEL/BEAR_DECEL, TTMSignal = BULL_FADING/BEAR_FADING, RSI beyond divergence penalty threshold (>65 or <35), and MicroCVDLate collapsing to $\leq 50\%$ of MicroCVDEarly on the dominant side. ≥ 2 hits → MOMENTUM_FADING.
2. **Structural / flow split.** Counts structural breakdown hits (VWAP, BBW/TTM, EMA ribbon, DMI, ADX, Donchian, 5m EMA200) vs flow hits (OFI, CVD, TFI, MicroCVD, OI Delta, ROC, Volume). Structural $\geq \text{ContextTagStructuralMin}$ (3) AND flow $\leq \text{ContextTagFlowMax}$ (1) → FLOW_UNCONFIRMED.
3. **Thin signal set.** Both structural < 2 AND flow < 2 → STRUCTURALLY_WEAK.
4. Otherwise → CONFIRMED (line suppressed).

Possible values:

Context	Meaning
MOMENTUM_FADING	Dominant side’s momentum primitives are rolling over. Price can still print the verdict direction but the driving force is weakening.

Context	Meaning
FLOW_UNCONFIRMED	Clean structural setup with no order-flow backing.
STRUCTURALLY_WEAK	Often the precursor to a stall or fakeout. Neither structure nor flow has enough signal density.
BELOW_MIN_MOVE (v35)	The verdict exists but rests on very little. The directional verdict's realistic move (post-cap, post-Step-5b) is smaller than the minimum tradeable move. [v62] That floor is now <i>composed</i> — scoring.trade_costs : round-trip execution cost for the assumed style plus your minimum acceptable net move (defaults $2 \times 1.5 \text{ bps maker} + 0.05\% = 0.08\%$ of price, $\approx \$50$ at $\$62k$ — the same number the retired flat min_tradeable_move_pct carried). Overrides the verdict to NO TRADE even though scores/levels/breakdown still render for diagnostics. Computed at the end of Calculate() , after the regime veto and MTF gate — same pattern as those vetoes. Most common on low-ATR Asia/London runs by design.
CONFIRMED (<i>hidden</i>)	Balanced hits on both axes; no fading flags.

Interpretation: This is a *veto layer for human judgement*, not for the engine — it does not alter the score. A **WEAK LONG** with **MOMENTUM_FADING** is borderline; the same verdict with line absent is cleaner. Use it to choose between “pass” and “enter cautious”.

CONFIDENCE

What: Tier label corresponding to which threshold the dominant effective score cleared.

Calculation:

Confidence	Set by
HIGH	$\text{effectiveScore} \geq \text{tStrong}$
MEDIUM	$\text{effectiveScore} \geq \text{tMed}$ and $< \text{tStrong}$
LOW	$\text{effectiveScore} \geq \text{tWeak}$ and $< \text{tMed}$
N/A	Verdict is NO TRADE (no tier cleared, or veto/MTF block fired)

Interpretation: Maps 1:1 to the verdict tier — exists for at-a-glance readability rather than extra information. N/A on a **NO TRADE [WEAK X]** means the veto path was taken, not that score was zero.

SCORE

What: Raw and effective score totals for both sides, against the regime-adjusted ceiling.

Calculation:

- **LongScore / ShortScore** — the raw **ls / ss** values after Step 2, all Pass 2 upgrades, Pass 2b **OI**×**CVD**, Pass 2c regime alignment, and Steps 3/3b funding modifiers. These are the numbers shown in the breakdown table **TOTAL** row.
- **EffectiveLongScore / EffectiveShortScore** — raw scores after Step 4 regime veto adjustments. In **TRANSITIONAL**, an ADX-graded penalty is applied (floored by **TierFloor(rawScore)**). In **TRENDING_UP/TRENDING_DOWN**, these equal the raw scores (regime veto is an all-or-nothing **NO TRADE**, not a graded reduction).

- **MaxScore** — regime ceiling: TRENDING 20, RANGE_BOUND 19, TRANSITIONAL 15 (under default `regime_weights.enabled = true`). Subtract the alignment bonus (1) if the gate is disabled to get the pre-v13 values 19/18/15.

Display logic:

- **Normal form:** Long N/M | Short N/M — used when `RegimePenalty = 0`.
- **Penalty form:** Long N/M (eff.E) | Short N/M (eff.E) | TRANSITIONAL penalty: -P — used when `RegimePenalty > 0`. The penalty magnitude P is sourced from `regime_gates.transitional_penalty_low` (ADX 20–22.5) or `transitional_penalty_mid` (ADX 22.5–25).

Interpretation: The TierFloor guard matters. When raw score is high, the penalty is fully applied; when raw is already low, the floor prevents over-penalising — so a WEAK verdict at the floor is not as weak as it looks. Eye the delta between raw and effective: a large delta in TRANSITIONAL means the ADX is genuinely borderline and should inform position sizing independently of the verdict tier.

TIME

What: Local timestamp (UTC+8, hardcoded for the trader's timezone) when this run executed.

Calculation: `DateTime.UtcNow.AddHours(8)` formatted `yyyy-MM-dd HH:mm:ss UTC+8`. Not the time the data was captured by Deribit — the time your local process rendered.

Interpretation: Use to verify auto-run freshness and to align verdicts with session boundaries (Asia 00:00–07:59 UTC = 08:00–15:59 UTC+8; London 08:00–12:59 UTC = 16:00–20:59 UTC+8; NY 13:00–23:59 UTC = 21:00 UTC+8 – 07:59 next day UTC+8).

LAST TRANSACTED PRICE

What: Price of the most recent trade fetched from `GetRecentTradesAsync(100)`, above the ATR block.

Calculation: `recentTrades[0].Price`. Displays N/A if the recent-trades fetch returned empty.

Interpretation: This is **not** the entry price used in ATR levels — that uses the close of the last 1m candle (`r.CurrentPrice`). Compare the two to spot slippage between the closed candle and the live tape. A large divergence in the second or two since the candle closed signals an in-progress impulse.

HOLD / EXIT

What: Guidance line for an open position. **Conditionally displayed** — only shown when `HoldStatus != "N/A -- no open position"`. Requires the internal `posState` to be `InLong` or `InShort` (set via UI controls or programmatic state).

Calculation: `CalcHoldStatus` in `ScoringEngine_Helpers.vb`. Layered priority:

1. **Microstructure fast exit:** ≥ 2 adverse signals (MicroCVD DECEL + OFI opposing + TFI opposing + CVD opposing, side-dependent) → EXIT -- **microstructure deterioration** (<list>).
2. **Structural breaks:** ROC crossing zero against the position → EXIT -- **momentum break**. OBV divergence against position → EXIT -- OBV <X> divergence. RSI divergence against position → EVALUATE -- RSI <X> divergence.
3. **Single adverse microstructure:** EVALUATE -- <signal> signal, confirm with price action.
4. **ROC/RSI structural assessment:** ROC beyond take-profit band → TAKE PROFIT -- **extreme momentum, tighten stops**. RSI above/below hold threshold → HOLD -- **momentum intact**. RSI in evaluate band → EVALUATE -- **momentum weakening, consider scaling out**. RSI past evaluate threshold → EXIT -- **retracement too deep** ($RSI < 40$) / ($RSI > 60$).

All RSI/ROC thresholds sourced from `cfg.Scoring.HoldRoc*` and `HoldRsi*`.

Interpretation: Read top-down — the first matching layer wins. A fast-exit microstructure signal means you're already late; a TAKE PROFIT means you're running an outlier extension and should tighten rather than chase. EVALUATE lines are cognitive prompts, not mandates — price action on the 1m is the arbiter.

2. ATR Entry Levels

```
ATR ENTRY LEVELS (ATR 38.40 size x1.27 | 1.6x stop / 1.75x target | EXEC 1m)
  Long: Stop 75976.6 [STOP_CLAMPED] | Entry 76038.0 | Target 76105.2 --
> 76129.4 [PLACED @ 76129.4 (SWING_HIGH_5M)]
  Short: Stop 76099.4 [FALLBACK_ATR] | Entry 76038.0 | Target 75970.8 [FALLBACK_ATR] R:R 1:1.1 (ri
```

Row anatomy (v51): the stop **always** carries its source label. A **structural-placed** target renders in the arrow form — fallback value, arrow, placed value, [PLACED @ ...] — and prints **no inline R:R** (the Long row above). A **fallback or noise-suppressed** target prints its label plus the TRUE placed R:R and risk/rwd distances (the Short row above).

Structural-first since v51 (B4b). These are no longer a pure ATR frame — the engine places the **target and stop from market structure first**, and falls back to ATR multiples only when no structural level qualifies. The multipliers below are the **fallback**, not the default path. One shared routine, `SignalEmitter.ComputeSideLevels`, computes the placed levels for all four surfaces (this snapshot, the card, the `verdict_signal.json` bridge payload, and the CSV `Placed*` columns) so they can never disagree. `scoring.structural_levels.enabled = false` reverts byte-identically to the legacy v50 geometry (pure ATR stop + closest-wins `CAPPED @ target`).

Section Header

Format: ATR ENTRY LEVELS (ATR <atr> size x<sizeMult> | <stopMult>x stop / <targetMult>x target | EXEC <execRes>m)

- **atr** — `r.ATR`, raw ATR(7) at the run's execution resolution.
- **sizeMult** — `norms.ATRRef / r.ATR` (note: inverted from the underlying `ATRScaleFactor` — see Dynamic Norms below). Mirrors the trader's own `Base × AvgATR/CurrATR` sizing formula. **Display/sizing-context only since v32** — it is no longer applied to the stop/target distances below (see Calculation).
- **stopMult** — `cfg.Scoring.AtrStopMultiplier`, default **1.6** (v51; was 1.2). The **fallback** stop multiplier, and the clamp ceiling that structural stops are held under (see Stop placement).
- **targetMult** — `cfg.Scoring.AtrTargetMultiplier`, default **1.75** global (v51; was 2.0), with **LONDON 2.0 / ASIA 1.25** per-session overrides (`scoring.structural_levels.sessions`, v40 pattern). The **fallback** target multiplier — used only when no structural target places.
- **execRes** — `r.ExecResolution` (v36): 1 on NY, 3 on Asia/London. The whole execution-indicator stack (ATR, ROC, RSI, volume, etc.) runs at this resolution for the session; only the 5m regime classifier, 15m MTF gate, and 5m/15m swing pivots stay fixed. Don't compare an EXEC 3m ATR/level reading directly against an EXEC 1m one — they're different bar sizes.

All read live from config; the label display is dynamic, not hardcoded — the header target multiplier is **session-resolved** (`ExecutionResolution.ResolveFallbackTargetMultiplier`): 1.75x on NY, 2.00x on LONDON, 1.25x on ASIA. The **fallback** R:R is `targetMult / stopMult` — $1.75/1.6 \approx 1:1.1$ global (LONDON $\approx 1:1.25$, ASIA $\approx 1:0.8$). When a structural target places, the row switches to the arrow form and prints no inline R:R — read the structural rows beneath the ATR block for the swing R:R, or compute from the placed values. Fallback/suppressed rows print the **TRUE placed R:R** inline, computed from the placed stop/target distances (not the multiplier ratio — the placed stop may be structural).

v32 D2 (S-1) note. Pre-v32, stop/target distances were `r.ATR × ATRScaleFactor × mult` — quadratic in volatility (the live ATR was already volatility-relative, then re-scaled again by the same ratio). This double-counted volatility and didn't match the eval pipeline, which measures barriers on raw ATR. Distances are now linear (`r.ATR × mult` only); the former scale factor survives purely as the `size × N` sizing-context display.

Stop / Entry / Target (Long) — fallback calculation

What: Long-direction trade frame. The formulas below are the **ATR fallback path**; the placement ladder and DG1 stop (next sections) override them whenever structure qualifies.

- `atrStop = r.ATR × stopMult` (also the DG1 clamp ceiling)
- `atrTarget = r.ATR × targetMult` (session-resolved multiplier)
- `longStop = r.CurrentPrice - atrStop`
- `longTarget = r.CurrentPrice + atrTarget`
- `Entry = r.CurrentPrice` (close of the last execution-resolution candle, not the live tape).

Stop / Entry / Target (Short)

Mirrored: `shortStop = r.CurrentPrice + atrStop`, `shortTarget = r.CurrentPrice - atrTarget`.

R:R / risk / rwd (v51: true placed geometry)

- **R:R** — `FormatRR(rwd, risk)` computed from the **placed** distances, not the multiplier ratio. Printed only on fallback/noise-suppressed rows (structural-placed rows use the arrow form with no inline R:R). Pure-fallback rows come out $\approx$ the multiplier ratio (1:1.1 NY); a row whose stop is structural (`SWING_STOP`) prints a genuinely different figure.
- **risk** — `|entry - placed stop|` in price points.
- **rwd** — `|placed target - entry|` in price points.

Target placement ladder (structural-first, v51)

The `atrStop / atrTarget` in the calculation above are the **fallback**. Before using them, `SignalEmitter.ComputeSideLevels` walks four target tiers and places the target at the **first** one that sits a workable distance from entry ($0 < \text{dist} \leq \text{scoring.structural_levels.target_max_atr_mult} \times \text{ATR}$, default bound $3.5 \times \text{ATR}$). Structure wins **even when it is farther than the ATR fallback would be** — that is the key v51 behavioural change (the old cap only ever pulled the target *closer*):

1. **Swing** — confirmed 5m swing high (long) / low (short). Label `SWING_HIGH_5M / SWING_LOW_5M`.
2. **Nearest HVN wall** — `VPFNearestHvnAbove` (long) / `VPFNearestHvnBelow` (short). Label `NEAREST_HVN_ABOVE / NEAREST_HVN_BELOW`.
3. **POC** — the VPFR point of control, only when the HVN gate is open. Label `POC`.
4. **ATR fallback** — `entry ± targetMult × ATR` when no structural level qualifies within the bound. Label `FALLBACK_ATR`.

Optional best-pivot tier (v63) — OFF in the shipped settings. `scoring.structural_levels.use_best_pivot_candidate` is a Boolean that defaults to `false`, and the tracked `settings.json` ships it `false`. When it is `false` the candidate is simply absent and every downstream branch is byte-identical to v56 — snapshot, card, bridge payload and CSV all render exactly as they did before the key existed. **So this is not something the engine currently does.** The key is a **what-if instrument**: it exists so the §10 promotion question (“should the volume-weighted pivot be a target tier at all?”) can be measured in the replay before it is answered live. Live promotion is a separate, later decision.

When the key is set `true`, the volume-weighted best 5m pivot (`BestPivotByVolume5m`, §10) joins the target candidate set:

- **Ladder mode** (`structural_levels.target_arbitration_mode: 0`, the shipped default) inserts it as the **first** tier, above swing — it wins whenever it qualifies.
- **NEAREST mode** (`target_arbitration_mode: 1`) gives it no priority at all; it competes on distance like every other candidate.
- **Side is decided by price against entry**, not by whether the pivot was a swing high or a swing low: above entry $\Rightarrow$ long-target candidate, below entry $\Rightarrow$ short-target candidate. A *low* pivot sitting above entry is still

a level the market defended on volume. (This is also the only rule the live path and the replay can share — the CSV logs `BestPivotByVolume5m` and `BestPivotVolumeRatio5m` but not `BestPivotIsHigh5m`.)

- **Same looseness bound as every other tier** — $0 < \text{dist} \leq \text{target_max_atr_mult} \times \text{ATR}$. A pivot price of 0 (fewer than 2 confirmed pivots in the lookback) means the candidate is absent, not guessed — the POC-tier precedent.
- `BestPivotVolumeRatio5m` is **not** consulted. There is no minimum-ratio gate.
- Label when it places: `BEST_PIVOT_5M`, rendered through the same `PLACED @ ...` string as every other tier — no new line, no format change.
- **The stop side is untouched.** This is a target-only candidate; the DG1 stop below is unchanged.

Sweeping it is covered in §26 (“Use best-pivot candidate”). Source: `Core/SignalEmitter.vb::ComputeStructuralSideLevels`
`spec docs/d2v2-whatif-candidate-mode-proposal.md`.

Stop placement (DG1)

`Stop = min(structural swing stop, stopMult × ATR)`, floored at `scoring.structural_levels.stop_min_floor_ticks` (default 4 ticks). Structure is used **only when it is tighter** than the $1.6 \times \text{ATR}$ ceiling — 5m swing stops run p50 $4-9 \times \text{ATR}$, usually too wide at v1 fixed sizing, so the clamp binds on most rows. Stop-row labels:

- `SWING_STOP` — the structural swing stop was tighter and used as-is.
- `STOP_CLAMPED` — a structural stop existed but was wider than $1.6 \times \text{ATR}$, so the ATR ceiling bound. **Expected on most structural-stop rows** at v1 fixed sizing; the un-clamp waits on the order-app’s sizing-by-stop-distance (placed-geometry derivation §6b).
- `FALLBACK_ATR` — no usable structural stop, so the pure $1.6 \times \text{ATR}$ stop was used.

Display / labels

A structural-placed target renders in the **arrow form** `Target <fallback> --> <placed> [PLACED @ <placed> (<LABEL>)]` — the raw ATR-fallback value first (dimmed), the placed value and reason in amber — with **no inline R:R** on that row. The stop always carries its source label (`[SWING_STOP]` / `[STOP_CLAMPED]` / `[FALLBACK_ATR]`). Legacy geometry (`structural_levels.enabled:false`) renders `CAPPED @ ...` instead of `PLACED @ ...`.

Sub-tick suppression (v30 rule, moved into the arbitration at v51): when $|\text{fallback} - \text{placed}| < \max(0.5, \text{ATR} \times 0.02)$ (0.5 = one tick) the placement is treated as noise inside `ComputeSideLevels`: the row renders in the fallback form but keeps the structural target label (e.g. `[SWING_HIGH_5M]` with inline R:R), and — a v51 semantic change — the suppression propagates to Step 5b, so `AdjustedTarget` stays 0 and the CSV `TargetCapReason` logs none (pre-v51 the suppression was renderer-only and the CSV still recorded the cap). Unsuppressed placements log the full `PLACED @ ...` string to CSV `TargetCapReasonLong` / `TargetCapReasonShort`.

Alignment with trader-profile (updated v51). These levels are now **structure-first**, much closer to your actual method (`trader-profile.md` §2/§4–5: structural swing targets and stops). They remain **display/advisory** — the engine does not place orders — but the gap between what the engine shows and how you trade narrowed sharply at v51. The surviving caveat: most structural **stops** are clamped to $1.6 \times \text{ATR}$ at v1 fixed sizing (`STOP_CLAMPED`), so the shown stop is often tighter than your true swing-invalidation level until stop-distance sizing lands on the order-app side.

3. Kelly Sizing

KELLY SIZING [CAPPED]

Advisory (ATR-basis) – R:R uses ATR multiples, not structural targets.

Treat as directional bias indicator only.

p(win): 45.0%

f* / Half-Kelly: 12.00% / 6.00%

Applied fraction: 5.00%
Risk \$: \$50.00
Contracts: < 1 contract (stop too wide for min size)

Rendering gate: Block only appears when `v.KellyPWin > 0`. `CalcKellySizing` early-exits (leaving all Kelly fields at 0) when verdict is empty/NEUTRAL/WAIT, or `stopDistanceUsd <= 0`, or `AtrStopMultiplier <= 0`. Under normal operation this means the block renders for every real verdict, including NO TRADE.

Section Header

Format:

- Normal: KELLY SIZING (optionally [CAPPED])
- NO TRADE: KELLY SIZING [BIAS ONLY – NO TRADE] (optionally [CAPPED])

[CAPPED] tag appears when `KellyCapped = true`, which is true when half-Kelly exceeded `MaxRiskFraction` (default 5%) and the applied fraction was clamped down.

[BIAS ONLY – NO TRADE] tag appears when `v.Verdict.StartsWith("NO TRADE")`. In this mode Kelly is computed but labelled as direction-only — do not trade.

Historic note: Earlier versions also showed [EST] or [CAL] mode tags. CAL mode was removed (see v14 changelog); `KellyPMode` is still assigned "EST" internally but no longer rendered. Tag will return once a back-testing module supplies empirical win rates.

Advisory label (always present)

Two dim-grey lines immediately under the header:

Advisory (ATR-basis) – R:R uses ATR multiples, not structural targets.
Treat as directional bias indicator only.

Fixed literal text. Rendered unconditionally whenever the Kelly block renders.

Interpretation: A deliberate reminder that the Kelly fraction is computed off the ATR R:R ratio (1.75/1.6 ≈ 1.09 since v51; 1.67 pre-v51), not the structural R:R the trader actually uses. Read the block as “the engine’s directional conviction, translated into a sizing hint” rather than a prescription.

p(win)

What: Win probability used in the Kelly formula.

Calculation: `ScoringEngine_Kelly.vb`. EST-mode mapping of Confidence onto a probability band:

Confidence	p
HIGH	$\text{EstProbFloor} + \text{EstProbScale} = 0.45 + 0.20 = \mathbf{0.65}$
MEDIUM	$\text{EstProbFloor} + \text{EstProbScale} / 2 = 0.45 + 0.10 = \mathbf{0.55}$
LOW / other	$\text{EstProbFloor} = \mathbf{0.45}$

Floor and scale are `cfg.Kelly.EstProbFloor` (0.45) and `cfg.Kelly.EstProbScale` (0.20).

Interpretation: Pre-calibration tiering. These numbers are engineered priors, not measured outcomes. HIGH confidence asserts a 65% win prior — treat with scepticism until backtesting can replace these with observed rates.

f* / Half-Kelly

What: Raw Kelly fraction and its half-Kelly damped variant, as percentages.

Calculation:

- $b = \text{cfg.Scoring.AtrTargetMultiplier} / \text{cfg.Scoring.AtrStopMultiplier}$ — default $2.0 / 1.2 = 1.667$.
- $q = 1 - p$.
- $f^* = (b \times p - q) / b$. Negative means no edge under these priors.
- $f_{\text{Half}} = f^* / 2$ if `cfg.Kelly.UseHalfKelly` (default true), else f^* .

If $f^* \leq 0$, the entire block exits silently (see gate above).

Interpretation: Half-Kelly is a standard industry damping to trade the drawdown-vs-growth trade-off. f^* above 20% with $p \approx 0.65$ is already aggressive; real-world edge is usually closer to $p \approx 0.5$, which gives the common Kelly ≤ 0 “stand down” result.

Applied fraction

What: The fraction actually used for the Risk \$ computation, after the hard cap.

Calculation: $f_{\text{Applied}} = \min(f_{\text{Half}}, \text{cfg.Kelly.MaxRiskFraction})$. Default cap 0.05 (5%). `KellyCapped = fHalf > MaxRiskFraction`.

Interpretation: Any time you see [CAPPED], the engine thinks you have more edge than the 5% safety cap permits. Don’t try to override it — cap exists because pre-calibration p-values are unreliable upward.

Risk \$

What: Dollar risk per trade at the applied fraction.

Calculation: $\text{cfg.Kelly.AccountSizeUsd} \times f_{\text{Applied}}$. Default account $1000 \times 5\%$ cap = \$50 maximum.

Interpretation: `AccountSizeUsd` is a placeholder pending trader input. Scale mentally: the displayed Risk \$ $\times$ (your real account / 1000) is the rough translation. Or just set the field correctly in `settings.json` and the math propagates.

Contracts / Lean

What: Recommended contract count, derived from dollar risk and ATR stop distance.

Calculation: $\text{KellyContracts} = \text{floor}(\text{KellyRiskUsd} / (\text{ContractFaceUsd} \times \text{stopDistanceUsd}))$. `ContractFaceUsd` default 10.

Display variants:

Condition	Label	Value
Normal, <code>Contracts</code> ≥ 1	<code>Contracts:</code>	N contracts
Normal, <code>Contracts</code> < 1	<code>Contracts:</code>	< 1 contract (stop too wide for min size)
NO TRADE, <code>Contracts</code> ≥ 1	Lean:	N contracts (not a trade signal)
NO TRADE, <code>Contracts</code> < 1	Lean:	< 1 contract (bias only; not a trade signal)

Interpretation: < 1 contract on a normal verdict means the ATR stop distance is wide enough that even your full risk budget doesn’t buy a single minimum contract — the trade is impractical at this volatility unless you widen the risk limit. In BIAS ONLY mode, any contract count is directional colour only; don’t work the order.

Risk-per-contract formula (fixed v32 D1). $\text{riskPerContract} = \text{ContractFaceUsd} \times \text{stopDistanceUsd} / \text{entryPriceUsd}$ — correctly dimensioned for the Deribit inverse contract. (Pre-v32 the formula omitted `entryPriceUsd` in the denominator, which made `riskPerContract` $\sim 1e4\times$ too large and `Contracts` always rounded to 0 — every run showed `< 1 contract`. Fixed; this is no longer a known limitation.)

Notional / leverage (v32 D1)

At the corrected sizing, leverage can bind before the dollar risk cap does. A new `kelly.max_leverage` setting (default 5.0 $\times$) caps contracts at `floor(AccountSizeUsd  $\times$  max_leverage / ContractFaceUsd)`; the engine applies `min(risk-derived contracts, leverage-derived contracts)`.

Notional: $\approx \$5,000 \cdot 5.0\times \text{lev}$ [LEV CAPPED]

- `Notional` — `KellyContracts  $\times$  ContractFaceUsd`.
- `lev` — `Notional / AccountSizeUsd`, the implied leverage at the suggested contract count.
- [LEV CAPPED] tag — appears (`VerdictResult.KellyLevCapped`) when the leverage ceiling, not the 5% dollar risk cap, was the binding constraint. Rendered only when `KellyContracts  $\geq 1$` .

4. Dynamic Norms

DYNAMIC NORMS [LIVE]

Vol threshold : H:3.96x M:2.44x (mean=6.2898 BTC s=7.6716)

VWAP dev thr : +/-0.30% (legacy ref)

ATR ratio : 0.79x (ATR=57.60 ref=72.58)

Label note (v32): this row was `ATR scale` before v32; relabelled `ATR ratio` because — since the v32 D2 linear-distance fix (see §2 ATR Entry Levels) — this value no longer scales the ATR stop/target distances. It is now purely a display read of current-vs-reference volatility; the `size  $\times N$` figure in the ATR Entry Levels header is the same underlying ratio (inverted) presented as a sizing multiplier.

The adaptive threshold layer (`DynamicNorms.vb`). Computed per run, applied to scoring thresholds downstream (Volume classifier, ATR-scaled targets/stops).

Section Header Tag

Format: DYNAMIC NORMS [LIVE] or DYNAMIC NORMS [STATIC FALLBACK].

- [LIVE] — `DynamicNorms.Compute` produced results from the current 1m candle history (≥ 30 candles, ≥ 10 volume samples).
- [STATIC FALLBACK] — cold start or insufficient history; values come from static constants in `cfg.Indicators.{Volume,VWAPDynamic,ATR}`.

Interpretation: STATIC FALLBACK after a long uptime means the Deribit API call failed or returned stale data — treat the verdict with suspicion until the next run flips back to LIVE.

Vol threshold

What: Dynamic upper and mid volume ratio thresholds used by the Volume signal in Step 2, and the mean / stdev `s` statistics underlying them.

Calculation: In `DynamicNorms.Compute`:

- `volMean` — rolling average of the last `min(100, candles - 1)` 1m candle volumes (BTC).
- `volSD` — population standard deviation over the same window.
- **Collapsed-variance fallback:** If `volSD < volMean  $\times$  0.05`, static values are used: `H = cfg.Indicators.Volume.StaticH (3.0)`, `M = StaticMid (2.0)`. This prevents division-by-noise when volume has flat-lined.
- **Normal case:**

- $\text{highRaw} = (\text{volMean} + 2 \times \text{volSD}) / \text{volMean}$
- $\text{midRaw} = (\text{volMean} + 1 \times \text{volSD}) / \text{volMean}$
- Each clamped to $[\text{DynamicHighClampMin}, \text{DynamicHighClampMax}] = [2.0, 6.0]$ (v14) and $[\text{DynamicMidClampMin}, \text{DynamicMidClampMax}] = [1.5, 4.0]$ (v14).
- **Session multiplier:** After clamp, `ApplySessionVolume` multiplies both thresholds by the bucket entry matching current UTC hour (`session_volume.sessions[]`). Defaults: ASIA 00–07 $\times$ 0.80/0.85, LONDON 08–12 $\times$ 1.00/1.00, NY 13–23 $\times$ 1.15/1.10. Bypassed when `session_volume.enabled = false` or no bucket matches.

Possible ranges: H: 1.6–6.9x. M: 1.2–4.4x. Extremes imply the session is either dead quiet or extremely expansive.

Interpretation:

- `VolumeRatio` $\geq$ H on a candle with price moving in-direction fires a full Volume signal. Below H, above M, and with cross-confirmation fires a mid-tier partial via Pass 2.
- A live H value near the clamp floor (2.0x NY-adjusted, $\approx$ 2.3x after session mult) in a quiet session is telling you “relative expansion is cheap here — don’t over-weight volume spikes”. Conversely a high H ($>5x$) during quiet intervals means only genuine flushes will register.
- The trader’s spec rule is “breakout needs $> 3x$ SMA(9)”. The dynamic H replaces the literal 3.0 threshold; when live H drops below 3.0, the engine is relaxing that rule based on session context. Override mentally if you want strict 3x.

VWAP dev thr

What: Dynamic VWAP deviation threshold (as percentage). Displayed with a `(legacy ref)` tag.

Calculation:

- Computed by running a rolling cumulative VWAP over the last $\text{min}(50, \text{candles} - 1)$ samples and collecting $|\text{close} - \text{vwap}| / \text{vwap} \times 100$ per sample.
- $\text{threshold} = \text{clamp}(\text{mean}(\text{devs}) + \text{stdDev}(\text{devs}), \text{cfg.Indicators.VWAPDynamic.DevClampMin}, \text{DevClampMax}) = \text{clamp}$ to $[0.30, 3.0]$.
- Falls back to `StaticFallback` (1.5) if fewer than 10 samples.

Possible values: 0.30–3.0 (%), clamp-bounded.

Interpretation: Marked `(legacy ref)` because the live VWAP scoring uses sigma bands (σ_1/σ_2) around the dual-session VWAP — this single-dev-% threshold is retained for display/historical compatibility but does not gate the VWAP scoring signal. Read it as a rough intraday volatility-around-VWAP indicator: values near the floor imply tight mean-reversion behaviour; values $> 1\%$ imply meaningful one-sided pressure. Not a trading input — use the sigma-band VWAP rows in the scoring breakdown instead.

ATR ratio (relabelled from “ATR scale” in v32)

What: The live ATR ratio — current ATR vs its rolling reference — plus the inputs. **Display/sizing-context only since v32;** it no longer feeds the ATR Entry Levels stop/target distances (see §2).

Calculation:

- `ATRRef` — rolling mean of ATR over the recent window ($\text{min}(100, \text{candles} - \text{period})$ rolling ATR values at the run’s execution resolution, default ATR period 7). On cold start or insufficient history falls back to `cfg.Indicators.ATR.StaticRef` (38.0 since v37 — see the note below; was 115 pre-v37).
- $\text{ATRScaleFactor} = \text{clamp}(\text{r.ATR} / \text{ATRRef}, \text{cfg.Indicators.ATR.ScaleMin}, \text{ScaleMax}) = \text{clamp}$ to $[0.25, 4.0]$.
- When `r.ATR = 0` or `ATRRef = 0`, factor defaults to 1.0 and ref falls back to `StaticRef`.

Displayed:

- `ratio` — the factor (e.g. 0.79x).

- **ATR** — current raw ATR (price points) from `r.ATR`, at the run's execution resolution.
- **ref** — rolling reference ATR or static fallback.

v37 ATR-band recalibration (2026-06-17). `cfg.Indicators.ATR.StaticRef` moved 115.0 → 38.0 — the old cold-start anchor was calibrated for BTC ~\$80–100k and ran ~3× the live 1-min ATR mean once price settled around \$62–67k. This only affects the cold-start fallback (the live ref self-calibrates from a recent rolling average once the window fills); it's not a scoring knob. Current reference bands, resolution-dependent since v36: **1-min (NY)** Low<20 / Normal 20–55 / High>55; **3-min (Asia/London)** Low<42 / Normal 42–115 / High>115 (~2.1× the 1-min bands). These track BTC's price regime and may drift — flag if you notice the live ATR distribution moving away from them.

Interpretation:

- **ratio > 1.0** → current ATR is above its rolling baseline — expansion regime. Per trader-profile sizing, scale down: low-ATR = larger size, high-ATR = smaller size. This is purely informational for your own position sizing now — it does not touch the ATR Entry Levels stop/target distances.
- **ratio < 1.0** → current ATR is below baseline — compressed. Size up within risk limits.
- **ratio = 1.0** exactly on any non-warm-up run means either the raw ATR matched the reference or the clamp bit — check the raw values.
- Static-fallback **ref = 38.0** vs live **ref = <computed>** is a useful freshness cross-check: if static ref is quoted while **[LIVE]** tag is shown, the live-ref path computed 0 and fell back inside an otherwise-live run (rare, usually means a data gap).

5. Regime

REGIME (5m): TRANSITIONAL

ADX: 24.3 | +DI: 27.5 | -DI: 9.6

The regime classifier. Computed **on the 5m timeframe** (not 1m), intentionally — regime is a slower-moving structural read.

Regime label

What: The classifier output, driving the **MaxScore** ceiling, the Step 4 regime veto, and the Pass 2c alignment gate.

Calculation: In `MainForm_Analysis.vb` after `CalcDMI(candles5m, cfg.Indicators.ADX.Period=9)`:

1. Raw classification:

- `ADX > trend_threshold (25) AND +DI > -DI` → `TRENDING_UP`
- `ADX > trend_threshold (25) AND -DI > +DI` → `TRENDING_DOWN`
- `ADX < range_threshold (20)` → `RANGE_BOUND`
- Else (`ADX 20–25`) → `TRANSITIONAL`

2. **1-bar hysteresis** (prevents flip-flop on noisy ADX boundary): If `rawRegime = RANGE_BOUND` AND the previous tick was `TRENDING_UP` / `TRENDING_DOWN` / `TRANSITIONAL`, the displayed regime holds `_prevRegime` for one bar. Otherwise the raw classification is used directly. `_prevRegime` updates to the raw value at the end of every run regardless.

Possible values:

Regime	MaxScore (v14 defaults)	Scoring impact
TRENDING_UP	20	Step 4 veto: NO TRADE if short side wins. Pass 2c TRENDING alignment active.

Regime	MaxScore (v14 defaults)	Scoring impact
TRENDING_DOWN	20	Step 4 veto: NO TRADE if long side wins. Pass 2c TRENDING alignment active.
RANGE_BOUND	19	No veto. Pass 2c RANGE_BOUND alignment active (VWAP+RSI+Donchian).
TRANSITIONAL	15	Step 4 graded ADX penalty (see below). Pass 2c suppressed.

Base values (RegimeWeights.Enabled = false): 19 / 18 / 15. The +1 / +1 / 0 bonuses come from `cfg.RegimeWeights.{Trending,RangeBound}.AlignmentBonus`.

TRANSITIONAL penalty ladder:

ADX band	RegimePenalty
[20.0, 22.5)	2 (transitional_penalty_low)
[22.5, 25.0)	1 (transitional_penalty_mid)

Applied with a TierFloor guard so the raw score cannot penalty-fall below its tier floor (`ls ≥ 12 → floor 9`, `ls ≥ 9 → floor 6`, `ls ≥ 6 → floor 3`, else floor 0).

ADX / +DI / -DI

What: Wilder smoothed ADX and directional indicator values on the 5m series, period 9.

Calculation: `CalcDMI` in `Indicators.Momentum.vb`. Wilder-smoothed true range and directional movement; $+DI = 100 \times \text{smoothedDMPlus} / \text{smoothedTR}$, $-DI = 100 \times \text{smoothedDMMinus} / \text{smoothedTR}$, $DX = 100 \times | +DI - -DI | / (+DI + -DI)$, ADX = Wilder-smoothed DX over period.

Interpretation:

- The spread (+DI) – (–DI) is the directional conviction. In the sample above, 27.5 vs 9.6 is a strong +DI bias — but ADX 24.3 is still sub-trend, so the regime label says TRANSITIONAL rather than TRENDING_UP. Read this as “direction is clean but the trend hasn’t *earned* the label yet”.
- Rising ADX through 25 with +DI dominant is the canonical TRANSITIONAL → TRENDING_UP transition; watch for it over a few consecutive runs.
- RANGE_BOUND after a trend (ADX dipping below 20 following a period above 25) is the regime that triggers the 1-bar hysteresis — meaning one tick of “false range” after a trending sequence will display the previous label. Actual RANGE_BOUND only appears after two consecutive qualifying reads.

Colour cue (display only)

Regime line colour: TRENDING_UP green, TRENDING_DOWN red, RANGE_BOUND amber, TRANSITIONAL dim grey. No scoring impact — visual at-a-glance state indicator.

6. Core Signals (1m)

CORE SIGNALS (1m):

ROC(9): 0.115 | Slope: FLAT
 RSI(9): 52.6 | Div: BEARISH
 Volume: 0.3701 BTC (\$28.1K) | vs SMA: 0.16x | SMA: 2.3850 BTC

Three always-scored primitives on the 1m execution timeframe.

ROC(9)

What: 9-period Rate of Change (percentage) plus a classified slope direction.

Calculation: `CalcROCSeries(candles1m, period=9, lookback=cfg.Indicators.ROC.SeriesLookback=3)`. Builds the last `lookback` ROC values; the displayed `r.ROC` is the most recent. `r.ROCSlope` is classified elsewhere by comparing recent ROC values:

- **RISING** — recent ROC series is trending up AND latest > 0
- **FALLING** — recent ROC series is trending down AND latest < 0
- **FLAT** — otherwise

Scoring use:

- **Full long:** `ROC > 0 AND ROCSlope = RISING`
- **Full short:** `ROC < 0 AND ROCSlope = FALLING`
- **Partial long:** `ROC > cfg.Indicators.ROC.SlopeSensitivity(0.1) AND ROCSlope != RISING` — can upgrade in Pass 2 with any Momentum cross-confirm.
- **Partial short:** mirrored.
- **Pass 2c TRENDING input:** active when `|ROC| ≥ SlopeSensitivity`; then aligned if sign matches dominant side.

Interpretation:

- ROC value is read as percent; +0.115 means the 1m close is 0.115% above 9 candles prior.
- **Slope FLAT** with `ROC > 0.1` is the classic “still positive, losing momentum” read — the scoring engine recognises this via the partial path.
- On 1m BTC scalping, `|ROC| > 0.3` is already a strong single-bar impulse; `> 0.6` is extension territory and triggers **TAKE PROFIT** hold status if in-position (default `HoldRocTakeProfitLong = 0.6, Short = -0.6`).
- ROC crossing zero is a structural exit trigger for open positions (Hold Layer 2).
- **Resolution/session-dependent thresholds (v36/v40/v41).** `MagnitudeThreshold` and the slope-classification delta are no longer single global constants. On NY (1-min execution) they stay at the base values (0.1 / 0.05). On Asia/London (3-min execution since v36), `roc_magnitude_threshold` is now **per-session**: ASIA 0.17, LONDON 0.11 (re-baselined v40→v41 by firing-rate-matching to NY’s selectivity — ASIA’s 3-min ROC genuinely runs hotter than LONDON’s). The slope delta is one shared 3-min value, 0.06. Resolved via `ExecutionResolution.ResolveRocMagnitudeForHour / resolution_profiles["3"]`. Read: the same raw ROC(9) number means a different thing depending which session/resolution produced it.

RSI(9)

What: 9-period Wilder RSI plus optional divergence tag.

Calculation: `CalcRSI(candles1m, 9)`. Wilder EMA-smoothed gain/loss ratio, standard formula. **RSIDivergence** is calculated separately by `CalcRSIDivergence` using **pivot-based** detection (not rolling averages):

- Scans the last `LookbackBars=20` bars for structural swing pivots with `PivotWing=2` confirmation bars on each side.
- **BEARISH** fires when price prints a higher high than the pivot AND RSI at that pivot was lower than current RSI — and both deltas clear `DivergencePriceGate=0.001` and `DivergenceRsiDelta=2.0`.
- **BULLISH** is the mirror for lower-low price pivots.
- Returns **NONE** otherwise.

Scoring use:

- **Full long:** `RSI > overbought (60)`
- **Full short:** `RSI < oversold (40)`

- **Partial long:** $RSI > \text{partial_overbought} (55) \text{ AND } RSI \leq 60$ — **dead band 45–55** (v14). Can upgrade in Pass 2.
- **Partial short:** $RSI < \text{partial_oversold} (45) \text{ AND } RSI \geq 40$.
- **Divergence penalty:** BEARISH AND $RSI > \text{DivPenaltyRsiHigh} (65) \rightarrow -1 [L]$. BULLISH AND $RSI < \text{DivPenaltyRsiLow} (35) \rightarrow -1 [S]$.
- **Pass 2c RANGE_BOUND input:** active always; aligned if $RSI > 50$ (for long side) or $RSI < 50$ (short).

Display variants:

- Div: line **appended only** when $\text{RSIDivergence} \neq \text{NONE}$ and not empty. No line means no structural divergence detected in the lookback window.
- Colour: red above 70, green below 30, grey between.

Interpretation:

- The 45–55 dead band is deliberate — RSI near the neutral line provides no directional edge and used to double-count momentum via the old 50/50 partial thresholds. Post-v14, you'll see RSI rows without $[L*]$ / $[S*]$ hits for every small drift off 50.
- A Div: BEARISH tag with $RSI < 65$ is visible but does **not** penalise scoring — the penalty gate is deliberately set above the overbought line to filter noise. If you see Div: BEARISH with RSI 58–64, it's a soft warning to tighten exits, nothing more.
- RSI is the hold manager, not an entry trigger — per trader profile, act on RSI during trade management (> 60 hold, < 40 exit for longs), not for entry.

Volume

What: Current 1m candle volume (BTC), USD notional, ratio to 9-period SMA, and the SMA value itself.

Calculation:

- **CurrentVolume** — `candles1m.Last().Volume` in BTC.
- **CurrentVolumeUSD** — computed elsewhere in `MainForm_Analysis` from current close $\times$ volume. Formatted with M / K / plain suffix per magnitude.
- **VolumeSMA9** — `CalcVolumeSMA(candles1m, 9)` over last 9 candles.
- **VolumeRatio** — $\text{CurrentVolume} / \text{VolumeSMA9}$.

Scoring use (with Dynamic Norms thresholds H and M):

- **Full long:** $\text{VolumeRatio} \geq H \text{ AND } \text{ROC} > 0 \text{ AND } \text{CurrentPrice} > \text{VWAP}$
- **Full short:** $\text{VolumeRatio} \geq H \text{ AND } \text{ROC} < 0 \text{ AND } \text{CurrentPrice} < \text{VWAP}$
- **Partial (mid-tier):** $\text{VolumeRatio} \geq M \text{ AND } \text{VolumeRatio} < H$. Upgrades to full score in Pass 2 only with a Volume-category cross-confirm (OBV).
- No score fires when $\text{VolumeRatio} < M$.

Display colour: green $\geq 1.5x$, red $< 0.7x$, grey otherwise.

Interpretation:

- The USD column is a sanity check against the BTC ratio — 0.16x may sound dead but if BTC is trading at \$76k, the actual notional still matters. On 1m BTC perp, $> \$500K$ notional usually means real participation regardless of the ratio.
- A $\text{VolumeRatio} < 0.7x$ (red) during price movement is a fade warning — price moving without volume rarely sustains. The engine doesn't penalise low volume directly, but the full/partial volume tiers won't fire, so directional signals lose a line of confirmation.
- Remember: the trader-profile 3x rule is subsumed by the Dynamic Norms H threshold. A 0.16x ratio isn't even close to any tier; a 2.5x ratio in NY session ($H \approx 4x$) still won't fire full volume but may qualify mid-tier.

7. VWAP

VWAP (reset 13:30 UTC):

Value: 75995.6 | Dev: 0.056% | Candles: 190
s1 band: [75715.9, 76275.3] | s2 band: [75436.2, 76555.0]

Dual-session auto-anchored VWAP with volume-weighted sigma bands.

Section header

Format: VWAP (reset HH:MM UTC)<[WARMUP]>

- HH:MM — whichever session anchor is currently active. Anchors at `Session1StartHour:Minute` (default 00:00 UTC) and `Session2StartHour:Minute` (default 13:30 UTC). The displayed anchor is the most recent one (the one that reset the VWAP accumulator for the current session).
- [WARMUP] tag appears when `r.VWAPSessionCandles < cfg.Indicators.VWAP.WarmupCandles (15)` — i.e. fewer than 15 candles since the last session anchor.

Warmup behaviour: Scoring of VWAP full/partial is **suppressed** during warmup. The breakdown row still shows the warmup message instead of a signal hit. Cross-category upgrades don't fire off VWAP during this window.

Interpretation: The [WARMUP] tag coming back ~15 minutes after 00:00 or 13:30 UTC is expected behaviour — don't trust VWAP-based scoring rows during that window. Outside warmup, the anchor in the header tells you which session's VWAP is live: 00:00 UTC anchor dominates Asian/European hours, 13:30 UTC anchor dominates US session.

Value / Dev / Candles

What:

- Value — session VWAP (volume-weighted typical price cumulated since anchor).
- Dev — $(\text{CurrentPrice} - \text{VWAP}) / \text{VWAP} \times 100$ as percent. Signed (positive = price above VWAP).
- Candles — `VWAPSessionCandles`, count of 1m candles since the session anchor.

Calculation (`CalcVWAP` in `Indicators_Volatility.vb`):

- Anchor selection: if `UtcNow` is before `Session2` (default 13:30 UTC), anchor = start of UTC day (00:00). Else anchor = 13:30 UTC today.
- Accumulate `typicalPrice × volume` and `volume` from anchor onwards where `typicalPrice = (H + L + C) / 3`.
- $\text{VWAP} = \text{cumTPV} / \text{cumVol}$.

Display colour: Dev shown amber when $|\text{Dev}| > \text{norms.VWAPDevThreshold}$; grey otherwise. (As noted in Dynamic Norms, the threshold is a legacy ref — scoring uses sigma bands, not Dev%.)

Interpretation:

- Dev reads as session “air gap” above/below fair value. On BTC perp, $|\text{Dev}| > 0.3\%$ is meaningful; $> 0.5\%$ is extended and often mean-reverts.
- Candles count lets you verify session integrity — after 13:30 UTC reset you should see it drop back to low single digits then climb. A stuck count (e.g. 200+ after what should have been a reset) means something broke in session detection.

s1 band / s2 band

What: 1-sigma and 2-sigma **volume-weighted** bands around VWAP. Not percentile bands — true weighted standard deviation of `typicalPrice` from VWAP.

Calculation (`CalcVWAPBands`):

- Over the same session candles, accumulate $\text{volume} \times (\text{typicalPrice} - \text{VWAP})^2$ and volume.
- $\text{sigma} = \sqrt{\text{cumWeightedSqDev} / \text{cumVol}}$.
- $s1 = \text{VWAP} \pm \text{sigma}, s2 = \text{VWAP} \pm 2\sigma$.

Scoring use:

- **Full long (Microstructure category):** $\text{VWAP} < \text{Price} \leq s1\text{Upper}$ — price is above VWAP but still within the first band.
- **Full short:** $s1\text{Lower} \leq \text{Price} < \text{VWAP}$.
- **Partial long** (can upgrade via Pass 2 with any Microstructure cross-confirm): $s1\text{Upper} < \text{Price} \leq s2\text{Upper}$.
- **Partial short:** $s2\text{Lower} \leq \text{Price} < s1\text{Lower}$.
- No score above $s2\text{Upper}$ or below $s2\text{Lower}$ — price is too extended for a fresh-direction signal.
- **Pass 2c RANGE_BOUND input:** active only when not in warmup; then aligned if price above VWAP (long) or below (short).

Interpretation:

- Price sitting just above VWAP with $\text{Dev} < 0.1\%$ typically means the $s1$ band contains it (full long score). This is the “clean mean-reversion long” posture.
- Between $s1$ and $s2$ is “leaning extended” — the engine requires cross-confirmation before awarding the score, hence the partial-upgrade gate.
- Beyond $s2$ is where you’d look for **short against** the extension (if you’re a mean-reverter) or **cut exposure** (if you’re running with the trend). The engine goes silent either way — by design, it doesn’t pick a side in stretched tails.
- A price in the middle of wide $s1 / s2$ bands implies high realised intraday volatility; tight bands imply consolidation. The width is itself information, even though not scored directly.

8. BBW / TTM Squeeze

BBW / TTM SQUEEZE:

BBW: 0.004 | Status: NONE

TTM: Histogram=70.26 Dir=FALLING Signal=BULL_FADING

Bollinger Band Width (compression detector) plus TTM Squeeze Momentum (directional momentum oscillator).

BBW

What: Bollinger Band Width (normalised), and a classified squeeze status.

Calculation (CalcBBW in Indicators_Volatility.vb):

- Rolling 20-period Bollinger Band over the last $\text{period} * 5 = 100$ candles.
- $\text{BBW} = (\text{upper} - \text{lower}) / \text{mid}$, where bands are $\text{mid} \pm \text{stdMult} * \text{stdDev}$ and mid is the 20-period SMA of closes.
- $r.\text{BBW}$ = most recent value in the series.

Squeeze threshold (v0.48):

- $\text{threshold} = 20\text{th percentile of the rolling BBW series (not min} * 1.5)$.
- **SqueezeStatus:**
 - **ACTIVE** — current $\text{BBW} \leq \text{threshold}$ (in the bottom 20% of recent volatility)
 - **RELEASING** — previous BBW was $\leq \text{threshold}$, current is above (just broke out of squeeze)
 - **NONE** — neither

Possible values: BBW typically 0.001–0.050 on 1m BTC (normalised). Status one of the three strings above.

TTM

What: Linear-regression momentum histogram, its slope direction, and a four-state signal combining the two.

Calculation (CalcTTMSqueeze):

- For each of the last `linRegPeriod=7` candles, compute `close - SMA20(close)`. This is the histogram input series.
- Fit a least-squares linear regression through these 7 values; `Histogram = intercept + slope × (n-1)` — the regression's rightmost (latest) projected value.
- **Direction** classified by comparing first-third-mean vs last-third-mean of the delta series:
 - `delta > flat_threshold (0.5) → RISING`
 - `delta < -flat_threshold → FALLING`
 - Else → FLAT
- **Signal** cross-tabulated:

Histogram sign	Direction	Signal
> 0	RISING	BULL_BUILDING
> 0	FALLING	BULL_FADING
< 0	FALLING	BEAR_BUILDING
< 0	RISING	BEAR_FADING
any	FLAT	FLAT

Combined scoring logic (BBW × TTM)

Implemented in `ScoringEngine_Calculate_Scoring.vb` as a single Select on `SqueezeStatus`:

SqueezeStatus	TTM Signal	Scoring effect
ACTIVE	any	<code>LongScore -= BbwSqueezePenalty (2)</code> AND <code>ShortScore -= 2</code> . Both sides penalised during compression.
RELEASING / NONE	BULL_BUILDING	<code>LongScore += 1</code> (Microstructure category). Shown as [L] in breakdown.
RELEASING / NONE	BEAR_BUILDING	<code>ShortScore += 1</code> (Microstructure). Shown as [S].
RELEASING / NONE	BULL_FADING / BEAR_FADING / FLAT	No score change, no penalty — breakdown row reads -- no award.

Display

- **BBW line** — value (3-decimal) and status. Colour is an intended `C_WARN` when status is `SQUEEZE`, but the code checks the literal string "SQUEEZE" which is never emitted — the actual emitted values are `ACTIVE/RELEASING/NONE`, so the BBW line always renders grey. (Cosmetic no-op; scoring is unaffected.)
- **TTM line** — histogram (2-decimal), direction, signal. Colour intended to be green/red on direction UP/DOWN, but the code emits `RISING/FALLING`, so the TTM line also always renders grey. (Same dead colour path.)

Interpretation

- **ACTIVE squeeze** is the engine's compression detector — both sides are penalised because direction is undefined during compression. Don't force entries into an `ACTIVE` squeeze; wait for the release.

- **RELEASING status + BULL_BUILDING signal** is the highest-quality compression-to-expansion long trigger the engine expresses. In the scoring table this shows as a single [L] on the BBW/TTM row, but its predictive value is higher than a generic +1 — volatility expansion *with direction* is the core setup the trader profile wants to catch.
- **BULL_FADING / BEAR_FADING** are inflection warnings: histogram is still on one side of zero, but direction is rolling over. The engine awards nothing, but the **CONTEXT: MOMENTUM_FADING** tag uses **TTMSignal** $\in$ {**BULL_FADING**, **BEAR_FADING**} as one of its three fading-count inputs. So even though TTM didn't score, it can still flip the context tag.
- **Histogram magnitude** (unscored) is worth eyeballing: $|\text{Histogram}| < 5$ on a **FADING** signal means the momentum wave is almost exhausted. Large histograms ($|\text{Histogram}| > 50$) with **BUILDING** direction means a real directional wave is under construction.
- The 20th-percentile squeeze threshold means **ACTIVE** fires genuinely into the tail of recent realised volatility — not on any session-low spike. Expect it to appear during genuine consolidation phases, typically between two trending segments.

9. EMA Ribbon

EMA RIBBON (1m):

9: 76048.6 | 21: 76007.3 | 50: 75946.6 | Align: BULL
5m EMA200: 76040.4 | Price: BELOW

Two-layer EMA view: 9/21/50 ribbon on 1m as the dynamic trend structure, plus 5m EMA(200) as the higher-timeframe anchor.

EMA 9 / 21 / 50 (1m)

What: Three exponential moving averages on the 1m execution timeframe and their alignment label.

Calculation: `CalcEMA(candles1m, period)` in `Indicators_Momentum.vb`. Standard EMA: seed = SMA of first period closes, then $\text{ema} = \text{close} \times k + \text{ema} \times (1 - k)$ where $k = 2 / (\text{period} + 1)$. Periods 9, 21, 50 sourced from `cfg.Indicators.EMA.{Fast,Mid,Slow}`.

EMAAlignment is classified by strict ordering:

Condition	Alignment
$\text{EMA9} > \text{EMA21} > \text{EMA50}$	BULL
$\text{EMA9} < \text{EMA21} < \text{EMA50}$	BEAR
Anything else	MIXED

Scoring use:

- **Full long (MarketStructure category):** **EMAAlignment** = BULL $\rightarrow$ +1 **LongScore**.
- **Full short:** **EMAAlignment** = BEAR $\rightarrow$ +1 **ShortScore**.
- **MIXED:** no score.
- **Pass 2c TRENDING input:** always active; aligned if alignment matches dominant side.
- **Proposed direction for MTF Gate:** BULL proposes LONG, BEAR proposes SHORT (if regime is also not countering) — feeds into `CalcMTFGate` pre-check.

Display colour: green on BULL, red on BEAR, amber on MIXED.

Interpretation:

- The ribbon is the most influential single scoring primitive in **TRENDING** regimes — both a direct +1 and a Pass 2c alignment vote.

- **MIXED** is a legitimate “standing-aside” signal. The most common cause is EMA9 crossing but EMA21 still ahead of EMA50 (or vice versa) — a ribbon in transition. No vote until the three lines reclaim monotonic order.
- Ribbon spread (EMA9 – EMA50) implies trend strength. A **BULL** alignment where the three EMAs are within a few dollars of each other is a weak ribbon about to churn into **MIXED**; a wide spread is a mature trend.
- The ribbon is slow — it will mis-fire during sharp reversals that haven’t yet propagated into EMA50. Cross-check against EMA200(5m) for whether the reversal is local noise or a higher-timeframe event.

5m EMA(200)

What: 200-period EMA on the 5m candles; regime anchor.

Calculation: `CalcEMA(candles5m, 200)`. Requires ≥ 200 candles of 5m history (~17 hours). Returns 0 when insufficient data — `PriceVsEMA200` shows N/A in that case.

`PriceVsEMA200` classification:

Condition	Label
<code>CurrentPrice > EMA200_5m</code> (and <code>EMA200_5m > 0</code>)	ABOVE
<code>CurrentPrice < EMA200_5m</code> (and <code>EMA200_5m > 0</code>)	BELOW
<code>EMA200_5m = 0</code>	N/A

Scoring use:

- **Full long (MarketStructure):** ABOVE → +1 LongScore.
- **Full short:** BELOW → +1 ShortScore.
- **N/A:** no score (cold start).

Display colour: green ABOVE, red BELOW.

Interpretation:

- This is a one-way regime veto in practice — price below the 5m EMA200 tilts the scoring towards short, above towards long, regardless of short-term 1m structure. The sample output shows the common conflict case: 1m ribbon **BULL** (+1 Long) but 5m EMA200 **BELOW** (+1 Short). That’s the engine expressing “local long, higher-TF short” — healthy disagreement that gets resolved by the rest of the scoring stack.
- A price within a few dollars of EMA200 (like the sample’s **76038** vs **76040**) is structurally on the line — a single 1m candle could flip the classification. Treat borderline EMA200 reads as noise; wait for decisive separation.
- **N/A** appearing after session start usually means the 5m candle fetch didn’t return enough history. Engine is flying with one side of the scoring disabled. Re-run once data’s caught up.

10. Market Structure

MARKET STRUCTURE:

```
Donchian(20): Upper=76210.5 Lower=75868.0 | Signal: NONE
OBV: Trend=RISING | Div=BULLISH
VPFR-lite: POC:75911.6 | NEAR_HVN_RESIST | HVN@POC:YES
```

Three structural primitives: breakout levels (Donchian), volume-trend confirmation (OBV), and session volume profile (VPFR-lite).

Donchian(20)

What: 20-period high/low channel plus a quartile-aware breakout signal.

Calculation: CalcDonchian returns `Upper = max(high) / Lower = min(low)` over the last 20 candles. DonchianSignal is set downstream (not in CalcDonchian) by comparing `CurrentPrice` to the channel and its quartiles:

- Price at or above `Upper` → LONG
- Price in upper quartile (below `Upper`, above `Upper - 0.25 × range`) → LONG_PARTIAL
- Price at or below `Lower` → SHORT
- Price in lower quartile (above `Lower`, below `Lower + 0.25 × range`) → SHORT_PARTIAL
- Mid-channel (middle half) → NONE

Scoring use:

- **Full long / short (MarketStructure):** LONG / SHORT → +1.
- **Partial:** LONG_PARTIAL / SHORT_PARTIAL — can upgrade in Pass 2 with any MarketStructure cross-confirm.
- NONE: no score, no partial. Breakdown row note reads MID-CHANNEL -- no signal.
- **Pass 2c RANGE_BOUND input:** aligned if signal matches dominant side (accepts both LONG and LONG_PARTIAL as “long-aligned”, same for short).

Display colour: green on LONG, red on SHORT, grey otherwise (partial signals display grey; the [L] / [S] hit still shows in breakdown).

Interpretation:

- Full LONG / SHORT = price just printed a 20-minute extreme. Genuine breakout that warrants cross-confirm from order flow and volume before acting.
- LONG_PARTIAL (upper quartile) is “pressing the highs” — the right bid environment for a breakout but the level hasn’t broken yet. The partial-upgrade path makes this score only when other MarketStructure signals (EMA ribbon, DMI, ADX, 5m EMA200, VPFR) agree.
- NONE in the middle half is silent by design — mid-channel price has no breakout edge. Don’t read absence as disconfirmation.
- Width of the channel (`Upper - Lower`) is an implicit volatility proxy separate from ATR; use it to sanity-check the ATR-based stop distances.

OBV

What: On-Balance Volume trend direction plus price/OBV divergence state.

Calculation (CalcOBV in Indicators_Structure.vb):

- OBV accumulated across all supplied candles: +volume on up-close, -volume on down-close, no change on flat.
- `obvChange = (obvLast - obvFirst) / |obvFirst|` if non-zero, else 0.
- OBVTrend:
 - `obvChange > trend_gate (0.001)` → RISING
 - `obvChange < -trend_gate` → FALLING
 - Else → FLAT
- OBVDivergence (only when `|priceChange| ≥ divergence_gate (0.001)`):
 - Price up + OBV down → BEARISH
 - Price down + OBV up → BULLISH
 - Else / price change below gate → NONE

Scoring use:

- **Full long (Volume category):** OBVTrend = RISING AND OBVDivergence != BEARISH → +1.
- **Full short:** OBVTrend = FALLING AND OBVDivergence != BULLISH → +1.

- **Partial long:** `OBVTrend = RISING AND OBVDivergence = BEARISH` — **adverse divergence blocks cross-category upgrade**. The partial does NOT promote to full in Pass 2 even with a Volume cross-confirm. Breakdown row appends `[upgrade blocked]`.
- **Partial short:** mirrored (`FALLING + BULLISH` blocks upgrade).

Display colour: always grey (single colour path — no conditional). Signal state is legible from the `Trend=` and `Div=` strings.

Interpretation:

- OBV aligned with its own trend (no divergence) is a clean Volume-category confirm — the +1 is secondary to the Volume indicator itself but still contributes to any Pass 2 upgrade targeting the Volume category.
- The divergence-blocks-upgrade gate exists for a reason documented in v0.42: OBV disagreeing with its own price trend means volume is not backing the move. Awarding the upgrade would reward a weak signal. Expect to see `Trend:RISING Div:BEARISH | [upgrade blocked]` during late-stage rallies where new highs are printed on fading participation.
- In the sample, `Trend=RISING Div=BULLISH` — trend and divergence pointing the same direction (bullish tilt). This fires `obvLong` normally and does NOT block upgrades.

VPFR-lite v2 (POC + VAH/VAL + nearest HVN/LVN)

What: Volume Profile Fixed Range over the available 1m window, with exponential decay weighting toward recent bars. Reports POC price, value-area boundaries (VAH / VAL), nearest HVN/LVN walls above and below, HVN-near-POC flag, and a signal classification.

v2 additions over v1: value-area boundaries (`VPFRVAH / VPFRVAL`) capture the 70%-volume zone; `VPFRNearHvnAbove / VPFRNearestHvnBelow` are the closest high-volume walls in each direction. v2 fields drive the Step 5b 3-tier target cap (swing → nearest HVN → POC).

v1 (legacy): Volume Profile Fixed Range over the available 1m window, with exponential decay weighting toward recent bars. Reports POC price, HVN-near-POC flag, and a signal classification.

Calculation (CalcVPFRLite in Indicators_Structure.vb):

- Price range: `[min(low), max(high)]` across supplied candles. Split into `numBuckets=50` equal-width buckets.
- Volume per bucket: each candle contributes `volume × decayBase^age` where `decayBase=0.985` and `age = (candleCount - 1 - candleIndex)`. Most recent candle has weight 1.0; weight falls ~22% per 15 bars.
- POC = midpoint of bucket with the highest accumulated weighted volume.
- `HVNNearPoc = |CurrentPrice - POC| / POC ≤ hvnProximityPct (0.002)` — i.e. within ±0.2% of POC.
- HVN threshold: `pocVol × hvnVolPct (0.6)`. LVN threshold: `pocVol × lvnVolPct (0.2)`.
- Current bucket's weighted volume → `curBucketVol`.

VPFRSignal classification:

Condition	Signal
<code>HVNNearPoc AND CurrentPrice < POC</code>	<code>NEAR_HVN_SUPPORT</code>
<code>HVNNearPoc AND CurrentPrice ≥ POC</code>	<code>NEAR_HVN_RESIST</code>
<code>!HVNNearPoc AND curBucketVol ≤ LVN threshold AND CurrentPrice > POC</code>	<code>IN_LVN_BULL</code>
<code>!HVNNearPoc AND curBucketVol ≤ LVN threshold AND CurrentPrice < POC</code>	<code>IN_LVN_BEAR</code>
Otherwise	<code>NEUTRAL</code>

Scoring use:

- **Full long (MarketStructure):** NEAR_HVN_SUPPORT OR IN_LVN_BULL → +1 LongScore.
- **Full short:** NEAR_HVN_RESIST OR IN_LVN_BEAR → +1 ShortScore.
- NEUTRAL: no score.
- **Step 5b ATR target cap** (separate from scoring): when NEAR_HVN_RESIST or IN_LVN_BEAR AND POC > CurrentPrice AND POC < rawLongTarget → long target capped at POC. Mirrored for NEAR_HVN_SUPPORT / IN_LVN_BULL on short side.

Display colour: green for support / LVN-bull, red for resist / LVN-bear, dim for neutral. HVN@POC shows YES / NO.

Interpretation:

- NEAR_HVN_RESIST reads as “price is sitting just below a high-volume node acting as resistance” — structurally bearish. If you see it on a breakout attempt, expect rejection. The engine turns this into a short signal + a long-target cap.
- IN_LVN_BULL / IN_LVN_BEAR fire when price is through a low-volume node — these are the “price moves fast through empty air” zones. Directional score only fires when price is on the “correct” side of POC relative to the vacuum. A NEUTRAL signal can mean either mid-profile price or normal-density bucket; read it as “no structural information” rather than “balanced”.
- POC itself is a useful reference even when the signal is NEUTRAL — it’s the volume centre of gravity for the current session window. Traders often watch POC cross as a separate context cue.
- Exponential decay means POC will drift with recent participation rather than anchor on early-session high-volume prints. If you see POC shifting alongside price through the session, that’s normal; if POC is static while price diverges, older volume is dominating — the signal is lagging actual structure.
- VAH/VAL define the 70%-volume zone. Price inside the value area is “accepted”; price outside is “rejected” or “extended”. v2’s value-area boundaries don’t currently feed scoring directly, but VPFRVAH / VPFRVAL are logged to CSV columns 72/73 for later auto-tweaker tuning.
- **Nearest HVN walls** (VPFRNearestHvnAbove / VPFRNearestHvnBelow) are the closest high-volume nodes in each direction. They feed the Step 5b 3-tier target cap as the second-priority cap (after swing target, before POC).

Swing Pivots (5m + 15m)

What: Most recent confirmed swing high and swing low on 5m and 15m candles, plus direction-aware target/stop bookkeeping for the long and short side.

Calculation (CalcSwingPivots in Indicators_Structure.vb):

- Walk backward from scanEnd = candleCount - pivotWing - 1 to scanStart = pivotWing looking for confirmed pivots.
- A bar at index i is a confirmed swing high iff high[i] > high[i±k] for all k ∈ [1, pivotWing] (strict inequality on both sides — equal-high bars don’t count).
- Mirror logic for swing low.
- First match wins — emits the most recent confirmed pivot, not the highest/lowest in the window.
- 5m: pivotWing = cfg.Indicators.Swing.PivotWing5m (default 3), lookback = LookbackBars5m (default 30).
- 15m: separate config keys (PivotWing15m, LookbackBars15m) — narrower wing acceptable due to slower bar pace.

Direction-aware bookkeeping (computed inline in MainForm_Analysis.RunAnalysisAsync, not in CalcSwingPivots):

- SwingTargetLong = LastSwingHigh5m if it’s above current price, else 0.
- SwingStopLong = LastSwingLow5m if it’s below current price, else 0.
- Mirror for short side.

Scoring use:

- **Step 5b 3-tier cap (top priority):** when a long verdict has `SwingTargetLong > 0` and `SwingTargetLong < rawLongTarget`, the target is capped to the swing high. Cap is closest-wins across (swing → nearest HVN → POC) — the tightest candidate wins. `TargetCapReason` records which tier won.
- **CalcHoldStatus Layer 1.5 (structural break exit):** during a long position, if `CurrentPrice < SwingStopLong - 0.5 × cfg.Indicators.Structure.SwingBreachAtrMult × ATR`, fires EXIT -- structural break (swing low breach). Sits between Layer 1 (microstructure exit) and Layer 2 (OBV divergence).
- **VerdictContext STRUCTURALLY_WEAK:** when `LastSwingHigh5m > 0` OR `LastSwingLow5m > 0` AND no clean target+stop pair can be placed for the verdict direction, fires the tag. Catches the “we have structure but the trade doesn’t have R:R” case.

Display: Renders under MARKET STRUCTURE as Last Swing High 5m: 102450.0 | Last Swing Low 5m: 101800.0 with corresponding 15m row dim. ATR Entry block includes a structural row showing LONG Stop: <swing low> Entry: <price> Target: <swing high> R:R <ratio> in cyan when both sides exist.

Interpretation:

- Confirmed pivots avoid equal-high false positives. A new swing high at the same level as the prior one does not register, which is the right behaviour — true breakout structure requires a strictly higher high.
- The walk-backward scan picks the most-recent pivot rather than scanning forward from the start. This means the engine always references the freshest actionable level, not the oldest historical reference in the lookback.
- 15m pivots are display-only context. They are not used for cap arbitration or structural break detection — only the 5m levels drive those.
- A STRUCTURALLY_WEAK tag with 0, 0 in `SwingTargetLong/SwingStopLong` is the diagnostic case: enough candle history exists for at least one swing detection, but the geometry doesn’t produce a clean R:R for the verdict direction.

Trend Structure (HH/HL/LH/LL)

What: Sequence-of-pivots classification on the 5m timeframe — UPTREND / DOWNTREND / EXPANSION / CONTRACTION / UNDEFINED.

Calculation (ClassifyTrendStructure in Indicators_Structure.vb):

- Walk backward through `candles5m` to identify the last `cfg.Indicators.TrendStructure.PivotCount` (default 6) confirmed pivots, each requiring `cfg.Indicators.TrendStructure.PivotWing` (default 3) confirmation bars on each side. Mix of highs and lows.
- Need at least 2 highs AND 2 lows in the result. If fewer (window too short or chop), return UNDEFINED.
- Compare the most recent two highs: HH if newer > older; LH otherwise.
- Compare the most recent two lows: HL if newer > older; LL otherwise.
- Map to enum:
 - HH + HL → UPTREND
 - LH + LL → DOWNTREND
 - HH + LL → EXPANSION
 - LH + HL → CONTRACTION

Pure function. Returned as `r.TrendStructure`. Two display tuples (`LastTwoHighs5m`, `LastTwoLows5m`) carry the four prices used in the comparison.

Scoring use (Pass 2c, after TRENDING / RANGE_BOUND alignment scoring, before snapshot for funding modifiers):

If `cfg.RegimeWeights.Enabled` AND `cfg.Indicators.TrendStructure.Enabled`:

```

UPTREND   AND LongScore > ShortScore → LongScore += structure_bonus (capped at regimeMax)
DOWNTREND AND ShortScore > LongScore → ShortScore += structure_bonus (capped at regimeMax)
EXPANSION → no score change (display caution flag)

```

CONTRACTION → no score change (range-narrowing context)
UNDEFINED → no score change

structure_bonus = cfg.Indicators.TrendStructure.StructureBonus (default 1). Suppressed in TRANSITIONAL regime (consistent with rest of Pass 2c). Bonus only applies when structure agrees with the dominant side — never both, never opposite.

Display: Renders under MARKET STRUCTURE as:

Trend Structure : UPTREND (HH 102450.0 > 102100.0 | HL 101800.0 > 101500.0)

Colours: UPTREND green, DOWNTREND red, EXPANSION amber, CONTRACTION dim cyan, UNDEFINED dim grey.

CSV column 87: TrendStructure5m — string enum value. Logged per row.

Interpretation:

- Trend Structure is structure-on-pivots, distinct from EMA / ROC / CVD which are price-momentum signals. The Pass 2c bonus is deliberately separate from the existing regime-alignment bonus to avoid double-counting.
- EXPANSION is the most informative non-scoring state — both higher highs AND lower lows indicate the market is widening its range. Often precedes a regime shift or volatile rejection of a recent breakout. Treat as a caution flag for fresh entries.
- CONTRACTION is range-narrowing, often paired with low BBW. The next decisive move is usually directional but the structure alone doesn't tell you which way.
- UNDEFINED typically appears in the first ~30 minutes of a session before enough confirmed pivots accumulate, or in deep chop where the wing-strict-inequality test fails. Don't read it as bearish or bullish.

Best Volume Pivot (Display-Only)

What: The pivot in the 5m lookback with the highest total wing-window volume, reported alongside its volume ratio against the average pivot in the same lookback. Display-only in v1 — no scoring or cap-arbitration impact.

Calculation (extension of CalcSwingPivots):

- For each confirmed pivot found in the lookback, sum the volume of all bars in [pivotIdx - pivotWing, pivotIdx + pivotWing] — total volume across the wing window.
- Track the pivot with the highest total wing-window volume (BestPivotByVolume5m).
- Compute average wing-window volume across all pivots in lookback (avgPivotVolume).
- BestPivotVolumeRatio5m = best / avgPivotVolume.
- BestPivotIsHigh5m = True if best pivot is a swing high; False if a low.
- If fewer than 2 confirmed pivots in lookback, all three fields return 0 / 0 / False.

Scoring use: none in v1. Logged for offline analysis and CalibrationReport BEST VOLUME PIVOT DISTRIBUTION section.

v2 promotion condition (parked in DeribitIndicatorProject.md §16.6 P1): if CalibrationReport shows “best is also most-recent” rate falls below 50% AND auto-tweaker output shows volume-weighted pivots correlate with subsequent target-hit rate, promote to a 4th cap tier (best-volume-swing > most-recent-swing > nearest HVN > POC).

Measurable since v63 — still not live. That promotion can now be *tested* without being shipped. scoring.structural_levels.use_best_pivot_candidate (Boolean) inserts this pivot into the placed-target candidate set as the first tier above swing (§2). It ships false, so nothing here changes and the engine still places no level from this pivot — the rows above stay display-only. Turning it on is a backtest exercise (§26, field “Use best-pivot candidate”), not a live settings change.

Display: Renders under MARKET STRUCTURE below the swing pivot rows:

Best Vol Pivot 5m: HIGH 102450.0 (vol×2.3 vs avg pivot)

Colour: dim cyan (informational, not actionable in v1).

CSV columns 85–86: BestPivotByVolume5m (price) and BestPivotVolumeRatio5m (ratio). Reserved in v0.4 schema; populated when D2 ships.

Interpretation:

- Ratios above 2.0× indicate a meaningfully stronger reference level than the average pivot. Under 1.5× the volume-weighting is barely differentiating from the most-recent pivot.
- When BestPivotByVolume5m differs from LastSwingHigh5m (or low), there's a stronger pivot back in the lookback than the most-recent one. Eyeball whether your structural target/stop should reference it instead.
- This is a chart-reading aid that exposes the volume-quality of the pivots being used elsewhere — not a fresh trade signal.

11. Open Interest

OPEN INTEREST:

OI: 1036558700 | d15m: 0.000% | d60m: 0.000% | Signal: NEUTRAL

Aggregate BTC-PERPETUAL open interest plus 15m / 60m percentage changes and a directional classifier.

OI / d15m / d60m

What:

- OI — current absolute OI value from Deribit `get_book_summary_by_instrument` (r.OI_Current).
- d15m — percentage change of OI vs the snapshot ~15 minutes ago.
- d60m — percentage change vs ~60 minutes ago.

Calculation: In `MainForm_Analysis.vb` using an `OiSnapshot` ring buffer keyed by timestamp (`_oiHistory`). Each run appends (`nowTs`, `OI_Current`). When computing deltas, looks up the stored snapshot closest to `nowTs - 15m` / `nowTs - 60m`. If no matching snapshot exists (cold start / recent restart), delta reads 0.000%.

Interpretation:

- On a fresh session, d15m and d60m both read 0.000% for the first ~15 and ~60 minutes respectively — this is warmup, not a real flat OI reading. The signal classifier will hold at NEUTRAL during this window.
- Absolute OI value is useful for magnitude context but doesn't directly score — the engine only reads the deltas.
- BTC-PERPETUAL OI typically swings ±0.1% to ±1.0% over 15 minutes in active sessions. Anything beyond ±1% in 15m is a meaningful positioning event worth eyeballing.

Signal

What: Directional classification of the OI change in combination with price direction.

Calculation: Derived from d15m / d60m against `cfg.Indicators.OI.NeutralBandPct (0.05)`, `ChangeThresholdPct (0.01)` cross-referenced with recent price direction. Produces one of five states.

Possible values:

Signal	Meaning	Condition (approximate)
NEW LONGS	Fresh long positioning — OI up + price up	OI rising past threshold AND price rising

Signal	Meaning	Condition (approximate)
NEW SHORTS	Fresh short positioning — OI up + price down	OI rising past threshold AND price falling
COVERING	Shorts covering — OI down + price up	OI falling past threshold AND price rising
CAPITULATION	Longs capitulating — OI down + price down	OI falling past threshold AND price falling
NEUTRAL	OI change within neutral band OR during warmup	Default / no qualifying move

Scoring use:

- **Full long (Microstructure):** NEW LONGS → +1 LongScore.
- **Full short:** NEW SHORTS → +1 ShortScore.
- **Partial long:** COVERING — upgradeable in Pass 2 with any Microstructure cross-confirm.
- **Partial short:** CAPITULATION — upgradeable similarly.
- **Pass 2b OI × CVD cross-confirm:** Full NEW LONGS + bullish CVD slope & value → +UpgradeBonus (1) long. Full + opposing CVD → -ConflictPenalty (1) long. Mirror for NEW SHORTS. Upgraded COVERING / CAPITULATION (after Pass 2 upgrade) can *confirm* but cannot trigger a conflict penalty — by design, to avoid double-penalising short-covering or capitulation transitions.

Display colour:

- Green: NEW LONGS, COVERING
- Red: NEW SHORTS, CAPITULATION
- Grey: NEUTRAL

Interpretation:

- NEW LONGS + CVD RISING is the highest-quality Pass 2b confirmation — genuine new long participation, confirmed by aggressor flow. +1 raw score + +1 Pass 2b bonus.
- NEW LONGS with CVD FALLING is the conflict case — OI building but sell-side aggression. Often means leveraged positions being built into selling, which historically unwinds violently. -1 Pass 2b penalty is deliberate.
- COVERING is mechanically bullish (shorts exiting pushes price up) but tactically weak — the buying is forced, not directional. The partial-only treatment reflects this: it can help but doesn't score standalone.
- CAPITULATION is mechanically bearish but often marks exhaustion. Partial-only treatment again: the engine won't short capitulation outright, only with cross-confirmation.
- During cold-start warmup ($d15m = 0.000\%$), Signal: NEUTRAL is structural — wait for the ring buffer to fill before trusting OI signals.

12. Order Flow

ORDER FLOW:

```

Spread: 1.4 bps | TIGHT
OFI Ratio: 10.08 | Bid Vol: 583380 | Ask Vol: 57900 | BUY DOMINANT | Momentum: RISING
CVD:      Net:182100 | Slope:RISING | Div:NONE
TFI:      0.606 | BUY PRESSURE
MicroCVD: E:2840 M:15250 L:-14720 | DECELERATING | BULL_DECEL

```

Five order-flow primitives, each sampling a different depth / aggressor / temporal segment.

Bid-Ask Spread (SpreadBps)

What: Bid-ask spread on the L2 order book snapshot, measured in basis points. Acts as an entry-side gate — wide spread fires a verdict-side penalty.

Calculation (inline in MainForm_Analysis.RunAnalysisAsync):

```
mid = (bestBid + bestAsk) / 2
SpreadBps = (bestAsk - bestBid) / mid × 10000
```

Computed once per run from the same GetOrderBookAsync snapshot used by OFI.

Classification (v17 thresholds):

Condition	Class
SpreadBps ≤ cfg.Indicators.Spread.TightThresholdBps (default 3.0)	TIGHT
cfg.Indicators.Spread.TightThresholdBps < SpreadBps ≤ cfg.Indicators.Spread.WideThresholdBps (default 5.0)	NORMAL
SpreadBps > cfg.Indicators.Spread.WideThresholdBps	WIDE

Scoring use:

- **WIDE penalty:** LongScore -= cfg.Indicators.Spread.WidePenalty (default 1) if a long verdict is dominant; mirrored for short. Penalty is dominant-side only — it does not penalise both sides nor flip directional bias.
- **TIGHT and NORMAL:** no scoring effect.

Display colour: green TIGHT, dim NORMAL, amber bold WIDE.

CSV column 69: SpreadBps — Double, 4dp.

Interpretation:

- The order book is already fetched for OFI; SpreadBps is a near-zero-cost microstructure read. The penalty exists to catch the “looks like a great breakout but the book is actually empty” trap during flush events.
- Normal BTC-PERPETUAL spread on Deribit sits 1–3 bps. Above 5 bps usually indicates an active flush / news event / deep order book withdrawal — not a clean entry environment.
- The penalty does not block a trade — at most it degrades the verdict tier by one (e.g., STRONG → MEDIUM). If the underlying score is high enough to absorb the penalty, the verdict still fires.
- During trending spread expansion (the spread widens because volatility is genuinely high but the book is not flushing), expect to see WIDE more often. The penalty is conservative — accept the false-negatives during high-vol regimes.

OFI (Order Flow Imbalance)

What: Book-depth-weighted bid/ask volume imbalance ratio from the L2 snapshot.

Calculation (CalcOFI in Indicators_OrderFlow.vb):

- Take the top book_depth (5) bid and ask levels from the order book snapshot.
- Build descending weights {5, 4, 3, 2, 1} — nearest level highest weight.
- bidVol = Σ(bid.Size × weight) over the 5 levels; askVol = Σ(ask.Size × weight).

- $OFIRatio = bidVol / askVol$.
- Classification (v14 thresholds):
 - $OFIRatio > buy_dominant_ratio$ (2.0) → BUY DOMINANT
 - $OFIRatio < sell_dominant_ratio$ (0.5) → SELL DOMINANT
 - Else → BALANCED

Scoring use:

- **Full long (Microstructure):** BUY DOMINANT → +1 LongScore.
- **Full short:** SELL DOMINANT → +1 ShortScore.
- BALANCED: no score.

Display colour: green when ratio > 1.2, red < 0.8, grey between. (Note: the visual threshold is stricter than the scoring threshold — so you can see a green ratio that hasn't yet cleared BUY DOMINANT. The BUY DOMINANT label is authoritative for scoring.)

Interpretation:

- 5-level depth means OFI is reading the *visible* book — it won't catch hidden / iceberg liquidity, but does catch the visible tilt that most liquidity-taking participants see.
- A ratio > 5x like the sample's 10.08 is extreme — usually the result of one side pulling liquidity (thin ask book, not necessarily heavy bid book). Cross-check the Bid Vol and Ask Vol absolute values: if the low side is very small (e.g. 57900 vs 583380), it's more "ask pulled" than "bid stacked".
- The v14 relaxation from 3.0 / 0.333 to 2.0 / 0.5 means OFI now fires earlier. In practice this makes OFI contribute a signal in more of the "meaningful tilt" range where the old thresholds stayed silent.
- OFI is a leading indicator — imbalance visible before price moves. Since the v42 WebSocket cutover the engine reads a live book stream rather than REST snapshots, so this concern is largely retired; REST fallback still applies on a degraded WS connection.

Time-averaged OFI (v46)

What changed: On the live WS path, `r.OFIRatio` is no longer a single book snapshot — it's a **time-weighted average** of the top-book imbalance over the run window (`indicators.OFI.avg_window_sec`, default 10s), folded continuously by a feed-side accumulator (`Core/OfiAccumulator.vb`, a time-aware EMA with $\alpha = 1 - \exp(-dt/\tau)$). A transient sweep or spoof can no longer flip the ratio for a whole run — the signal now reflects *sustained* imbalance.

- Controlled by `indicators.OFI.averaging_enabled` (default true). `false` reverts to snapshot OFI (the v45 behaviour), byte-identical by construction.
- Falls back to a single REST/WS snapshot automatically until the accumulator has warmed up ($\geq avg_window_sec$ of fold coverage), and on every reconnect (the accumulator resets so a stale pre-gap average can't bleed across).
- At `network.transport=rest`, or on any per-run REST fallback, OFI is always the snapshot calculation — averaging is WS-only.
- The classification thresholds (`BuyDominantRatio` 2.0 / `SellDominantRatio` 0.5) and the OFI Momentum ring buffer are unchanged in mechanism — only the value feeding them shifted from snapshot to average.
- **Cosmetic note:** the OFI row shows `ratio · bid · ask`. Because the ratio is now an *average of ratios* rather than a ratio computed from *averaged* bid/ask volumes, the displayed Bid Vol / Ask Vol (still single-snapshot) won't always divide out to exactly the displayed Ratio. This is by design, not a data error.
- The dominance-ratio thresholds (2.0 / 0.5) themselves are tuned for the old snapshot signal and have **not yet been re-baselined** for the averaged signal — that's a flagged future data-gated pass (mirrors the v36→v40 ROC re-baseline sequence).

OFI Momentum

What: Direction of OFI signal change over the last 3 samples. RISING / FALLING / FLAT. Computed against a 10-sample ring buffer (`_ofiHistory` in `MainForm_Layout`).

Calculation (`CalcOFIMomentum` in `Indicators_OrderFlow.vb`):

- Buffer holds the last 10 numerical OFI signal codes (BUY_DOMINANT = +2, BUY_LEAN = +1, NEUTRAL = 0, SELL_LEAN = -1, SELL_DOMINANT = -2).
- Compare mean of last `cfg.Indicators.OFI.MomentumWindow` (default 3) entries against the entry `MomentumWindow + 1` positions back.
- If $\text{delta} \geq \text{cfg.Indicators.OFI.MomentumThreshold}$ (default 0.5) → RISING.
- If $\text{delta} \leq -\text{cfg.Indicators.OFI.MomentumThreshold}$ → FALLING.
- Otherwise → FLAT.
- If buffer has fewer than $2 \times \text{MomentumWindow}$ entries → FLAT (warmup).

Scoring use (in Step 2 OFI block, after the level signal scores):

- OFI level = BULL AND Momentum = RISING → `LongScore += cfg.Indicators.OFI.MomentumAmplify` (default 1) capped at `regimeMax`. Breakdown row note: [+momentum: amplify on rising flow].
- OFI level = BULL AND Momentum = FALLING → `LongScore -= cfg.Indicators.OFI.MomentumSoften` (default 1). Note: [-momentum: soften on fading flow].
- Mirror for SELL_DOMINANT / SELL_LEAN.
- FLAT → no modifier. Most common state.
- Controlled by `cfg.Indicators.OFI.MomentumEnabled`. When false, momentum is computed and displayed but not scored.

Display: Inline on the OFI line as Momentum: RISING / FALLING / FLAT. Colour matches OFI level direction when momentum agrees, dim otherwise.

CSV column 70: `OFIMomentum` — string enum.

Interpretation:

- OFI level measures the current imbalance; OFI momentum measures whether that imbalance is accelerating or decelerating. A level signal being amplified by rising momentum is structurally stronger than a level signal that's fading.
- The pattern matches `FundingMomentum` (same ring-buffer + windowed-delta design, see Section 15). Internal consistency was the goal — both adjuncts modify their parent score in the same way.
- A momentum shift (RISING → FALLING in consecutive runs) on a held position is an early warning that the flow is rolling over — fires before the level signal itself flips.
- During quiet sessions where OFI level rarely leaves NEUTRAL, expect mostly FLAT. Don't read it as bearish or bullish.

CVD

What: Cumulative Volume Delta across recent trades — net signed aggressor notional, a slope classification, and a divergence state against price.

Calculation (`CalcCVD`):

- Trades fetched via `GetRecentTradesAsync(100)` (count hardcoded in `MainForm_Analysis.RunAnalysisAsync`; the v14 `cvd.trade_lookback` config key was never wired to this fetch and was removed in v15).
- Each trade: `signedDelta = +amount` if `Direction = buy` else `-amount`. (Amount is in USD notional on Deribit BTC-PERPETUAL.)
- Split window into 3 equal segments — early, mid, late thirds.
- `CVDValue = early + mid + late` (total net delta).
- Weighted slope: `weightedSlope = lateDelta × 2 - earlyDelta × 1`.
- Slope threshold: `max(slope_min_usd (12000 v14), |CVDValue| × slope_pct_of_value (0.01))`.

- **CVDSlope:**
 - $\text{weightedSlope} > +\text{threshold} \rightarrow \text{RISING}$
 - $\text{weightedSlope} < -\text{threshold} \rightarrow \text{FALLING}$
 - Else $\rightarrow \text{FLAT}$
- **CVDDivergence:**
 - Requires $|\text{priceChange}| \geq \text{divergence_price_gate}$ (0.0005) (0.05% over the last 2 candles).
 - Price up + CVD negative $\rightarrow \text{BEARISH}$
 - Price down + CVD positive $\rightarrow \text{BULLISH}$
 - Else $\rightarrow \text{NONE}$

Scoring use:

- **Full long (Microstructure):** $\text{CVDSlope} = \text{RISING AND CVDValue} > 0 \rightarrow +1 \text{ LongScore}$.
- **Full short:** $\text{CVDSlope} = \text{FALLING AND CVDValue} < 0 \rightarrow +1 \text{ ShortScore}$.
- **Divergence penalty:** $\text{BEARISH} \rightarrow \text{LongScore} -= \text{DivergencePenalty}$ (1). $\text{BULLISH} \rightarrow \text{ShortScore} -= 1$.
- **Pass 2b OI $\times$ CVD:** uses $\text{cvdBullish} = (\text{RISING AND Value} > 0) / \text{cvdBearish} = (\text{FALLING AND Value} < 0)$ as the confirmation/conflict axis against OI.
- **Pass 2c TRENDING input:** aligned if slope and sign both match dominant side.

Display colour: green on RISING, red on FALLING, grey on FLAT.

Interpretation:

- The 3-segment weighted slope (late $\times 2$ – early $\times 1$) is deliberately late-weighted. A single large trade early in the window can't dominate the classification — you need the back third of the window to be genuinely trending. This reduces false RISING / FALLING flips.
- v14's $\text{slope_min_usd} = 12000$ (raised from 1000) means small order flow no longer qualifies as trending. On active BTC perp, net notional over 100 trades can easily reach 100K-500K — 12K is a meaningful floor that filters dead sessions.
- Div: BEARISH with Slope: RISING is possible — price rising + CVD rising but slower than price would expect. Penalty fires against LongScore. Read as “the rally is real but buy aggression is slowing”.
- FLAT slope with large absolute CVDValue means strong directional bias built earlier in the window but current aggression is balanced — no fresh score, but the positional lean is still there.

TFI (Trade Flow Index)

What: Short-burst aggressor pressure ratio over the most recent 30 trades.

Calculation (CalcTFI):

- Take first tfi_window_size (30) trades from the recent-trades list.
- $\text{buyFlow} = \sum(\text{buy amounts}), \text{sellFlow} = \sum(\text{sell amounts})$.
- $\text{TFIValue} = (\text{buyFlow} - \text{sellFlow}) / (\text{buyFlow} + \text{sellFlow})$ — normalised $[-1, +1]$.
- Classification against threshold (0.15):
 - $\text{TFIValue} > +0.15 \rightarrow \text{BUY PRESSURE}$
 - $\text{TFIValue} < -0.15 \rightarrow \text{SELL PRESSURE}$
 - Else $\rightarrow \text{NEUTRAL}$

Scoring use:

- **Full long (Microstructure):** $\text{BUY PRESSURE} \rightarrow +1 \text{ LongScore}$.
- **Full short:** $\text{SELL PRESSURE} \rightarrow +1 \text{ ShortScore}$.
- NEUTRAL: no score.

Display colour: green on BUY PRESSURE, red on SELL PRESSURE, grey on NEUTRAL.

Interpretation:

- TFI's 30-trade window is deliberately small — it measures very recent (often last 30–120 seconds) aggressor burst, not structural flow. Window was separated from MicroCVD (50) precisely because a burst signal and a structural segmentation signal need different horizons.
- $|TFIValue| > 0.6$ (like the sample's **0.606**) is extreme — 60%+ imbalance across 30 trades means one side is clearly clubbing the other. Pair with MicroCVD direction for conviction.
- NEUTRAL with $|TFIValue|$ just below **0.15** means there's a mild aggressive lean that didn't clear the threshold. Watch the direction — if the next run also reads mild-lean same side, the signal is building.

MicroCVD

What: Intra-window CVD segmentation — splits the 50-trade window into thirds and classifies whether net delta is accelerating or decelerating.

Calculation (CalcMicroCVD):

- Take first `micro_window_size` (50) trades.
- Split into 3 equal segments: `MicroCVDEarly` / `Mid` / `Late` — each the net signed amount for that third.
- `netDelta = Early + Mid + Late`; `isBull = netDelta > 0`.
- Momentum classification (using `accel_threshold` (10000) v14):
 - **Bull case** (`isBull = true`):
 - * `Late > 0 AND Late > Early + 10000` → ACCELERATING
 - * `Late < 0 OR Late < Early - 10000` → DECELERATING
 - * Else → FLAT
 - **Bear case** (`isBull = false`):
 - * `Late < 0 AND Late < Early - 10000` → ACCELERATING
 - * `Late > 0 OR Late > Early + 10000` → DECELERATING
 - * Else → FLAT
- MicroCVDSignal cross-tabulation:

isBull	Momentum	Signal
true	ACCELERATING	BULL_ACCEL
true	DECELERATING	BULL_DECEL
false	ACCELERATING	BEAR_ACCEL
false	DECELERATING	BEAR_DECEL
any	FLAT	FLAT

Scoring use:

- **Full long (Microstructure):** BULL_ACCEL → +1 LongScore.
- **Full short:** BEAR_ACCEL → +1 ShortScore.
- **Deceleration penalty** (`DecelPenalty = 1`):
 - BULL_DECEL → ShortScore -= 1. Logic: net flow is still bullish (`isBull = true`) but weakening; shorts are clearly NOT in control, so penalise any short-side score they've accumulated. Breakdown note: PENALTY -1 opposing.
 - BEAR_DECEL → LongScore -= 1. Mirror.
- **FLAT stall penalty:** when `MicroCVDSignal = FLAT` AND:
 - `CurrentPrice > VWAP AND CVDValue ≤ 0` → LongScore -= 1. Note: STALL PENALTY -1 [L] (`price > VWAP, CVD ≤ 0`). Read: price holding above VWAP but no buy flow confirms it — long posture is stalling.
 - `CurrentPrice < VWAP AND CVDValue ≥ 0` → ShortScore -= 1. Mirror.
- **Hold Status microstructure exit:** BULL_DECEL / BEAR_DECEL counts as one adverse signal towards the 2-of-N fast-exit layer (see Verdict block, Hold / Exit).
- **Context tag input:** BULL_DECEL (if long dominant) or BEAR_DECEL (if short) contributes one count toward MOMENTUM_FADING classification. Also `MicroCVDLate < 0.5 × MicroCVDEarly` (same-side collapse) counts independently.

Display fields:

- E — MicroCVDEarly (first third's net delta, USD notional, signed)
- M — MicroCVDMid
- L — MicroCVDLate
- DECELERATING / ACCELERATING / FLAT — the Momentum classification
- BULL_ACCEL / BULL_DECEL / BEAR_ACCEL / BEAR_DECEL / FLAT — the Signal

Display colour:

- Green — BULL_ACCEL
- Red — BEAR_ACCEL
- Soft green — BULL_DECEL
- Soft red — BEAR_DECEL
- Grey — FLAT

Interpretation:

- **Negative segment values are valid and expected**, explicitly documented in the trader profile. L: -14720 in the sample means the last third of the window saw net sell-side aggression — which, with a positive net delta overall ($E + M + L = 3370 > 0$), produces $\text{isBull} = \text{true} + L < E - \text{threshold} = 2840 - 10000 = -7160 \rightarrow \text{DECELERATING} \rightarrow \text{BULL_DECEL}$.
 - BULL_DECEL is visually “bullish but weakening”. The scoring engine uses it to penalise *shorts* (not longs) because net bias is still up — but the context tag uses it as evidence that bullish momentum is fading. Both behaviours coexist deliberately.
 - The v14 raise of `accel_threshold` from 5000 to 10000 means DECEL/ACCEL classifications now require larger segment-to-segment moves before firing — quiet sessions produce more FLAT signals, active sessions are unaffected. Interim-static fix until dynamic scaling ships.
 - **FLAT stall penalty is subtle**: it fires when the directional structure (price vs VWAP) disagrees with accumulated flow (CVD sign). This catches the “price drifting above VWAP without buy aggression” or mirror case — the trade thesis is stalling. Rare but meaningful when it fires.
 - Reading E / M / L as a trend: $E < M < L$ with all positive = accelerating bull (clean). $E > M > L$ with all positive = mature rally running out of energy. $E > 0, L < 0$ with net positive = last-third flip (often a local top). Don't over-read — the engine's classification is the authoritative call; the numbers are for manual cross-checking.
-

13. Liquidations

LIQUIDATIONS:

Long: 0 | Short: 0 | Signal: NONE

Penalty-only signal derived from forced-liquidation trades in the recent trade stream. No positive reward — a liquidation cascade on one side penalises that same side's score (directional conviction evaporates when forced exits dominate).

Long / Short / Signal

What:

- Long — total BTC size of long liquidations in the recent trades window.
- Short — total BTC size of short liquidations in the recent trades window.
- Signal — classification of which side's liquidations dominate, filtered by `DominanceRatio`.

Calculation (CalcLiquidations in Indicators_OrderFlow.vb):

- Iterate the recent-trades list. For each trade with `Liquidation != "none"`:
 - `Direction = "buy"` (counterparty was buying, forced short closing) $\rightarrow \text{liqShortSize} += \text{amount}$.

- Direction = "sell" (counterparty was selling, forced long closing) → liqLongSize += amount.
- Classification using `cfg.Indicators.Liquidations.DominanceRatio` (2.0 in settings.json):
 - liqLongSize > 0 AND liqLongSize ≥ liqShortSize × DominanceRatio → LONG LIQS
 - liqShortSize > 0 AND liqShortSize > liqLongSize × DominanceRatio → SHORT LIQS
 - Else → NONE

Scoring use:

- LONG LIQS: LongScore -= LiqLargePenalty (2) if liqLongSize > LargeLiqSize (200 BTC), else LongScore -= LiqStandardPenalty (1).
- SHORT LIQS: mirrored on ShortScore.
- NONE: no score change.
- Breakdown note appends PENALTY -N [L] or PENALTY -N [S] when penalty fired.

Display colour: amber when Signal != NONE, dim grey when NONE.

Interpretation:

- Penalty-only since v0.17 — was previously a two-way reward that fired ~95% of the time as non-directional padding. Now silent in the normal case; only speaks when there's an actual cascade.
- DominanceRatio = 2.0 means the dominant side must be ≥ 2× the other before the classification fires. Mixed liquidations (say, 100 BTC long + 80 BTC short) read as NONE. Trader profile flagged 1.2–1.5 as a candidate tune; currently held at 2.0 pending backtesting.
- LargeLiqSize = 200 BTC is the threshold between -1 (standard) and -2 (large) penalties. At BTC \$76k that's ~\$15M in a single direction over the recent window — a genuine cascade event. The -2 penalty punishes the cascade *direction* because forced exits at that magnitude imply the level just printed is a local extreme that's already been hit hard, not a fresh setup.
- An “inverted” read like LONG LIQS penalising LongScore can look counterintuitive — but longs being liquidated means price moved against longs, i.e. price is falling or just fell. Penalising further long entries during active long-side capitulation is correct. The mirror applies to SHORT LIQS.
- Long: 0 | Short: 0 | Signal: NONE is the normal resting state during a quiet session — not a data issue.

14. MTF Gate

MTF GATE (15m): PASS

15m Trend: BULL | ADX: 20.5 | EMA: BEAR

Reason: MTF PASS [LONG] 15m +DI:25.4 -DI:20.0 ADX:20.5 EMA:BEAR | Bull:2 Bear:1 (need 2)

The 15m multi-timeframe confluence gate. A **hard veto** — when it blocks, the verdict is forced to NO TRADE regardless of 1m score.

Section header

Format: MTF GATE (15m): <PASS | BLOCK>

State:

- PASS — gate did not veto. Either the 15m state aligns with the proposed 1m direction, is flat (neutral), or no direction was proposed.
- BLOCK — gate vetoed. The 1m signal would have been WEAK/MED/STRONG LONG or SHORT, but 15m state opposed the direction.

15m Trend / ADX / EMA

What:

- 15m Trend — MTF15mTrend, one of BULL / BEAR / FLAT.
- ADX — MTF15mADX, Wilder-smoothed ADX on the 15m series.
- EMA — MTF15mEMAAlignment, one of BULL / BEAR / MIXED.

Calculation (CalcMTFGate in Indicators_Structure.vb):

- 15m candles fetched with a 60-second TTL cache (MTF_TTL_SECONDS) — doesn't re-fetch on every 1m run.
- candleLookback = cfg.MTFGate.CandleCount (60) — last 60 × 15m candles = 15 hours of history.
- Compute CalcDMI(window, adxPeriod=cfg.MTFGate.DmiPeriod=9) on the 15m window → plusDI, minusDI, mtfADX.
- dmiIsBull = plusDI > minusDI.
- adxStrong = mtfADX ≥ cfg.MTFGate.AdxMin (20.0) — **v14 fix**: this value now reads from the dedicated adx_min setting. Pre-v14 it silently borrowed the 1m trend threshold (25), effectively disabling the gate's ADX component in most mid-range sessions.
- Compute CalcEMA(window, 9/21/50) → emaBull = EMA9>EMA21>EMA50, emaBear = EMA9<EMA21<EMA50, MIXED otherwise.
- **2-of-3 majority vote** accumulates bullScore / bearScore:
 - dmiIsBull → bullScore += 1, else bearScore += 1.
 - adxStrong AND dmiIsBull → bullScore += 1; adxStrong AND !dmiIsBull → bearScore += 1.
 - emaBull → bullScore += 1; emaBear → bearScore += 1; MIXED → no contribution.
- Classification against cfg.MTFGate.RequiredConfirms (2):
 - bullScore ≥ 2 → MTF15mTrend = BULL
 - bearScore ≥ 2 → BEAR
 - Else → FLAT

Gate decision using proposedDirection (set upstream from 1m regime + EMA alignment):

Proposed direction	MTF15mTrend	Result
LONG	BULL or FLAT	PASS
LONG	BEAR	BLOCK
SHORT	BEAR or FLAT	PASS
SHORT	BULL	BLOCK
NONE	any	PASS (no direction proposed — gate inactive)

Proposed direction logic (in MainForm_Analysis.vb):

- Regime = TRENDING_UP OR 1m EMAAlignment = BULL → mtfProposed = LONG
- Regime = TRENDING_DOWN OR 1m EMAAlignment = BEAR → mtfProposed = SHORT
- Else → NONE

Veto trigger (Step 4b): The gate only actually turns the verdict into NO TRADE when:

- cfg.MTFGate.Enabled = true, AND
- A direction cleared tWeak on effective score, AND
- MTFGatePass = false.

In that case the verdict becomes NO TRADE [WEAK LONG] / NO TRADE [WEAK SHORT] (bracketed tag shows the suppressed lean).

Reason

What: Verbose description of the gate outcome. Always present.

Format: One of:

- MTF PASS [LONG] 15m +DI:<+DI> -DI:<-DI> ADX:<adx> EMA:<align> | Bull: Bear:<r> (need <n>)
- MTF BLOCK [LONG vs BEAR] 15m ... (analogous for BLOCK)
- MTF PASS [SHORT] ...
- MTF BLOCK [SHORT vs BULL] ...
- MTF state: <trend> | 15m ... (when no direction proposed)
- MTF: insufficient 15m candles (<N>) (when fewer than $\text{adxPeriod} + 2 = 11$ candles available)
- MTF gate: no data (cold-start placeholder before first 15m fetch)

Breakdown row

Rendered as MTF Gate (15m) in the signal breakdown table. [L] / [S] hit:

- LongHit = MTFGatePass AND proposedDirection = LONG
- ShortHit = MTFGatePass AND proposedDirection = SHORT

Note field is the same MTFGateReason string as rendered in the MTF section.

Display colour: green on PASS, red on BLOCK for the 15m Trend / ADX / EMA line. Reason line always rendered dim grey (it's explanatory text, not a primary value).

Interpretation

- The gate is *soft* by design: FLAT on 15m always passes, because a flat higher timeframe neither confirms nor disconfirms the 1m setup. The veto only fires on active disagreement (15m BULL vs proposed SHORT, or vice versa).
- The sample output shows a subtle case: 15m Trend says BULL, but EMA says BEAR. Bull:2 came from `dmi-IsBull + adxStrong confirm` (two votes); bear:1 came from `emaBear`. The 2-of-3 majority still tilts bull, gate passes LONG. Read as “DMI/ADX say bull, but the EMA structure hasn't caught up yet” — a regime in early transition.
- A BLOCK event is the one place where a strong 1m score gets silenced. If you see a NO TRADE [WEAK LONG] with MTF BLOCK, the trade-off is clear: the 1m pipeline liked it, the 15m pipeline hated it. Trust the higher timeframe.
- The 60-second TTL cache means the 15m values can be up to 60 seconds stale relative to the 1m fetch. On the 15m timeframe this is imperceptible (candles are 15 min each). The cache exists to save API quota — 15m candles don't move enough to warrant fetching every 1m run.
- If Reason reads MTF: insufficient 15m candles (N), the gate opens (PASS) and does not veto — it's a degraded-mode fallback. Watch for this on cold start or after a long disconnect.

15. Funding

FUNDING:

Rate: 0.0007% | NEUTRAL

Momentum: FLAT | Enabled: YES | Soften: +1 | Amplify: -1

Two-row funding block: the raw rate + crowd bias on the first row, and momentum metadata (Step 3b inputs) on the second.

Row 1 — Rate / Bias

What:

- Rate — the funding rate, displayed as percent ($\text{FundingRate} \times 100$, formatted to 4 decimal places).
- Bias — `r.FundingBias`, one of NEUTRAL / LONGS CROWDED / LONGS HEAVILY CROWDED / SHORTS CROWDED / SHORTS HEAVILY CROWDED.

Calculation:

- **FundingRate** fetched from Deribit via `GetFundingRateAsync` — raw 8h-scale funding value. Published roughly every 8 hours by Deribit; samples between publications return the same value.
- **FundingBias** classified elsewhere in `MainForm_Analysis` by comparing **FundingRate** to crowd thresholds. **HEAVILY CROWDED** requires a larger magnitude than plain **CROWDED**.

Scoring use (Step 3 — baseline funding modifier, in `ScoringEngine_Calculate_Scoring.vb`):

Condition (v14 thresholds)	Action
Rate > funding_high_positive (0.0003)	ls -= FundingHighPenalty (2), ss += FundingHighBoost (1). Note: STEP3: -2[L] +1[S]
Rate > funding_low_positive (0.00005)	ls -= FundingLowPenalty (1). Note: STEP3: -1[L]
Rate < funding_high_negative (-0.0003)	ss -= FundingHighPenalty (2), ls += FundingHighBoost (1). Note: STEP3: -2[S] +1[L]
Rate < funding_low_negative (-0.00005)	ss -= FundingLowPenalty (1). Note: STEP3: -1[S]
Else	No change. Note: STEP3: none

Score is post-clamped to `max(0, ...)`.

Display colour (Row 1):

- Red — any **HEAVILY CROWDED** bias.
- Grey — **NEUTRAL**.
- Amber — plain **CROWDED** on either side.

Interpretation:

- **v14 raised the high-threshold from 0.0001 to 0.0003** — BTC perp routinely sits at $\pm 0.01\%/8h$ (= 0.0001 raw), which previously fired the max penalty every run. Post-v14, only genuinely extreme funding ($\geq 0.03\%/8h$ = 0.0003 raw) fires the **FundingHighPenalty (2) + FundingHighBoost (1)** contrarian tilt.
- Contrarian framing: extreme positive funding → longs overcrowded → small nudge *against* longs, *for* shorts. The boost is deliberately smaller than the penalty (1 vs 2) to reflect asymmetric conviction.
- Funding is adjunct — it never votes in Step 2 scoring (rejected pattern; would double-count). Row 1 breakdown appears as **Funding (info)** with no [L] / [S] hit marker, carrying the STEP3 + STEP3b notes as informational text only.
- The rate displayed **0.0007%** = raw 0.000007, well below any threshold — **STEP3: none** is correct.

Row 2 — Momentum / Enabled / Soften / Amplify

What:

- **Momentum** — `r.FundingMomentum`, one of **RISING / FALLING / FLAT**.
- **Enabled** — **YES / NO**, from `cfg.Indicators.Funding.MomentumEnabled`.
- **Soften** — displayed as +N, from `cfg.Indicators.Funding.MomentumSoften` (default 1).
- **Amplify** — displayed as -N, from `cfg.Indicators.Funding.MomentumAmplify` (default 1).

Calculation (`CalcFundingMomentum` in `Indicators_OrderFlow.vb`) — time-anchored since v53:

- Uses the `_fundingHistory` ring of **timestamped** samples. A sample is appended on **every** run and evicted once it is older than **30 minutes** — no count cap, and no dedup (the pre-v53 “only append on change” rule went away with the count window it existed to protect).

- `W = cfg.Indicators.Funding.MomentumWindowMinutes (5), T = MomentumThreshold (0.0000002).`
- The **anchor** is the *newest* sample at least *W* minutes old. `delta = current rate - anchor rate.`
- Cold start, or a gap long enough that eviction emptied the ring (engine restarted, feed outage), leaves nothing old enough to anchor → FLAT.
- Classification:
 - `delta > +0.0000002` → RISING
 - `delta < -0.0000002` → FALLING
 - Else → FLAT

Why the window is measured in minutes: before v53 it was measured in funding *changes* (3 of them). On the WebSocket feed funding changes almost every run, so that window's real span was about three times the run cadence — the same funding path read RISING on a 3-minute session and FLAT on a 1-minute one. Reading `FundingMomentum` now means “*funding moved more than T over at least 5 minutes*”, and that means the same thing at every cadence. Rows logged before this build are not comparable to rows after it.

Scoring use (Step 3b — funding momentum modifier):

Applied on top of Step 3. Behaviour depends on `FundingBias + FundingMomentum`:

Bias	Momentum	Action
LONGS CROWDED / LONGS HEAVILY CROWDED	RISING	<code>ls -= min(MomentumAmplify, FundingHighPenalty)</code> (amplifies crowding penalty; capped at high penalty to avoid double-counting). Note: STEP3b: <code>-1[L]</code> crowding↑
LONGS ...	FALLING	<code>ls += MomentumSoften (1)</code> (de-crowding; capped at <code>regimeMax</code>). Note: STEP3b: <code>+1[L]</code> de-crowding
SHORTS CROWDED / SHORTS HEAVILY CROWDED	FALLING	<code>ss -= min(MomentumAmplify, FundingHighPenalty)</code> . Note: STEP3b: <code>-1[S]</code> crowding↓
SHORTS ...	RISING	<code>ss += MomentumSoften</code> . Note: STEP3b: <code>+1[S]</code> de-crowding
NEUTRAL	RISING AND <code>Rate > 0</code>	<code>ls -= amplify</code> (neutral→crowding transition). Note: STEP3b: <code>-1[L]</code> neutral→crowding
NEUTRAL	FALLING AND <code>Rate < 0</code>	<code>ss -= amplify</code> . Note: STEP3b: <code>-1[S]</code> neutral→crowding
Any	Other combinations	No change. Note: STEP3b: none

Enabled gate: entire Step 3b skipped if `cfg.Indicators.Funding.MomentumEnabled = false`. Note: STEP3b: disabled.

Display colour (Row 2):

- Amber — RISING (penalty-amplifying direction).
- Green — FALLING (de-crowding direction).
- Grey — FLAT.

Display convention for Soften / Amplify:

- Displayed as **Soften**: `+N` and **Amplify**: `-N` even though both are stored as positive magnitudes in config. The sign convention reflects their **effect on score** — soften *adds back*, amplify *deducts further*. This is a readability choice, not a sign error.

Interpretation:

- **Dedup matters:** pre-v14, `_fundingHistory` filled with 10 identical 1m snapshots of the same 8h-published rate — `delta` was always 0 → FLAT → Step 3b almost never fired. Post-v14, history only grows on genuine rate transitions, so RISING/FALLING classifications actually correspond to Deribit publishing a changed funding rate. Expect Step 3b to be dark between funding publications and active only at the boundaries.
- Asymmetric amplify/soften logic: `Amplify` is capped at `FundingHighPenalty` (2) to prevent a compounding penalty (baseline + momentum) from obliterating the score on a mild crowding event. `Soften` has no explicit cap but is subject to the `regimeMax` ceiling.
- The NEUTRAL + RISING + Rate>0 case catches the *transition into* crowding — funding is still in the neutral band but momentum is building toward positive. Engine pre-penalises the long side before the rate actually crosses the high threshold. This is the one place Step 3b fires in a neutral session.
- **Enabled:** NO effectively disables this whole row's scoring contribution. Use to isolate Step 3b's effect when debugging or during periods of suspect funding data.

16. Signal Breakdown Table

SIGNAL BREAKDOWN			
Signal	Long	Short	Note
ROC(9)	[L]		0.115 Slope: FLAT PARTIAL->UPGRADED [L]
RSI(9)			52.6 zones OB:60 OS:40 DIV:BEARISH
...			
TOTAL	8	2	

The itemised scoring ledger. Every signal that contributes to (or penalises) the score appears as a row; the TOTAL row at the bottom must match the header's `LongScore` / `ShortScore`.

Layout

Columns:

- **Signal** — 18-char left-justified label.
- **Long** — 5-char field, shows [L] if `item.LongHit = true`, else blank.
- **Short** — 6-char field, shows [S] if `item.ShortHit = true`, else blank.
- **Note** — free text; per-indicator diagnostic string.

Row colour:

- **C_HIT** (brighter) — row has a hit (`LongHit` or `ShortHit`).
- **C_DIM** — no hit (score-neutral or penalty-only row).

Rows (in display order)

Populated by `RunScoringPipeline` in `ScoringEngine_Calculate_Scoring.vb` (rows 1–20) and `Calculate` in `ScoringEngine_Calculate_Verdict.vb` (MTF Gate). Row 21 (Regime Align) appears only conditionally.

#	Label	Hit logic (source)	Note content
1	ROC(9)	Full rocLong/rocShort OR upgraded partial via Pass 2	<ROC:F3> \ Slope: <ROCSlope> + optional partial/upgrade tag
2	RSI(9)	Full rsiLong/rsiShort OR upgraded partial	<RSI:F1> \ zones OB:60 OS:40 + divergence tag + penalty tag + partial/upgrade tag
3	DMI +/-DI	dmiLong/dmiShort	+DI:<+DI> -DI:<-DI>
4	ADX<threshold>	adxLong/adxShort (requires ADX > adxTrend AND dmi aligned)	<ADX:F1> \ thr:<adxTrend:F0>
5	Volume	Full volLong/volShort OR upgraded mid-tier	<VolumeRatio:F2>x \ thr H:<volHigh>x M:<volMid>x [<normMode>] + partial/upgrade tag
6	VWAP	Full vwapLong/vwapShort OR upgraded partial	During warmup: WARMUP (N/15 candles) -- signal suppressed; otherwise price/VWAP/band summary + partial/upgrade tag
7	BBW/TTM	bbwLongHit/bbwShortHit (fires only on RELEASING/NONE + BUILDING)	BBW value + SqueezeStatus + TTM summary + penalty-active indicator
8	EMA 9/21/50	emaBull/emaBear	9:<e9> 21:<e21> 50:<e50> \ <alignment>
9	Funding (info)	Never hits (display-only row)	Rate%, Bias, Momentum, STEP3: / STEP3b: notes
10	OI Delta	Full oiLong/oiShort OR upgraded partial	15m:<d15>% 60m:<d60>% \ <OISignal> + partial/upgrade tag + Pass 2b confirm/conflict tag
11	OFI	ofiBuy/ofiSell	Ratio:<ratio> \ <OFISignal>
12	CVD	cvdLong/cvdShort	Net:<cvd> \ Slope:<slope> \ Div:<div> + divergence penalty tag
13	TFI	tfiLong/tfiShort	<TFIValue:F3> \ <TFISignal>
14	MicroCVD	microLong/microShort (ACCEL only)	E:<e> M:<m> L:<l> \ <Momentum> \ <Signal> + decel penalty tag + stall penalty tag

#	Label	Hit logic (source)	Note content
15	Liq Penalty	liqLongPenalty>0 / liqShortPenalty>0 (penalty side marked as hit)	L:<long> S:<short> \\ <LiqSignal> + per-side penalty tag
16	5m EMA(200)	ema200Bull/ema200Bear	<EMA200_5m> \\ <PriceVsEMA200>
17	Donchian(20)	Full donchLong/donchShort OR upgraded partial	U:<upper> L:<lower> \\ <DonchianSignal>; MID-CHANNEL -- no signal when DonchianSignal = NONE
18	OBV	obvLong/obvShort OR upgraded partial	Trend:<trend> Div:<div> + [upgrade blocked] tag when adverse divergence + partial/upgrade tag
19	VPFR-lite	vpfrLong/vpfrShort	POC:<poc> \\ <VPFRSignal> \\ HVN@POC:<YES/NO>
20	MTF Gate (15m)	MTFGatePass AND pro- posedDir=LONG/SHORT	MTFGateReason (pass/block text with DMI/ADX/EMA breakdown)
21	Regime Align (2c)	regimeAlignLongHit/regimeAlignShortHit — row appears only when Pass 2c fired (aligned or conflicted)	MARSHALL ATTENTION [<regime>: <sigs> √<suffix>] or -N REGIME CONFLICT [...: <sigs> X<suffix>]

Note annotations — conditional tags

The **Note** field carries cross-cutting annotations that only appear in specific circumstances.

Tag	Meaning	When it appears
PARTIAL [L*] / [S*]	Partial signal detected but not upgraded this run	ROC, RSI, VWAP, OI, Donchian, Volume (mid), OBV
PARTIAL->UPGRADED [L] / [S]	Partial was upgraded to full via Pass 2 cross-category confirmation	Same set as above
[upgrade blocked]	OBV has adverse divergence, blocking cross-category upgrade	OBV only
WARMUP (N/15 candles) -- signal suppressed	VWAP session candle count below warmup threshold	VWAP only
MID-CHANNEL -- no signal	Price in middle half of Donchian channel	Donchian only
DIV:<state>	RSI divergence detected	RSI only (when RSIDivergence != NONE)
PENALTY -N [L] / [S]	Direct score penalty applied	RSI (RSI-div-plus-extreme), CVD (divergence), MicroCVD (DECEL opposing), Liq (size-dependent)

Tag	Meaning	When it appears
STALL PENALTY -N [L] / [S]	MicroCVD FLAT with price/CVD direction mismatch	MicroCVD only
PASS2b: +N[L] / [S] OI×CVD confirmed	OI full-or-upgraded + CVD aligned → UpgradeBonus	OI Delta row only
PASS2b: -N[L] / [S] OI×CVD conflict	OI full (not partial) + CVD opposed → ConflictPenalty	OI Delta row only
+N REGIME ALIGN [...]	All Pass 2c signals aligned with dominant side	Regime Align (2c) row — only rendered when gate fired
-N REGIME CONFLICT [...]	All Pass 2c signals conflict with dominant side	Regime Align (2c) row
<regime>: EMA+ROC+CVD / EMA+CVD	Pass 2c TRENDING signal set; EMA+CVD when ROC was inactive	Regime Align row only
<regime>: VWAP+RSI+DON / RSI+DON	Pass 2c RANGE_BOUND signal set; RSI+DON when VWAP was in warmup	Regime Align row only
(ROC neutral) / (VWAP warmup)	Suffix clarifying why the “reduced” signal set was used	Regime Align row only
[LIVE] / [STATIC]	DynamicNorms mode	Volume row only

TOTAL row

Format: TOTAL <LongScore> <ShortScore> — displayed in white bold.

Authority: These are `v.LongScore` / `v.ShortScore` — the **raw** scores after all Step 2 scoring, Pass 2/2b/2c adjustments, Steps 3/3b funding modifiers. They are the same values shown in the header SCORE line’s non-effective fields.

Reconciliation check: Sum the [L] marks in the table (each = +1) and subtract any PENALTY / CONFLICT amounts that landed on the long side. The result should equal LongScore. Same for short. Small discrepancies usually mean a penalty applied to a score that was already 0 and got floored (e.g. the sample’s MicroCVD BULL_DECEL -1 [S] which landed when short was at 0, so the penalty was absorbed — the [L] marks sum correctly, and short=2 is explained by 5m EMA(200) + VPFR with the micro penalty having been zero-floored at the moment it fired).

Interpretation

- Read the table top-down: the top rows are the core trend/momentum primitives, the middle rows are order flow, the bottom rows are structural context + gates. A verdict leaning one way with hits clustered in a single section (e.g. all order flow, no structural) is the classic FLOW_UNCONFIRMED setup — the CONTEXT tag will usually call it out, but the breakdown tells you *where* the unconfirmed side is thin.
- The Funding (info) row is the trader’s readout of what Step 3 / Step 3b just did to the score — it’s the only way to see funding modifier magnitudes directly. No [L] / [S] marker, but the note text is informative.
- Any line carrying PENALTY, STALL, CONFLICT, or PASS2b: -N is a score deduction — you won’t find these in the TOTAL reconciliation as positive hits, but they’re part of why TOTAL may be lower than the raw [L] count.
- The Regime Align (2c) row is a binary event indicator — **its presence alone is notable**. Most runs don’t fire it (needs all active Pass 2c signals to unanimously agree or unanimously conflict). When you see it, the engine is expressing strong regime conviction.
- When debugging a surprising verdict, the fastest path is: (1) scan the TOTAL row to see the raw score, (2) check the SCORE header line for (eff.N) delta, (3) look for any PASS2b, REGIME CONFLICT, PENALTY tags in the note column to explain the delta, (4) if everything reconciles and the verdict still surprises you, check the CONTEXT line — it may be flagging a quality issue the raw score doesn’t capture.

17. CSV Logging

The engine appends one row per analysis run to `bin/Debug/net8.0-windows/analysis_log.csv`. Current schema is **v0.8 — 111 columns** (rotated at the v0.5 boundary, 2026-07-03; the prior book survives as `analysis_log.csv.v0.7.bak`, never deleted — two standing watches read it). Used by the offline analysis script (Section 18) and the auto-tweaker (Section 19).

The table below documents the **v0.4 core (87 columns)**. The v0.5–v0.8 additions (24 columns) are catalogued per rotation in the settings `change_log` and project-doc §15 — the v0.8 rotation alone added 16: `AggrVel-BurstRatio/Net/Signal`, `TFIValue/TFISignal`, five reserved `Absorption*` columns (null until #6 builds), `PlacedTargetLong/PlacedStopLong/PlacedTargetShort/PlacedStopShort` (the four-surface placed geometry), and `InstanceId/SignalId` (bridge attribution).

Schema versioning and rotation

`AnalysisLogger.EnsureLogFile()` reads the first line of the existing log on startup. If the header doesn't match the current expected v0.4 header, the existing file is renamed to `analysis_log.csv.<schema-tag>.bak` and a fresh file is started with the current header. Idempotent — if no log exists, just writes the header.

Two backups commonly present after Bundle 1 + Bundle 3 shipped:

- `analysis_log.csv.v0.3.bak` — pre-Bundle-1 log with 68 columns
- `analysis_log.csv.v0.4.bak` — post-Bundle-1 log with 86 columns (rotated when d1 added column 87)

Schema (v0.4 core — 87 columns; v0.5–v0.8 additions per the note above)

1	Timestamp	ISO 8601 UTC
2	Price	Last transacted price at run time
3	Verdict	String: STRONG_LONG / LONG / WEAK_LONG / NO_TRADE / etc.
4	Confidence	String: HIGH / MEDIUM / LOW / N/A
5	LongScore	Raw long score after Pass 2/2b/2c + funding modifiers
6	ShortScore	Raw short score (same point in pipeline)
7	EffectiveLongScore	After Step 4 regime veto + Step 4b MTF veto
8	EffectiveShortScore	Same
9	MaxScore	Regime-adjusted ceiling
10	RegimePenalty	TRANSITIONAL ADX-proximity penalty (0 elsewhere)
11	Regime	TRENDING_UP / TRENDING_DOWN / RANGE_BOUND / TRANSITIONAL
12–13	ADX, PlusDI, MinusDI	DMI on 5m
15–16	ROC, ROCsSlope	1m rate-of-change
17–18	RSI, RSIDivergence	1m RSI(9)
19	VolumeRatio	vs Volume SMA(9)
20–26	VWAP fields	Value, dev%, candle count, s1/s2 bands
27–31	BBW + TTM fields	Squeeze status, histogram, direction, signal
32–37	EMA fields	EMA9/21/50, alignment, 5m EMA200, price-vs-200
38–39	Funding rate + bias	
40–43	OI fields	Current, 15m delta, 60m delta, signal
44–47	OFI fields	Ratio, bid vol, ask vol, signal
48–50	CVD fields	Value, slope, divergence
51–53	Liquidation fields	Long size, short size, signal
54–56	Donchian fields	Upper, lower, signal
57–58	OBV fields	Trend, divergence
59–63	MTF Gate fields	Pass, 15m trend, 15m ADX, 15m EMA alignment, reason
64–65	ATR + multiplier	
66	VerdictContext	FLOW_UNCONFIRMED / MOMENTUM_FADING / STRUCTURALLY_WEAK / CONFIRMED (v0.3)
67	FundingMomentum	RISING / FALLING / FLAT (v0.3)

68	OiCvdOutcome	Pass 2b output (v0.3)
69	SpreadBps	Bid-ask spread in basis points (v0.4)
70	OFIMomentum	RISING / FALLING / FLAT (v0.4)
71	FundingDelta	Period-over-period funding rate change (v0.4)
72-73	VPFRVAH, VPFRVAL	Value area boundaries (v0.4)
74-75	VPFRNearestHvnAbove/Below	Nearest high-volume walls (v0.4)
76-79	LastSwingHigh/Low 5m + 15m	(v0.4)
80-83	SwingTargetLong/Short, SwingStopLong/Short	(v0.4)
84	TargetCapReason	swing / hvn / poc / none – Step 5b 3-tier winner (v0.4)
85-86	BestPivotByVolume5m, BestPivotVolumeRatio5m	(v0.4 reserved, populated by D2)
87	TrendStructure5m	UPTREND / DOWNTREND / EXPANSION / CONTRACTION / UNDEFINED (Bundle 3 d1)

Skipped runs do not write a row. When `RunAnalysisAsync` aborts due to a missing required result (1m / 5m candles, funding, book summary, order book, recent trades), the CSV is untouched.

Companion log. `settings_snapshots/manifest.csv` is a separate CSV maintained by the auto-tweaker (see §20). It is not part of `analysis_log.csv` — different schema, different cadence — but it lives in the repo alongside the snapshot `.json` files it indexes.

CalibrationReport

Generated on demand from the status bar **Calibration** Check link. Counts coverage along multiple axes against gate thresholds:

- **Total rows** ≥ 300 (gate)
- **Sessions (UTC days)** ≥ 3 (gate)
- **Liquidation events** ≥ 2 (gate; rare-event blocker — accepted as deferred 2026-05-05)
- **Per-regime row counts** ≥ 50 each (gate)
- Plus distribution sections (informational only): regime, verdict context, funding momentum, OI×CVD outcomes, spread, OFI momentum, target cap reason, trend structure, best volume pivot.

Verdict line at the end reads **READY** or **NOT YET READY**.

18. Analysis Report Viewer

The status bar **Analysis** Report link runs the offline analyser (`analysis/AnalysisRunner.Run`) over the current `analysis_log.csv` and opens a non-modal viewer (`AnalysisReportForm`) with the rendered markdown.

Output files

Written to the engine’s working directory each click:

- `analysis_report_<yyyyMMdd_HHmss>.md` — full markdown report
- `analysis_summary_<yyyyMMdd_HHmss>.csv` — flat failure-rate matrix (one row per **population × tier × window × threshold**; leading `Population` column). Artifact only — the auto-tweaker computes its own matrix via `FailureRateMatrix.Compute` over its filtered rows and does **not** read this file.

Report sections

Since the resolution-segmentation fix (`offline-analysis-report-audit-proposal.md`) the matrix and every barrier-based section are computed **per (session × resolution) population** — NY×1, LONDON×3, ASIA×3 — never pooled across execution regimes (1-min NY blended with 3-min Asia/London was a confounded mix of two ATR scales). Layout is **tier-major**: each verdict tier shows one sub-table per session.

- **Global Summary** — rows in CSV, global verdict counts, a “Populations detected” line, and a per-population barrier-diagnostics table (rows / no-OHLC-excluded / ATR-invalid / below-min-move / adverse-source).

below-min-move = 0 is expected on an all-post-v35 book — the live gate already NO-TRADE'd sub-floor signals before they were logged.

- **2. Failure-Rate Matrix** — per tier, one sub-table per session: hold window (5/10/15 min) × ATR threshold. Each cell shows rate% n=sample [ci_low-ci_high] (95% Wilson). Each sub-table is headed with that session's resolution + directional-row ATR caption (p50, p25-p75) + the \$ move-floor, so × ATR reads in dollars. ★ = lowest CI width, ♦ = lowest failure rate (both need n ≥ 30), picked **within** each sub-table.
- **3. Recommended (window, threshold)** — per tier × session: ★ most-precise + ♦ lowest-failure picks.
- **4a. Barrier-Hit Decomposition** — per tier × session: success / adverse-hit / window-expiry / ambiguous counts.
- **4. Verdict Context Tag × Outcome** — per session: failure rate per VerdictContext at that session's recommended cell.
- **8. Hold Window Selection Stats** — per tier × session: the ★/♦ recommended hold window.
- **9. Pending data** — per tier × session: cells with n < 30 (insufficient sample).
- **Global Diagnostics** (not segmented — book-wide, resolution-independent): **5. Funding Momentum** (empirical FundingDelta distribution + percentiles; the "current threshold" now reads the live momentum_threshold), **6. OFI Outlier Audit** (OFIRatio > 100 / > 1000 + top-10), **7. OI×CVD Asymmetry Audit** (confirmed-long vs -short by Regime and Funding Bias).

Outcome definition (constants in analysis/AnalysisConstants.vb)

```
Public ReadOnly HoldWindowsMinutes As Integer() = {5, 10, 15}
Public Const MinSamplesPerCell As Integer = 30
Public Const MinSamplesForAutoTweakerTrigger As Integer = 60
' Legacy-fallback multipliers (pre-v0.8 rows only, since the placed-target
' migration on 2026-07-21):
Public Const EngineTargetAtrMultiplier As Double = 2.0 ' legacy favourable
Public Const AdverseFallbackAtrMultiplier As Double = 1.2 ' legacy adverse
```

Barrier-hit on placed geometry (v2 semantics on the placed-target migration, 2026-07-21). Walk the 1-min OHLC bars across the hold window (the verdict bar and the next two minutes are excluded for execution latency — the first eligible bar closes at T+3 min):

- **SUCCESS** = price wicks through the **favourable** barrier (the logged PlacedTargetLong / PlacedTargetShort — used unfloored, because the live Step 5c min-move gate already vetted that exact price) before any adverse hit.
- **FAILURE** = the **adverse** barrier (the logged PlacedStopLong / PlacedStopShort) is hit first, OR the window expires without a favourable hit, OR both barriers are touched in the same 1-min bar (conservative ambiguous-bar rule).
- **NO_DATA** (F4, v6 eval-cache schema, 2026-07-21) = the OHLC window had no bars — excluded from both numerator and denominator, mirroring FailureRateMatrix.Compute. Fixes the invisible bias that stamped uncoverable rows as FAILURE.
- Cell space is **(tier × session × window)**, one placed-geometry column per cell — the retired per-tier ATR-threshold grid (StrongAtrThresholds {0.5, 0.8} / MediumAtrThresholds {0.3, 0.5}) was collapsing at low ATR (every column sat below the min-move floor), so threshold sweeping moved to the What-If runner which does it with EV and a split-half holdout.
- NO-TRADE rows are excluded from every denominator (informational only). WEAK_* rows are stored but excluded at DISPLAY time (E2a, v55) from both the strip's headline rate and the offline matrix — the strip surfaces the WEAK aggregate as a separate WEAK excl. : tooltip line.
- Pre-v0.8 rows (no Placed* columns) keep the legacy formula on both sides via the two multipliers above and are labelled [LEGACY_YARDSTICK] (never silently pooled with placed rows).

Success orientation (v55, 2026-07-21). All rates in the report are SUCCESS rates. The auto-tweaker's internal comparison is unchanged (it reads Failures / SampleSize directly), converted to success at render only.

Picked-cell history

`analysis/picked_cell_history.csv` (gitignored): one line per auto-tweaker run with timestamp, tier, window, threshold. Drives the “Hold Window Selection Stats” report section.

Portability

All classes in `analysis/` are host-agnostic except `AnalysisReportForm.vb` (the viewer). The non-form classes are referenced from both the WinForms app and the auto-tweaker console app. Future Linux CLI port reuses them directly.

19. Tweak Settings & Auto-Tweaker

The auto-tweaker is a separate .NET 8 console app (`tools/AutoTweaker/AutoTweaker.dll`, target framework `net8.0` — no Windows dependency) that periodically reviews verdict accuracy and proposes targeted `settings.json` adjustments via the Anthropic API. Configured and triggered via the **Tweak Settings** dialog in the main window.

Architecture

<code>tools/AutoTweaker/</code>	
├ AutoTweaker.vbproj	.NET 8 console project, zero WinForms refs
├ AutoTweakerProgram.vb	Entry point. Walks up to find <code>DeribitVerdictEngine.sln</code> (sets working dir), loads <code>tweaker_config.json</code> , invokes <code>Core</code> .
├ AutoTweakerCore.vb	Pipeline. Eligibility → window → failure rate → trigger → prompt → API call (or dry-run write) → diff parse → apply.
├ PromptBuilder.vb	Builds system + user message from <code>settings.json</code> + recent CSV slice + failure-rate matrix + picked-cell history + trader-profile constraints (rejected approaches inlined).
├ ClaudeApiClient.vb	/v1/models discovery for latest Opus, /v1/messages call. API key from <code>ANTHROPIC_API_KEY</code> env var only.
├ SettingsDiffApplier.vb	Validate + apply diff. Hard rejection list, 3-key cap, stale-value check, version-monotonicity bump.
├ TweakerConfig.vb	POCO for <code>tweaker_config.json</code> . Hot-read each invocation.
└ TweakerState.vb	POCO for <code>state.json</code> . Persistent across runs.

Eligibility checks (run at start of every invocation)

1. **Cooldown:** $(\text{current CSV row count}) - (\text{state.last_run_csv_row_count}) \geq \text{tweaker_config.cooldown_rows}$ (default 10).
2. **Session-aligned window:** walk back from end of CSV. The last `window_size_verdicts` rows (default 120) must all fall within the same UTC session bucket per `cfg.SessionVolume.Sessions[]`. If a session boundary appears earlier, the run is ineligible until the current session accumulates a full window.
3. **Tier-eligible row count:** within the window, count `STRONG_*` + `MEDIUM_*` verdicts. Must be $\geq \text{min_tier_eligible_rows}$ (default 60). Prevents auto-tweaks driven by edge cases where most rows are `NO_TRADE`.

If any check fails, exit code 2 (INELIGIBLE), no API call, no settings change.

Trigger logic

After eligibility, compute the failure-rate matrix over the window (reuses `analysis/FailureRateMatrix.vb`). Pick the most stable cell per tier (lowest CI width with $n \geq 30$). Aggregate failure rate is the weighted mean across picked cells.

If `aggregate < failure_rate_threshold_pct` (default 40%) → exit code 0, outcome `BELOW_THRESHOLD`. Engine is performing fine, no tweak needed. This is the normal happy state.

If above → proceed to prompt build + API call.

Latest-Opus model resolution

`ClaudeApiClient.ResolveLatestOpusModel()`:

1. GET `/v1/models` with `x-api-key: $ANTHROPIC_API_KEY`.
2. Filter `data[]` for id starting with `claude-opus-`.
3. Sort by `created_at` descending.
4. Return `data[0].id`. Cache for the process duration; refetch on next process start.
5. Fallback sentinel `claude-opus-latest` if API call fails.

Version-agnostic by design — a future `claude-opus-5-0-20271015` is picked up automatically on next run.

Dry-run mode

When `tweaker_config.dry_run_enabled = true` (default for fresh installs):

- Full prompt + JSON request body written to `tools/AutoTweaker/dry_run_payloads/<yyyyMMdd_HHmss>.txt`
- File contains: trigger reason, system message, user message, JSON body, instructions for human
- No API call made
- State updated: `last_run_outcome = DRY_RUN_WRITTEN`

User opens a separate Claude conversation, pastes the messages, gets back a JSON diff, saves it at `tools/AutoTweaker/manual_diffs/<timestamp>.json`, then runs `AutoTweaker.exe --apply-manual <path>` to apply.

SettingsDiffApplier — hard rejection list

Validates the diff before any application. Returns invalid (exit code 1) if any of:

- **3-key scope cap** — proposal touches more than 3 keys (`max_keys_per_proposal`, default 3)
- **Banned path fragments** — any of: `_fixed_pct_`, `bbw_none_bonus`, `oi_prev15m`, `oi_prev60m`, `atr_avg20d`, `static_vol_high`, `static_vol_mid`, `static_vol_low` (last 6 are dead v15 cleanup keys; first two are explicitly rejected patterns)
- **Disabling gated paths** — `mtf_gate.enabled = false` or `regime_weights.enabled = false`
- **Off-surface subtree prefixes** — any key under `kelly.`, `resolution_profiles.`, `network.`, `exit_guard.`, `auto_run.`, `live_strip.`, `scoring.hold.`, `signal_bridge.`, `indicators.aggressor_velocity.default.`, `indicators.aggressor_velocity.sessions.`, `indicators.ofi.momentum.` or `scoring.structural_levels.session.` (HARD CONSTRAINTS 11–21 — trader-owned re-baseline / display / plumbing keys with no failure-rate linkage)
- **Off-surface prefixes** — `scoring.trade_costs.` (v62: the whole fee model + the min-net risk preference behind the min-move floor — HARD CONSTRAINT 26; the retired flat `scoring.min_tradeable_move_pct` no longer resolves at all)
- **Off-surface exact keys** — `indicators.ofi.averaging.enabled`, `indicators.aggressor_velocity.enabled` / `.scoring.enabled`, `scoring.structural_levels.enabled` / `.stop_too_loose_mode` (feature switches, not thresholds)
- **Direct version edit** — applier manages version bump; diff must not touch `version`
- **Stale diff** — proposed `old_value` doesn't match current settings value

The PromptBuilder HARD CONSTRAINT prose (1–21) and the `SettingsDiffApplier` reject lists are two halves of the same fence — the prose tells the model, the applier enforces it in code.

Settings ownership tiers (P13 — who tunes what)

Every `settings.json` key falls into one of three ownership tiers. Only Tier 1 is auto-tuned; Tiers 2 and 3 are the trader’s, enforced by the reject lists above.

Tier	Who owns it	What it is	Representative keys
1 — Auto-tweaker-tunable	The tweaker (failure-rate levers)	Thresholds the optimiser may propose to cut the empirical failure rate	<code>scoring.verdict_*_pct</code> , <code>scor-</code> <code>ing.regime_max_score.*</code> , <code>scoring.tier_floor.*</code> , <code>scor-</code> <code>ing.context_tag_thresholds.*</code> , <code>scor-</code> <code>ing.bbw_squeeze_penalty</code> , <code>scor-</code> <code>ing.atr_target_multiplier</code> <code>/ atr_stop_multiplier</code> , <code>scor-</code> <code>ing.structural_levels.target_max_a</code> <code>/ stop_max_atr_mult /</code> <code>stop_min_floor_ticks</code> , <code>regime_gates.*</code> , <code>regime_weights.{trending,range_bou</code> and most indicators.* thresholds (RSI, ROC magnitude/slope FLAT keys, ADX, <code>OFI.book_depth /</code> <code>buy_dominant_ratio /</code> <code>sell_dominant_ratio /</code> <code>avg_window_sec</code> , Donchian, TTM <code>flat_threshold</code> , VPFR, Liq, CVD, MicroCVD, <code>fund-</code> <code>ing.momentum_threshold</code> , OI×CVD, Spread, Swing, the flat <code>aggressor_velocity.*</code> params)

Tier	Who owns it	What it is	Representative keys
2 — Hand-tuned re-baseline	Trader / coordinator, by manual firing-rate-match — never the tweaker	Per-session / per-resolution overrides set from measured distributions, not from failure rate	<code>resolution_profiles.*</code> , <code>session_volume.sessions[].*</code> (volume multipliers, <code>execution_resolution</code> , <code>roc_magnitude_threshold</code>), <code>indicators.aggressor_velocity.default.*</code> <code>+ .sessions.*</code> , <code>scoring.structural_levels.sessions.*</code> (LONDON 2.0 / ASIA 1.25)
3 — Hand-toggle switch	Trader, deliberate on/off	Feature switches, risk sizing, display/ops preferences — no failure-rate meaning	<code>kelly.*</code> , <code>network.*</code> , <code>exit_guard.*</code> , <code>auto_run.*</code> , <code>live_strip.*</code> , <code>signal_bridge.*</code> , <code>scoring.hold.*</code> , <code>scoring.trade_costs.*</code> (v62), <code>mtf_gate.enabled</code> (never disable), <code>regime_weights.enabled</code> (never disable), <code>indicators.OFI.averaging_enabled</code> , <code>indicators.OFI.momentum.*</code> (retired v50), <code>indicators.aggressor_velocity.enabled</code> <code>/ scoring_enabled</code> , <code>scoring.structural_levels.enabled</code> <code>/ stop_too_loose_mode</code>

Rule of thumb: if a key answers “*does this reduce the failure rate?*” it’s Tier 1. If it answers “*what does this session / resolution look like?*” it’s Tier 2 (measured, not optimised). If it answers “*do I want this feature on, and how much risk?*” it’s Tier 3.

Enforcement note (2026-07-13). The tier fences are code-level (the reject lists above) with three nuances: (1) the two `session_volume.sessions[].*` strategy keys (`execution_resolution`, `roc_magnitude_threshold`) are fenced at prompt level only (HC 11); (2) in practice **every** array-path `session_volume.sessions[].*` key is additionally un-applicable because the diff applier cannot resolve array paths — harness fixture A15g pins this de-facto block; (3) `session_volume.enabled` is a scalar and **currently passes validation** — an unfenced feature switch (HC16 class); flagged as a candidate for an exact-match fence, trader’s call.

Apply path

When `auto_commit_enabled = true` and the diff passes validation:

- Bump `settings.json.version` by 1
- Set `modified_by = "auto-tweaker-vN"` where N = new version
- Append `change_log` entry: timestamp + summary + cited failure rate + Claude reasoning excerpt
- Write file. `SettingsLoader` `FileSystemWatcher` hot-reloads the engine on its next run.

When `auto_commit_enabled = false`:

- Diff written to `tools/AutoTweaker/proposed_diffs/<timestamp>.json`
- State updated with `last_pending_diff_path`
- User reviews, applies via `AutoTweaker.exe --apply-manual <path>` if accepted.

TweakSettingsForm controls

Non-modal dialog opened from the Tweak Settings link.

Control	Binds to (in <code>tweaker_config.json</code>)
<code>chkAutoCommit</code> (Checkbox)	<code>auto_commit_enabled</code> (default false)
<code>chkDryRun</code> (Checkbox)	<code>dry_run_enabled</code> (default true)
<code>txtWindowSize</code> (TextBox, <code>int ≥ 10</code>)	<code>window_size_verdicts</code> (default 120)
<code>txtFailThreshold</code> (TextBox, <code>int 1-99</code>)	<code>failure_rate_threshold_pct</code> (default 40)
<code>txtCooldownRows</code> (TextBox, <code>int ≥ 1</code>)	<code>cooldown_rows</code> (default 10)
<code>txtSnapshotStreakX</code> (TextBox, <code>int ≥ 1</code>)	<code>snapshot_streak_x</code> (default 3)
<code>txtMaxKeysPerProposal</code> (TextBox, <code>int ≥ 1</code>)	<code>max_keys_per_proposal</code> (default 3 — previously hard-coded)
<code>txtStreakWeight</code> (TextBox, <code>double ≥ 0</code>)	<code>streak_weight</code> (default 1.5)
<code>lblActiveSnapshot</code> (read-only)	Live: Streak: N/X Active snapshot: <filename none>
<code>btnShowRoundStats</code> (Button)	Opens <code>RoundStatsForm</code> non-modally with the last 5 rounds
<code>btnOpenSnapshotsDir</code> (Button)	Opens <code>settings_snapshots/</code> via <code>Process.Start</code>
<code>lblConfigPath</code> (read-only)	full path to <code>tweaker_config.json</code>
<code>lblCsvPath</code> (read-only)	full path to <code>analysis_log.csv</code>
<code>lblStatePath</code> (read-only)	full path to <code>state.json</code>
<code>lblTweakerStatus</code> (dynamic)	Ready / Cooldown: N rows remaining / Waiting for session-aligned window: M/120 rows / Insufficient tier-eligible rows: K/60
<code>btnRunNow</code> (Button)	Disabled unless status is Ready. On click: <code>Process.Start(AutoTweaker.exe)</code>
<code>btnSave</code> (Button)	Validates input, writes to <code>tweaker_config.json</code> , <code>MessageBox</code> confirmation
<code>lblLastTweakSummary</code> (multi-line read-only)	<code>last_run_at_iso</code> + <code>last_run_outcome</code> + <code>last_proposal_summary</code>

The four new fields (`snapshot_streak_x`, `streak_weight`, `streak_length_clamp`, `max_keys_per_proposal`) live alongside the existing `tweaker_config.json` settings. `max_keys_per_proposal` is the previously-hard-coded 3 lifted to configurable; the conservative-bias safeguard is preserved as a default of 3.

Status polling

`lblTweakerStatus` updates on:

1. Subscribe to `MainForm.AnalysisCompleted` event (raised by `RunAnalysisAsync` at the end of every analysis whether successful or skipped). Update on event.
2. 30-second `System.Windows.Forms.Timer` fallback when the dialog is open.

`UpdateStatusLabel()` reads `state.json` and inspects current CSV row count + session alignment + tier-eligible counts. Cheap I/O.

Outcomes (state.json last_run_outcome field)

- BELOW_THRESHOLD — engine performing fine, no tweak needed (happy state)
- INELIGIBLE — cooldown / session-not-aligned / insufficient tiers
- DRY_RUN_WRITTEN — payload file generated, awaiting manual handling
- PROPOSED — diff parked, awaiting manual apply (auto_commit off)
- APPLIED — settings updated, version bumped, change_log appended
- ERROR — API call or validation failure

Linux portability

tools/AutoTweaker/AutoTweaker.vbproj targets net8.0 (no -windows suffix). Zero WinForms references — confirmed by build inspection. Runs unmodified under dotnet AutoTweaker.dll on Linux. Future port: same codebase, scheduled via cron or systemd timer instead of WinForm Process.Start.

Constraints from trader-profile

The auto-tweaker is bounded by the same conservative-bias rules as manual tuning:

- No false-positive growth — auto-tweaker should optimise toward maintaining or *raising* the false-positive bar
- No double-counting reintroduction — the rejected-pattern list catches the obvious cases; reviewer (human if auto_commit = false) catches subtler ones
- Per-proposal key cap (default 3, configurable via max_keys_per_proposal) is the conservative-bias safeguard at the structural level — small steps, frequent review. Reverts bypass the cap because the snapshot's provenance is the validation gate (§20g).
- Latest-Opus auto-discovery means the tuning quality scales with model improvements over time without code change

20. Settings Snapshot History

When the auto-tweaker has run for several consecutive BELOW_THRESHOLD rounds without proposing a change, the engine treats those settings as “proven” under the current market regime and saves a snapshot. Snapshots are bucketed by regime × volatility tier (12 buckets) and one is kept active per bucket — the highest-scoring one stays, the rest are rotated out.

20a. Concepts

Term	Meaning
Round	One auto-tweaker invocation that produced an evaluable outcome (BELOW_THRESHOLD, APPLIED, PROPOSED, DRY_RUN_WRITTEN). INELIGIBLE and ERROR are NOT rounds.
Successful round	A round whose outcome is BELOW_THRESHOLD.
Streak	Consecutive successful rounds. Resets to 0 on any change-triggering outcome. Persisted in state.json.
Streak X	The configurable threshold for snapshot creation (snapshot_streak_x, default 3).
Active snapshot	The snapshot file representing the current settings during an ongoing streak. Status flips to ACTIVE in the manifest on creation, ROTATED when superseded by a higher-scoring snapshot in the same bucket.

Term	Meaning
Condition bucket	A categorical key for snapshot retention: regime × volatility tier . 12 buckets total.

20b. Streak tracking

AutoTweakerCore increments `state.CurrentBelowThresholdStreak` after each evaluable round:

- **BELOW_THRESHOLD**: `streak += 1`. When `streak == snapshot_streak_x` and no active snapshot exists, `SnapshotManager.Create()` fires. When the streak grows past X, `SnapshotManager.AccumulateConditions()` re-extracts the conditions vector across the full streak window and updates the manifest row in place.
- **APPLIED / PROPOSED / DRY_RUN_WRITTEN**: if an active snapshot exists, `SnapshotManager.Finalise()` populates `FinalisedIso` (the **last successful round's timestamp**, not this interrupting round's timestamp) and runs the bucket-rotation check. Streak resets to 0.
- **INELIGIBLE / ERROR**: no-op. Streak unchanged. Engine restart preserves the streak via `state.json`.

20c. Snapshot creation trigger

When the streak first hits `snapshot_streak_x`:

1. Copy `settings.json` verbatim to `settings_snapshots/settings_snapshot_<yyyyMMddHHmmss>.json`. No wrapper, no metadata — the file is a directly-applicable `settings.json`.
2. Compute the initial conditions vector via `ConditionsExtractor.Extract()` over the streak's CSV row range.
3. Append a manifest row with `Status = ACTIVE` and the full conditions vector. `FinalisedIso` stays empty until the streak is interrupted.

20d. Conditions vector

Stored in the manifest CSV (`settings_snapshots/manifest.csv`):

Field	Source
RegimeMix	Distribution of Regime column across the streak (pipe-format e.g. <code>UP:25 DN:10 RB:60 TR:5</code>).
AtrScaleAvg / AtrScaleMin / AtrScaleMax	Aggregates of the ATRMultiplier column (col 65).
FundingMin / FundingMax	Range of the FundingRate column (col 38).
NetPriceMovePct	$(\text{lastPrice} - \text{firstPrice}) / \text{firstPrice} \times 100$ over the streak window.
VolumeRatioAvg	Average of the VolumeRatio column (col 19).
VerdictTierMix	Distribution of Verdict column across the seven tier codes (SL/L/WL/NT/WS/S/SS).
VWAPDevAvg / VWAPDevMin / VWAPDevMax	Aggregates of <code> VWAPDevPct </code> (col 21).
SpreadRegimeMix	SpreadBps (col 69) bucketed by <code>cfg.Indicators.Spread.{Tight,Wide}ThresholdBps</code> into T / N / W.
OFIImbalanceMix	OFSignal (col 47) counted into BD / SD / BAL.
ConditionBucket	<code>{Regime}_{VolatilityTier}</code> where vol tier is LOW (<0.85), NORMAL (0.85..1.15), HIGH (≥1.15) of AtrScaleAvg .
AvgFailureRatePct	Average of round-level AggregateFailureRatePct across the streak.

The conditions are re-extracted on each accumulation and on finalisation — cleaner than incremental updates, cost is sub-millisecond for typical 360-row streaks.

20e. Composite score and bucket rotation

The composite score is used to compare snapshots within a bucket:

$\text{StreakLengthClamped} = \min(\text{StreakLength}, \text{StreakLengthClamp})$
 $\text{Score} = (100 - \text{AvgFailureRatePct}) + \text{StreakLengthClamped} * \text{StreakWeight}$

Defaults: $\text{StreakWeight} = 1.5$, $\text{StreakLengthClamp} = 20$. Failure rate dominates (1 point per percentage point); streak adds a secondary nudge (1.5 per round, capped at 20).

Worked examples:

Snapshot	Fail %	Streak	Score	Notes
A	25%	5	82.5	Moderate both
B	35%	10	80.0	Higher fail, long streak
C	20%	8	92.0	Low fail, decent streak
D	15%	3	89.5	Lowest fail, short streak
E	40%	30 (→ 20)	90.0	Higher fail, very long streak

This exact formula must remain identical in CompositeScorer.vb, the PromptBuilder system message, and this section. Any change here requires the same edit in those two locations.

Rotation rule (run on each finalisation):

- If no existing **ACTIVE** snapshot in the same bucket, the new snapshot stays **ACTIVE**.
- Else, compute both scores fresh and compare:
 - New > existing → existing is marked **ROTATED** (reason = "superseded by <new.Filename>"), its .json file is deleted (manifest row retained as historical record). The new snapshot stays **ACTIVE**.
 - Otherwise → the new snapshot is immediately marked **ROTATED** (reason = "score X <= existing Y"), its .json file is deleted.

20f. Manifest schema

Single CSV at settings_snapshots/manifest.csv (gitignored). One row per snapshot. Columns:

Filename, CreatedIso, FinalisedIso, StreakLength, AvgFailureRatePct, RegimeMix, AtrScaleAvg, AtrScaleMin, AtrScaleMax, FundingMin, FundingMax, NetPriceMovePct, VolumeRatioAvg, VerdictTierMix, VWAPDevAvg, VWAPDevMin, VWAPDevMax, SpreadRegimeMix, OFIImbalanceMix, ConditionBucket, Status, RotationReason

Status is **ACTIVE** or **ROTATED**. RotationReason is empty unless rotated. Fields with embedded commas are RFC-4180 quoted.

20g. Revert mechanism

When a change-triggering round fires, the prompt to Claude is extended with the **ACTIVE** manifest rows and the current conditions vector. Claude may respond with either:

- action="tweak" — a normal diff (subject to the max_keys_per_proposal cap).
- action="revert" with a revert_target filename — a wholesale replacement from a past snapshot.

For a revert:

1. `SettingsDiffApplier.ApplyRevert()` is invoked. It validates that `revert_target` matches an ACTIVE manifest row and the snapshot file exists on disk.
2. The snapshot content is run through the **same rejected-pattern / disabled-gate validation** as a normal diff (banned fragments like `_fixed_pct_`, disabling `mtf_gate.enabled`, etc.) — snapshot integrity is not absolute trust.
3. The diff-scope cap is **NOT** applied to reverts — by definition a revert is many keys at once. The snapshot's provenance (a proven-successful streak) is the validation gate.
4. `settings.json` is replaced with the snapshot content, `version` bumped, `modified_by` = "auto-tweaker-revert", and a `change_log` entry is appended citing the snapshot filename + reasoning excerpt.

Reverts honour the `auto_commit_enabled` toggle the same way tweaks do — when off, the revert is written to `tools/AutoTweaker/proposed_diffs/<timestamp>_revert.json` for manual review.

20h. Round Statistics display

The Tweak Settings dialog exposes a `Show Round Stats` button that opens `RoundStatsForm` (non-modal). The form renders the last 5 rounds in `state.RoundHistory` with:

- Round timestamp + outcome
- Aggregate failure rate for the window
- For APPLIED / PROPOSED / DRY_RUN_WRITTEN rounds: the diff summary and a reasoning excerpt
- For each round, per-tier accuracy using the v2 barrier-hit semantic from `FailureRateMatrix.WalkBars`. This panel covers **all directional verdicts** (STRONG_LONG / LONG / WEAK_LONG / STRONG_SHORT / SHORT / WEAK_SHORT) — not just the tier-eligible subset used by the failure-rate matrix. NO_TRADE rows are reported as an informational row-count only.

OHLC bars for the row windows are fetched via `DeribitOhlcFetcher` once per `Refresh`. The build is async; expected cost is well under a second for 5 rounds × 120 rows.

The form also subscribes to the `RoundStatsForm.Refresh` button — re-run any time to pick up newer rounds.

20i. State persistence

`state.json` (gitignored, in `tools/AutoTweaker/`) carries the snapshot-related state across runs:

- `current_below_threshold_streak` — running streak counter
- `active_snapshot_filename` / `active_snapshot_created_iso` — pointer to the current ACTIVE snapshot
- `last_successful_round_iso` — timestamp of the most recent BELOW_THRESHOLD round, used to populate `FinalisedIso`
- `round_history` — last 50 `RoundSummary` entries (older entries dropped on save)

21. Live Performance Display

A compact strip of six labels positioned on the auto-run row (to the right of the Start/Stop button) that shows how the engine's directional verdicts have been performing, updated after every analysis run. The strip is read-only and observational — it does not feed into scoring or auto-tweaker decisions.

21a. Concepts

Six time windows:

Label	Window definition
<code>Cur.Wk</code>	Monday 00:00 UTC+8 → now

Label	Window definition
3d	D-2 00:00 UTC+8 → now
Cur.Day	Today 00:00 UTC+8 → now
Asia	Most-recent Asia block — 08:00–15:00 UTC+8
London	Most-recent London block — 16:00–20:00 UTC+8
NY	Most-recent NY block — 21:00–07:00 UTC+8 (midnight straddle)

Success rate = $\text{SUCCESS_count} / (\text{SUCCESS_count} + \text{FAILURE_count}) \times 100$, expressed as an integer percent (e.g. 57%). PENDING rows and EXCLUDED rows do not enter the denominator.

Sample-size threshold. When fewer than `min_sample_for_render` evaluable predictions exist in a window, the label shows --% (uncoloured). Default threshold: 4. Configurable via `performance_display.min_sample_for_render`.

21b. Most-recent-block algorithm

Each session label covers the most recently active or completed block for that session, regardless of calendar date. There are three states:

State	Displayed window
now is inside the session hours	Session start today → now (partial, growing)
now is after today's session	Today's full session block (start → end)
now is before today's session	Yesterday's full session block

The NY session straddles midnight UTC+8 (21:00–07:00). Three sub-cases:

- **Hour 21:00–23:59 UTC+8** — inside NY head. Block = today 21:00 → now.
- **Hour 00:00–06:59 UTC+8** — inside NY tail. Block = yesterday 21:00 → now.
- **Hour 07:00–20:59 UTC+8** — between sessions. Block = yesterday 21:00 → today 07:00 (completed).

Worked examples (UTC+8):

- **02:30 UTC+8** — Asia: yesterday 08:00–15:00 (completed). London: yesterday 16:00–20:00 (completed). NY: yesterday 21:00 → 02:30 (running, partial).
- **10:00 UTC+8** — Asia: today 08:00 → 10:00 (running). London: yesterday 16:00–20:00. NY: yesterday 21:00–07:00 today (completed, 10h block).
- **17:30 UTC+8** — Asia: today 08:00–15:00 (completed). London: today 16:00 → 17:30 (running). NY: yesterday 21:00–07:00 (completed).
- **22:00 UTC+8** — Asia: today 08:00–15:00. London: today 16:00–20:00. NY: today 21:00 → 22:00 (running).

The tooltip on each label confirms the exact range and sample size.

21c. Success metric — v2 barrier-hit

Reuses `FailureRateMatrix.WalkBars` from the v2 failure-definition spec.

Eligible verdicts: STRONG LONG, LONG, WEAK LONG, STRONG SHORT, SHORT, WEAK SHORT. NO TRADE and NO TRADE [WEAK X] are not counted.

Excluded rows (omitted from numerator and denominator): - $\text{ATR} \leq 0$ — degenerate barriers. Marked EXCLUDED_ATR_INVALID. - Non-directional verdicts. Marked EXCLUDED_NO_PREDICTION.

Favourable barrier: - LONG direction: $\text{AdjustedLongTarget}$ if > 0 , else $\text{entry} + 2.0 \times \text{ATR}$. - SHORT direction: $\text{AdjustedShortTarget}$ if > 0 , else $\text{entry} - 2.0 \times \text{ATR}$.

Adverse barrier: - LONG: SwingStopLong if > 0 , else entry - $1.2 \times \text{ATR}$. - SHORT: SwingStopShort if > 0 , else entry + $1.2 \times \text{ATR}$.

v51 note — eval yardstick vs displayed levels (D6). These fallback multipliers (favourable $2.0\times$, adverse $1.2\times$ — $\text{FAV_ATR_MULT} / \text{ADV_ATR_MULT}$) were deliberately **left unchanged** when v51 moved *placement* to $1.75\times$ target / $1.6\times$ stop, so post-v51 failure rates stay comparable to the historical book. Two consequences: (1) on a fallback row the eval's favourable barrier ($2.0\times$) is wider than the displayed target ($1.75\times$); (2) the adverse barrier uses the **raw** swing stop, not the clamped $\min(\text{swing}, 1.6\times\text{ATR})$ stop the app shows. So don't read the perf-strip win/loss as measured against the exact levels displayed on a fallback row. Known bounded gap; migrating the eval onto the logged `Placed*` columns is a scheduled follow-up (roadmap Q8 / D6).

Eligible bars: T+3 min through T+15 min (13 bars) on 1-min rows, skipping T+1 and T+2; **3-min-session rows walk T+3 through T+45** (the same 5/10/15-bar windows scaled — post-v41 recalibration, $\text{EvalHorizonMinutes}(\text{execResolution})$), so their outcomes display ~45 min after the verdict. Bars are identified by `CloseTime` in the OHLC cache ($\text{CloseTime} = \text{openTime} + 1 \text{ min}$); barrier detection always walks 1-min bars — only the window length scales.

Classification: - SUCCESS — favourable wick hit before adverse hit. - ADVERSE_HIT — adverse hit before favourable hit → FAILURE. - AMBIGUOUS — both barriers hit in the same bar → FAILURE (conservative-bias rule). - WINDOW_EXPIRED — neither barrier hit by T+15 → FAILURE.

21d. Cache architecture

Two sidecar files in the same directory as `analysis_log.csv` (`bin/Debug/net8.0-windows/` or `bin/Release/...`). Both are gitignored.

`analysis_eval_cache.csv` Schema comment: # schema=v1 (live-performance-display)

Columns: `Timestamp`, `Verdict`, `EntryPrice`, `FavBar`, `AdvBar`, `EvalOutcome`

- One row per analysis run.
- `EvalOutcome` values: SUCCESS, ADVERSE_HIT, AMBIGUOUS, WINDOW_EXPIRED, EXCLUDED_NO_PREDICTION, EXCLUDED_ATR_INVALID, PENDING.
- PENDING rows are resolved in-place when their 15-min window completes. The file is rewritten when any PENDING row resolves (read-all → modify → write-back).

`ohlc_1m_cache.csv` Schema comment: # schema=v1 (1m ohlc cache)

Columns: `CloseTime`, `Open`, `High`, `Low`, `Close`, `Volume` (Volume = 0 placeholder).

- Rolling 7-day window: $\text{cap} = 10,080$ bars ($7 \times 24 \times 60$).
- Trim fires when the in-memory count exceeds $\text{cap} \times 1.05$ (~10,584), batching disk writes to every ~500 bars rather than every bar.

21e. Cold-start backfill flow

On engine startup (when `eager_backfill_on_startup = true`):

1. **OHLC cache check.** If `ohlc_1m_cache.csv` exists and its newest bar is within 7 days, load it and fetch only the gap (last bar → now) via `DeribitClient.GetCandlesAsync`. Otherwise fetch the full 7-day range (~10,080 bars). Typical cost: 1–3 seconds.
2. **Eval cache backfill.** Walk `analysis_log.csv`. For each row not already in the eval cache, compute barriers and evaluate or mark PENDING.
3. **Initial display.** Once backfill completes, all 6 labels populate from the in-memory eval cache.

4. **Status.** `lblLogInfo` shows “Loading performance history...” during backfill. Controls remain responsive — backfill runs on a background thread. New analysis runs wait for backfill to complete via an internal init gate (typically < 5 seconds).

On subsequent launches the cached files load in under 1 second and the gap fetch covers only the time since the last save.

21f. Per-analysis update flow

After each `RunAnalysisAsync` completes (after `RenderOutput` and `AnalysisOutputDump.Append`):

1. **Append new OHLC bars.** Walk `candles1m` from oldest to newest. Bars with `CloseTime > cache_max` are appended to disk and added to the in-memory dictionary. Typically 1 new bar per analysis run.
2. **Append current verdict as PENDING.** Compute `FavBar` and `AdvBar` from live `VerdictResult` and `IndicatorResults`. Write the row with `EvalOutcome = PENDING`.
3. **Resolve PENDING rows.** Find all eval-cache rows with `EvalOutcome = PENDING` where `Timestamp + 15 min ≤ nowUtc`. For each, call `WalkBars` against the in-memory OHLC cache. At most 1 row resolves per run (the row from ~15 min ago). Rewrite the eval cache file.
4. **Recompute 6 window aggregates.** Pure in-memory scan; no additional I/O. Typical cost: < 5 ms.
5. **Update labels.** `UpdatePerformanceLabels()` applies colours and tooltip text on the UI thread.

21g. Render rules and tooltips

Label text: - Sample size ≥ threshold: “{prefix}: {rate}%” — e.g. Asia: 64% - Sample size < threshold: “{prefix}: --%” — e.g. Asia: --%

Colour: - Rate > 50% → C_GOOD (green, #50DC78) - Rate ≤ 50% → C_BAD (red, #FF5050) - --% → C_DIM (dim grey, #646464)

Tooltip on hover: “{n} predictions evaluated. {start} → {end} UTC+8.”

Example: 12 predictions evaluated. 2026-05-13 08:00 → 2026-05-13 10:45 UTC+8.

21h. Settings reference

Block: `performance_display` in `settings.json`.

Key	Type	Default	Description
<code>enabled</code>	bool	<code>true</code>	Master switch. <code>false</code> = strip hidden, no cache I/O.
<code>min_sample_for_render</code>	int	4	Minimum evaluable rows before showing a numeric rate.
<code>eager_backfill_on_start</code>	bool	<code>true</code>	Fetch 7-day OHLC gap and backfill eval cache on launch.
<code>session_block_semantic</code>	string	<code>"most_recent"</code>	Reserved. Currently always <code>most-recent-block</code> (§21b).

Added in `settings.json` v26 (`modified_by = "live-performance-display"`).

22. WebSocket Health Status Line

The engine cut over from REST polling to a live WebSocket feed in v42 (`network.transport = "ws"`). A health segment in the status-bar log line (`lblLogInfo`, built by `BuildWsStatusSegment` in `UI/MainForm_Layout.vb`) reports feed state on every run.

Not a card or snapshot surface — this is a live in-memory status-bar element, like the perf strip. It is not part of the RTF/card verdict output, is not emitted by `BuildPlaintextSnapshot`, and is not logged to CSV or the output dump (per the CLAUDE.md engine display-string parity rule — no card-binding obligation).

States

Format	Meaning
WS OK · 1/3/5/15 fresh · trades N	Connected; all candle series (1m/3m/5m/15m) fresh within <code>network.ws_stale_after_sec</code> ; N recent trades buffered. Normal operating state.
WS OK · streams Xs stale · trades N	Connected but the last frame is older than the freshness threshold — typically a quiet market, not a fault.
WS DEGRADED – REST fallback (stream stale)	The WS stream was stale at run time; <code>RunAnalysisAsync</code> used <code>RestMarketDataSource</code> for that one run rather than skip it. The CSV row for that run is REST-sourced but otherwise normal.
WS DOWN – reconnecting (Xs backoff, R reconnects)	Disconnected. Reconnect loop running 1→60s exponential backoff; R is the cumulative reconnect attempt count. The app keeps functioning via per-run REST fallback while down.

Calculation

`WsMarketDataSource` / `DeribitWsFeed` expose `IsConnected`, `IsCoolingDown`, `ReconnectCount`, `CurrentBackoffSec`, and per-series freshness (`IsFresh`, gated on `network.ws_stale_after_sec`). `BuildWsStatusSegment` composes the line from these each time the log info cascade renders; it returns an empty string when no WS feed is active (`transport=rest` and `shadow_parity=false`), so the segment disappears cleanly on a pure-REST configuration.

Interpretation

- A flicker into DEGRADED once in a while is normal network jitter — the per-run fallback means it never costs you a row or a skipped analysis.
- Persistent DOWN with a climbing reconnect count means something is actually wrong with connectivity (or Deribit's WS endpoint) — the engine is still running on REST fallback, but you're not getting the latency benefit of the live feed, and the Exit Guard / TAPE strip (both WS-only) will show "WS only" / be inert until it recovers.
- This line is the fastest way to sanity-check feed health before trusting a run that looks unusual.

23. Realtime Exit Guard

Shipped v43 (`exit_guard` settings block). While a position is declared (`posState ≠ None`), a WinForms timer re-evaluates the **fast** microstructure exit conditions every `exit_guard.interval_sec` (default 3s) against the live WS `MarketState` — independent of, and faster than, the normal auto-run cadence.

Display/alert only. It never calls `Calculate()`, never writes a CSV row, never changes the verdict. **No card-binding obligation** — like the WS-health line, it's a status-bar element, not part of the RTF/card/snapshot surface.

Mechanism

The same fast-exit logic `CalcHoldStatus` uses for its Layer 1 (2+ adverse microstructure signals) and Layer 1.5 (structural swing breach) checks is shared via `ScoringEngine.ComputeFastExitPrimitives` — extracted so the guard and the full pipeline can never drift on “what counts as adverse.” `ExitGuardEvaluator.vb` recomputes MicroCVD/TFI/OFI/CVD straight from `MarketState` using the same indicator functions, config params, and 500-trade window as a full run — just fresher.

Gate (must all be true to render/evaluate): `exit_guard.enabled`, a position is declared, and the WS feed is connected, not cooling down, and not stale. The strip is paused — not silently stale — when the feed is down.

States (three only — no “Warn” tier)

Display	Meaning
EXIT GUARD · clear	No adverse condition present.
EXIT GUARD · Δ EXIT? confirming n/d	An adverse condition is present but hasn't held for <code>exit_guard.debounce_evals</code> consecutive checks yet (default 2). A single adverse signal alone maps here, not to a separate warn state — by design (coordinator ruling D3, 2026-06-25).
EXIT GUARD · Δ EXIT - <reason>	Latched. <reason> is the same adverse-signal list / break-level text <code>CalcHoldStatus</code> would produce. An alarm sound (<code>System.Media.SystemSounds.Exclamation</code>) fires on this transition if <code>exit_guard.sound_enabled</code> . Auto-clears and re-arms once the condition clears for <code>debounce_evals</code> consecutive checks.

Interpretation

- Read it the same way as a HOLD/EXIT fast-exit line — it's the identical condition, just checked every few seconds instead of once per analysis run.
- **confirming n/d** is informational, not actionable yet — wait for the latch or for it to clear.
- A latched EXIT with an alarm means act; this is the fastest exit cue the engine produces.
- WS-only (`transport=ws`) — shows “WS only” / hidden at `transport=rest`, since the guard reads `MarketState` directly rather than fetching its own snapshot.

24. On-Close Trigger Mode

Shipped v44 (`auto_run.trigger_mode`). A pure run-**trigger** change — `RunAnalysisAsync / Calculate()` are byte-identical; only *when* a run fires moves.

Modes

- **interval** (default) — the existing fixed-interval timer.
- **on_close** — the full analysis run fires the instant the execution-resolution bar closes (`Core/BarCloseDetector.DetectBar` watching `MarketState`'s forming-bar open-time at ~1s granularity) — NY's 1-minute bar, Asia/London's

3-minute bar (v36 resolution). Eliminates up-to-one-interval poll lag at the bar-close decision moment a structural-breakout trader actually acts on.

On-close does not drop the forming bar. The verdict at an on-close fire is exactly what an interval-timer fire at that same instant would have produced — same pipeline, same data, different trigger timing only.

UI

- A radio toggle (INTERVAL / ON-CLOSE) sits beside the SINGLE/REPEAT cluster.
- In on-close mode, the interval NUD's label relabels AUTO EVERY → BACKSTOP — the interval value still applies, but only as a feed-stall safety net (`now - lastFire ≥ interval` triggers a run even if no bar-close was detected, so a stalled WS feed never goes silent).
- **Next close:** M:SS [SINGLE <res>m] / [REPEAT <res>m] — countdown to the next execution-resolution bar boundary; the bracket echoes the SINGLE/REPEAT run mode and the resolution (1 NY, 3 Asia/London) the countdown is timed against.
- A multi-bar gap (e.g. after a reconnect) produces a single catch-up fire, not a burst of backlogged runs.
- Session boundary changes (execution resolution 3↔1 switching between Asia/London and NY) are re-resolved on every tick, so the watcher adopts the new resolution and fires cleanly on the first roll under it.

REST fallback

`on_close` mode requires the live WS bar stream for roll detection. At `transport=rest`, or with no `MarketState`, the engine falls back to interval mode for the session (status note [`on-close: WS only`]) rather than going silent.

Interpretation

On-close mode is the cleaner choice if you trade off bar closes (the structural-breakout style this engine is built for) — it removes the “verdict is already a few seconds stale relative to the close” lag that a 10–60s interval timer carries. Interval mode remains useful for a steady, predictable polling cadence (e.g. for calibration/data-collection runs where a fixed cadence is easier to reason about statistically — see §12 in the project handover on cadence as a calibration dimension).

25. Live Microstructure Strip (TAPE)

Shipped v45 (`live_strip` settings block). A continuously-updating one-line strip showing fast streaming microstructure *between* full analysis runs, refreshed every `live_strip.refresh_sec` (default 2s).

Deliberately not a verdict. It never calls `Calculate()`, writes the CSV, or emits a direction/score. Visually distinct from the verdict colour ramp (neutral/dim) so it reads as a raw readout, not a call. **No card-binding obligation** — same class as the WS-health line and Exit Guard strip.

Format

76038 · SL 75920 (−118) | SH 76210 (+172) · TFI BUY +0.42 · 1.8 bps · book 2.3× bid · 4.1 tr/s (\$312k/s)

Composed by `ComposeLiveStrip` in `UI/MainForm_LiveStrip.vb`:

- **Last price.**
- **Nearest structural levels** above and below price (SL/SH/HVN↑/HVN↓, with signed delta to current price) — **carried, not recomputed**, from the last full run's swing-pivot and VPFR-HVN levels (those are slow-moving; refreshing them every 2s would be noise). If price has broken through all carried levels on one side, only the nearest-above (or nearest-below) renders and the other bracket is genuinely empty until the next full run maps a level there — not a bug.

- **TFI** — recomputed live from the streaming trade buffer (BUY / SELL / NEUT + signed value).
- **Spread (bps)** — recomputed live.
- **Top-book imbalance** — $N.N \times \text{bid}$ or $N.N \times \text{ask}$, recomputed live.
- **Tape speed** — trades/sec and \$/sec over a short rolling window (`Live_strip.tape_window_sec`, default 10s).

Any field with no data yet renders -- rather than a stale or zero value.

Toggle

A visible **TAPE** checkbox (mirrors the SINGLE/REPEAT and INTERVAL/ON-CLOSE toggles) writes `Live_strip.enabled` live. The strip is **always-on when enabled** — not gated on having a position declared, since it's equally useful flat (watching a level for entry) or in a hold.

REST fallback

Requires the live WS feed (`MarketState`) to be enabled, connected, and fresh. The checkbox is labelled TAPE; the data label beside it reads `WS only` (not stale numbers) whenever the feed isn't usable — `transport=rest`, no feed, or the feed present but not yet connected/fresh. Confirmed live: in a sandboxed/no-network environment the strip correctly shows `WS only` rather than rendering anything stale.

Interpretation

Read this as the same raw inputs the verdict pipeline consumes, just faster and unfiltered — useful for watching a level develop between runs, but it is explicitly **not** a substitute for the full verdict. The multi-indicator pipeline exists precisely because single-glance microstructure reads (a TFI flicker, a one-sided book) are noisy in isolation; don't let the TAPE strip tempt a marginal entry the full run wouldn't support.

26. What-If Replay (Backtesting)

Shipped 2026-07-16 (`tools/WhatIfRunner`, launcher UI/`WhatIfLauncherForm.vb`). An **offline backtesting instrument**: it takes the runs the engine has already logged (`analysis_log.csv`, v0.8+ rows) plus 1-minute price history, applies a **hypothetical settings change** (an “overlay”), and re-computes — on the exact same historical rows — what levels the engine *would* have placed and what verdict it *would* have called under that change. It then re-walks each trade's outcome against the price that actually followed and prints a **baseline-vs-overlay** report: current settings on one side, your hypothesis on the other.

It answers what the calibrate-forward loop can't answer quickly — “*would a wider LONDON stop have helped on the book so far?*”, “*is the MEDIUM threshold too tight?*” — as a repeatable, guard-railed instrument rather than a one-off hand analysis.

Where: SETTINGS & TOOLS card → **What-If Replay** link → a non-modal dialog. The report opens in the same **Analysis Report Viewer** (§18).

What it is NOT. It never changes the engine: it does not write `settings.json`, place orders, or touch the live verdict. A promising result is evidence for a *spec proposal* — the normal spec-first + test + own-watch discipline still gates any real change. Wind tunnel, not aircraft.

Reuse, not a copy. Placed levels come from the shipped `SignalEmitter.ComputeSideLevels` (the same arbitration the live card, snapshot, bridge payload and CSV `Placed*` columns use), and outcomes from the shipped `FailureRateMatrix`. The replay geometry therefore cannot drift from production — it *is* production. The baseline column is the replay under **live** settings, so it reproduces the standing failure matrix for the span.

The fields

Every field maps to one engine setting (except the last three, which are backtest-only). Blank = “use whatever the engine runs live”, so you only fill what you want to test. Setting paths below are all under **scoring**; the value in brackets is the current live setting.

- **ATR target mult** — `atr_target_multiplier` (1.75). The **fallback** take-profit distance ($\times$ ATR), used only when no structural target qualifies. *Seen in:* ATR Entry Levels → TARGET on FALLBACK_ATR rows, and the header’s ...x target.
- **ATR stop mult** — `atr_stop_multiplier` (1.6). The **fallback** stop distance ($\times$ ATR), used only on rows with no usable structural stop. **Not** the clamp ceiling — that’s *Stop max* $\times$ ATR. *Seen in:* ATR Entry Levels → STOP on [FALLBACK_ATR] rows, and the header’s ...x stop.
- **Verdict STRONG %** — `verdict_strong_pct` (0.70). Fraction of the regime max at/above which the call is STRONG. *Seen in:* the Verdict headline and the Score gauge tier.
- **Verdict MED %** — `verdict_med_pct` (0.53). Threshold for a MEDIUM (LONG / SHORT) call. *Seen in:* the Verdict headline.
- **Verdict WEAK %** — `verdict_weak_pct` (0.35). Threshold for WEAK LONG / WEAK SHORT; below it the dominant side is NO TRADE. *Seen in:* the Verdict headline.
- **Min net move % (after fees)** — `trade_costs.min_net_move_pct` (0.0005). **[v62]** The smallest take-profit distance a directional call must clear on its **placed** target, measured *after* execution costs. The gate’s actual floor is this plus the round-trip fee (0.0003 at the default maker/maker 1.5 bps), i.e. 0.0008 — so sweeping this knob sweeps the whole floor, one-for-one. The fee keys themselves are deliberately not sweepable. *Seen in:* Verdict flipping to NO TRADE, with Context BELOW_MIN_MOVE.
- **Target max $\times$ ATR** — `structural_levels.target_max_atr_mult` (3.5). Looseness bound: a structural target places only within this $\times$ ATR of entry. *Seen in:* ATR Entry Levels → whether TARGET reads PLACED @ ... or FALLBACK_ATR, and the Structural · 5m Pivots rows.
- **Stop max $\times$ ATR** — `structural_levels.stop_max_atr_mult` (1.6). The stop **clamp ceiling**: placed stop = min(structural swing stop, this $\times$ ATR). *Seen in:* ATR Entry Levels → the STOP label, SWING_STOP vs STOP_CLAMPED.
- **Stop min floor ticks** — `structural_levels.stop_min_floor_ticks` (4). Degenerate-tightness floor (\$0.5 a tick) — a structural stop tighter than this falls back to ATR. *Seen in:* ATR Entry Levels → STOP (rare).
- **NY / LONDON / ASIA fallback $\times$ ATR** — the `fallback_target_atr_mult` key under `structural_levels.sessions.<NAME>` (NY inherits 1.75, LONDON 2.0, ASIA 1.25). Per-session override of the fallback target multiplier. *Seen in:* the ATR Entry Levels header and TARGET on FALLBACK_ATR rows **in that session only**.
- **Use best-pivot candidate** — `structural_levels.use_best_pivot_candidate` (0). **[v63]** A Boolean entered as 0 / 1: switch the volume-weighted best 5m pivot into the placed-target candidate set — first tier above swing under the shipped ladder arbitration (§2, §10). 0 is **what the engine runs live**, so 0:1:1 is the whole useful sweep. *Seen in:* ATR Entry Levels → a [PLACED @ ... (BEST_PIVOT_5M)] target row, a label that never appears on a live run because the key is off.
- **Eval window (bars)** — *backtest only*. How many bars forward the replay walks to score each trade for the **EV ranking** (5/10/15, scaled by resolution). No app element.
- **Constraints** — *backtest only*. Optional rules that prune grid combinations before they run. No app element.
- **From / To** — *backtest only*. Limits which logged rows the replay covers (yyyy-MM-dd); blank = the whole book. No app element.

Field nuances — read before you sweep

Several fields don’t behave the way the surface reading suggests.

Verdict STRONG / MED / WEAK %. The threshold is `⌈maxScore × pct⌉` — **rounded up to a whole score** — and the test is `effectiveScore ≥ threshold`. So 70% of a max-15 regime is `⌈10.5⌉ = 11`, not 10.5: a score of 11 is STRONG, 10 is not. And the max isn’t fixed — it’s the **regime’s** ceiling:

Regime	Max	STRONG ($\geq$)	MED ($\geq$)	WEAK ($\geq$)
TRENDING	20	14	11	7
RANGE_BOUND	19	14	11	7
TRANSITIONAL	15	11	8	6

(The / 20 on the Score gauge is the trending max.) Consequences: **sub-step no-ops** — a % change that doesn't cross a `max_pct` boundary yields the same threshold and an identical cell (sweeping MED % 0.50:0.56:0.01 on a max-19 regime gives threshold 10 for 0.50–0.52 and 11 for 0.53–0.56: seven inputs, two real outcomes). **Keep them ordered** (strong $\geq$ med $\geq$ weak) — the tier walk checks STRONG → MED → WEAK. And what's compared is the **dominant side's effective score** (post regime/MTF adjustment); the overlay re-tiers logged scores, it never rebuilds the raw Step-2 tally.

ATR stop mult vs Stop max ×ATR — the one to internalise. Both default to 1.6, but they hit different rows. **Stop max ×ATR** is the clamp ceiling; most 5m swing stops run wider than it, so most structural rows come out STOP_CLAMPED at this value — **it moves the stop on the majority of directional rows**, the high-leverage lever. **ATR stop mult** is the fallback stop, used only where no structural stop exists — a minority. Sweeping it and seeing little change is expected, not a null result. To test “wider stops,” sweep **Stop max ×ATR**.

ATR target mult. Same fallback caveat — it sets the target only where no swing/HVN qualifies. In a structured tape most targets sit on structure, so its footprint is narrow; the session overrides are where the fallback target is actually tuned.

Target max ×ATR. An *eligibility bound*, not a level. It decides whether a structural target may place, never where it sits. Raising it lets farther structural targets place (structure wins even when farther than the ATR fallback, up to the bound); lowering it pushes rows onto the fallback. Its real effect is the **structural-vs-fallback mix**, which shifts R:R and can trip the min-move gate.

Stop min floor ticks. A guard, not a lever — structural stops are almost never that tight, so **expect a sweep to show ~no change**.

NY / LONDON / ASIA fallback ×ATR. Session-scoped *and* fallback-only. NY's value touches only NY×1 rows, LONDON's only LONDON×3, etc. Watch what blank inherits: NY blank → the global 1.75, but **LONDON blank → 2.0 and ASIA blank → 1.25** (those overrides are live). To test “no session override,” pin the field to the global — don't blank it. A corollary: sweeping the *global* ATR target mult moves NY only, because LONDON/ASIA are shielded by their own overrides.

Min net move % (after fees). A fraction of **price**, not ATR (the composed floor 0.0008 ≈ \$49.6 at \$62k; auto-scales with BTC). Checked against the **placed** target, so a near swing target trips it just as a small ATR target does. Coupled to the geometry knobs: lower MED % to add trades and see the population barely grow, and the min-move gate is usually vetoing the small ones. [v62] Because fees are fixed during a sweep, a **min_net** grid IS a floor grid shifted by the round-trip cost — 0.0003:0.0009:0.0002 here is the old 0.0006:0.0012:0.0002 floor sweep.

Use best-pivot candidate. A Boolean swept as an integer — 0:1:1 gives exactly two cells, off and on. It is the only field on this form whose **on** state the live engine has never run: every other field sweeps around a shipped number, this one sweeps around a shipped **false**. It is gated by the same **Target max ×ATR** bound as every other structural tier, so a 0:1:1 sweep that moves almost nothing may mean the pivot rarely qualified *inside the bound* rather than that the tier is inert — put **Target max ×ATR** in the same grid if you want to separate those two readings. Note also that the tier's priority depends on `structural_levels.target_arbitration_mode`, which this form does **not** expose: under the shipped ladder mode (0) the pivot outranks swing whenever it qualifies.

Eval window (bars). Affects **only the EV ranking** used to pick a grid winner — not any placed level, verdict, or the failure matrix (which always reports all three windows). On a single-cell run it has no visible effect. Values are a bar-count budget scaled by resolution: 15 bars is 15 min on NY, 45 min on Asia/London.

How the grid works

Each field accepts one of three things, and that's the whole input language:

- **Blank** → inherit the live value.
- **A single number** (2.0) → **pin** that knob for the whole run.
- **A range** from:to:step (1.6:2.4:0.2) → **sweep** it (1.6, 1.8, 2.0, 2.2, 2.4).

The runner builds a **grid** — every combination of every field's values — and runs each combination ("cell") as a full backtest:

- **All blank or single** → **one cell**: the plain baseline-vs-overlay backtest. Test one concrete idea.
- **One field swept** → a **1-D sweep**: find the best value of one knob with everything else fixed.
- **Two or more swept** → a **grid**: every combination (sweep **MED** % over 4 values and **Min move** % over 4 → 16 cells). Explore how knobs interact.

Every cell is a **complete, runnable overlay** — the swept/pinned fields plus live values for the blanks — so a winner is always a full settings combination you could propose, never a bare number.

Sweep endpoints are from + k×step. 1.5:2.0:0.2 yields 1.5 / 1.7 / 1.9 — **2.0 is skipped** (the next step overshoots). Use 1.5:2.0:0.25 or 1.6:2.0:0.2 to land on it.

Prefill checkbox. "Prefill default sweep ranges" fills every knob with the sweep shown in its hint (and clears them when unticked), so you can start from the full set and edit down rather than retyping ranges.

Cap: 3,000 cells. Above that, compute climbs (the replay re-walks every row per cell — ~9 s at 1,000 cells, ~23 s at 3,000 on a ~4k-row book, growing with the book) and multiple-comparisons risk mounts. Narrow a range or pin a field. The ranking table shows only the **top 50 cells** regardless of grid size (the winner is always rank 1), so the report stays readable.

Constraints prune combinations before they run. To keep two knobs coupled while sweeping both:

```
[{"ratio": {
  "of": ["scoring.structural_levels.stop_max_atr_mult",
        "scoring.atr_stop_multiplier"],
  "min": 1.0, "max": 1.0
}]
```

That keeps only cells where the ratio is exactly 1.0 — turning a 9-cell square into the 3-cell diagonal.

From / To limit the rows by date; blank runs the whole book. A shorter span runs faster and lets you compare a recent regime against an older one.

How outcomes are scored, and how a winner is chosen

For each row the overlay makes tradeable, the replay re-replaces stop and target under that overlay, then walks the **actual** 1-minute bars that followed: target touched first → **+target distance**; stop (or both in one bar) first → **-stop distance**; neither within the window → marked to the price at the window's end. Distances are in **ATR units** so trades are comparable, and the average is the cell's **EV per trade in ATR** — the ranking objective. Win-rate is *not* used to rank (a 90%-win/tiny-target setup can lose money) and is context only.

Split-half validation (any swept grid): the winner is chosen on **half** the book (alternating session-days) then re-checked on the **unseen** half. If it falls apart there it's flagged **DIVERGENT** — the tool telling you the win is likely curve-fitting. **Trust the holdout, not the selection half.**

Matrix and EV answer different questions. The failure matrix uses a fixed k×ATR favourable barrier, so a target-multiplier change barely moves it (it shows up mostly as population change); the EV uses the *actual placed* target and stop, so it does. A wider stop can improve the matrix (fewer stop-outs) while *hurting* EV (bigger loss per stop-out). For a fixed-size trader **EV is the bottom line; the matrix is the mechanism.**

Reading the report

1. **Guard-rail banner** — the four binding cautions, including the overfit counter.
2. **Grid ranking** (swept grids) — cells by EV/trade in ATR with selection-half and holdout figures, DIVERGENT flags, then the winner's full effective overlay.
3. **Population shift** — per session, directional trades added/removed (**baseline** → **overlay**), plus the **BELOW_MIN** excluded tally. Verdict-threshold changes move this; pure geometry changes usually don't.
4. **Baseline vs overlay failure matrix** — per session × resolution × tier on the *same* rows: **SUCC%**, **ADV%**, **EXP%**, each with a 95% Wilson CI; cells with $n < 30$ flagged †.

The four binding cautions, printed on every report:

1. **It motivates, it isn't.** A result feeds a spec proposal; the runner never writes settings.
2. **Overfit.** The header states how many overlays have been tested against this span and roughly how many false winners to expect from noise. Treat single-cell wins on a big sweep with suspicion.
3. **Touch-based.** Barriers are mid-price wick touches — **no fills, slippage, or queue position**. Real execution is worse.
4. **POC rows excluded.** VPFR POC inputs aren't logged, so rows whose live target used the POC tier are excluded and counted in the header (near-zero in practice). Only v0.8+ rows are replayed.

Worked examples

- **Test one idea (single cell).** 2.0 in **Stop max ×ATR**, everything else blank, a recent span → baseline (1.6) vs 2.0 on the same rows; read the LONDON row.
- **Best value of one knob (1-D sweep).** 1.6:2.4:0.2 in **Stop max ×ATR** → 5 cells, EV-ranked, split-half validated. The winner is the best *holdout* EV, not the best headline.
- **Two knobs together (2-D grid).** 0.45:0.60:0.05 in **Verdict MED %** and 0.0003:0.0009:0.0002 in **Min net move % (after fees)** → 16 cells; watch the population-shift line.
- **Coupled sweep (constraint).** Sweep **ATR stop mult** and **Stop max ×ATR** over 1.6:2.4:0.2 with the ratio constraint above → the 9-cell square prunes to the 3-cell diagonal.

Interpretation

A result worth acting on shows a **positive holdout EV without a DIVERGENT flag**, on a cell with a decent n , in a session whose population didn't shrink to get there. Anything else is a null result — which is itself valuable: sweeps of the ATR fallback geometry on the current book have consistently come back flat with divergent winners, which is the tool steering you off a phantom rather than handing you one. And a genuine winner is still only the *start* of a spec proposal, never a change in itself.

27. MIN NET MOVE % Row (SETTINGS & TOOLS)

The one engine setting editable from the main window. Built by `BuildMinNetMoveRow` in `UI/MainForm_Layout.vb`. Like the EXIT GUARD strip (§23) it is a **live status element**, not a card/snapshot surface, so it sits outside the card ↔ snapshot parity rule.

Where: SETTINGS & TOOLS card, third row — below LOG / AUTO-RUN and the EXIT GUARD strip, above TOOLS. Always visible: unlike the guard strip, it does not collapse when no position is declared.

Format: a fixed label, an editable text box, and a derived read-out.

MIN NET MOVE % (after fees) [0.0005] → floor 0.0800% (fee 0.0300%, maker_maker)

What the box edits

`scoring.trade_costs.min_net_move_pct`, and **only** that key. It is the trader-owned half of the composed minimum-move floor behind the **BELOW_MIN_MOVE** context tag (§1):

`EffectiveMinMovePct = RoundTripFeePct(round_trip_style) + min_net_move_pct`

- `min_net_move_pct` — your minimum acceptable move **after** execution costs, as a fraction of entry price. A risk preference. Default `0.0005` (0.05%).
- `RoundTripFeePct` — derived from `maker_fee_bps`, `taker_fee_bps` and `round_trip_style` in the same `scoring.trade_costs` block: `maker_maker = 2 × maker`, `maker_taker = maker + taker`, `taker_taker = 2 × taker`, and any unrecognised style falls back to `maker_maker`. These are venue facts, **not editable from this row** — edit `settings.json` when Deribit changes its schedule.

At the shipped defaults: $2 \times 1.5 \text{ bps} + 0.0005 = 0.0008$, i.e. 0.08% of price.

The derived read-out

→ `floor <n>% (fee <n>%, <style>)` — printed from `EffectiveMinMovePct` and `RoundTripFeePct` themselves, not re-derived in the UI. The number on screen therefore cannot drift from the number the Step 5c gate uses.

Editing

Action	Result
Type a value, then Enter or click away	Validated, then saved
Escape	Reverts the box to the live setting; nothing is written
Value outside <code>0 ... 0.01</code> , or unparseable	Box turns red, read-out shows → <code>invalid (expected 0 ... 0.01)</code> . Not saved and not reverted — the bad text is left in place for you to correct
Value equal to the current setting	No file write at all
Save throws	Box turns red, read-out shows → <code>save failed: <message></code>

A successful save is an **operational save** — `SettingsLoader.Save(..., bumpVersion:=False)`. The settings **version** does not move and **no change_log entry is appended**, because a risk-preference turn is not a feature version; only `last_modified` is refreshed.

When it takes effect: the next analysis run, via the normal `SettingsLoader` hot-reload. No restart. The row is also re-read at the end of every run (`RefreshMinNetMoveRow`, called from `UI/MainForm_Analysis.vb`), so a **file-side** edit to `scoring.trade_costs` — a Deribit fee-schedule change, for instance — shows its new composed floor without a restart either.

Interpretation

Raise it to demand more room before the engine will call a trade; lower it to let tighter setups through. Because it is a fraction of **price**, not of ATR, it auto-scales with BTC — `0.0005` is $\approx \$31$ at \$62k and needs no recalibration as price moves. The effect to watch is §1's `BELOW_MIN_MOVE` context tag: that is the gate this knob moves, and only the **placed** target is measured against it, so a near structural target trips it exactly as a small ATR fallback target does.

The whole `scoring.trade_costs` prefix is **off the auto-tweaker surface** (HARD CONSTRAINT 26), so nothing but you or a file edit ever changes it. To test a value before committing to it, sweep the **Min net move % (after fees)** field in What-If Replay (§26) rather than turning the live knob — and note that because the fee half is fixed during a sweep, a `min_net` grid is a floor grid shifted by the round-trip cost.

28. BacktestRunner (offline CLI)

A console application — `tools/BacktestRunner/`, project `BacktestRunner.vbproj`, in the solution — that does the offline data work the WinForms app cannot: build a local historical store, replay the shipped pipeline over it, validate that replay against live rows, run the standard analysis report over an arbitrary CSV, and audit raw-trade capture health.

It is **read-mostly and separate from the live engine**: it never writes `settings.json`, never places orders, and never touches the running app's `analysis_log.csv`. It reads a settings file (`--settings`, default `settings.json`) purely as input. It is host-agnostic (no WinForms), so it survives the planned Linux port.

Running it. Build the solution, then run the exe from `tools/BacktestRunner/bin/Release/net8.0/` (or `bin/Debug/net8.0/`). It walks up from its own location to the directory holding `DeribitVerdictEngine.sln` and **sets that as the working directory**, so every relative path below — the store, `analysis_log.csv` — resolves against the repo root regardless of where you invoke it from.

Verbs

```
BacktestRunner fetch    --from yyyy-MM-dd --to yyyy-MM-dd
BacktestRunner replay   --from yyyy-MM-dd --to yyyy-MM-dd
                        [--settings <path>] [--out <path>] [--closed-bars]
BacktestRunner validate --from yyyy-MM-dd[Thh:mm] --to yyyy-MM-dd[Thh:mm]
                        --live <liveCsvPath> [--live2 <secondCsvPath>]
                        [--replay <existingSyntheticCsv>] [--report <markdownOut>]
                        [--settings <path>]
BacktestRunner report   --csv <analysisLogCsv> [--settings <path>]
BacktestRunner coverage --from yyyy-MM-dd --to yyyy-MM-dd
                        [--gap-ms <ms>] [--out <path>] [--strict] [--verify-venue]
                        [--evidence-dir <dir>] [--store-dir <dir>]
```

`--from / --to` are **required by every verb except report**, are parsed as UTC, and `--to` must be strictly greater than `--from`. `report` derives its own range from the CSV's row timestamps, so the two flags are meaningless there.

fetch — backfills the local historical store at `backtest_data/` from Deribit's public endpoints: 1m / 3m / 5m / 15m candles, raw trades (carrying the liquidation flag), and funding-rate history, one CSV per month per stream plus a `.state.json` of resumable cursors. It starts **20 hours before** `--from` so indicators have warm-up bars. Fetch-once: complete months are skipped on a re-run and a partial trade month resumes from its last stored timestamp. Paginated calls are spaced 200 ms apart.

replay — runs the shipped scoring pipeline over the store for the range and writes a synthetic `analysis_log.csv`-shaped file (default `backtest_log_<yyyyMMdd_HHmmss>.csv`, or `--out`). Prints rows written, rows per session, rows per verdict, and the first three sample rows. `--closed-bars` drops the forming-bar stub and replays **closed bars only**; the console echoes which bar mode is active.

validate — the honesty check on **replay**. It joins the synthetic rows to real logged rows over the same window and reports verdict-agreement and tier-agreement counts and percentages. `--live` is required; `--live2` joins a second live CSV (two collectors); `--replay` reuses an existing synthetic CSV instead of replaying again. Full markdown goes to `--report`, or to stdout when that is omitted.

report — runs the shipped `analysis/AnalysisRunner` pipeline over any CSV and writes the standard markdown report plus summary CSV **beside the input file**, not into the repo root. This is how a pooled book (local plus collector rows concatenated externally) gets reported on; the in-app Analysis Report link (§18) is hard-wired to the engine's own working-directory `analysis_log.csv` and cannot. It fetches forward-bar OHLC over the network, exactly as the in-app path does.

coverage — audits raw-trade capture health against the store (spec: `docs/trade-store-coverage-report-proposal.md`). Read-only: it never fetches (except under `--verify-venue`) and never writes to the store. Every

weekday UTC hour in range is classified and counted as one of **captured · DEFECT · trailing-edge · expected-missing · not-capturing · unknown-scope · out-of-scope-weekend**, alongside the longest observed trade gap against the `--gap-ms` threshold (default 300000, i.e. 300 s), the longest trailing-edge silence, and candle completeness at all four resolutions plus funding.

- `--gap-ms <ms>` — the gap threshold, in milliseconds.
- `--out <path>` — also write the full markdown report there (the console summary always prints).
- `--strict` — exit 1 when any DEFECT hour exists. Without it the verb **always exits 0**, so it is safe to run interactively.
- `--verify-venue` — adds the S0 venue diff over the last 24 hours of the range. This is the only part of coverage that touches the network; a failed fetch prints a warning and skips S0 rather than failing the run.
- `--evidence-dir <dir>` — aim the store *and* all three evidence files (`analysis_log.csv`, `ws_health.log`, `capture_marker.log`) at a copy-back's `aws_fetch/<stamp>/` directory instead of the repo root.
- `--store-dir <dir>` — override just the store, applied after `--evidence-dir`.

Both directory flags are validated: a path that does not exist is an error, not a silent fall-through. Every **coverage** run — including a plain one — prints the **resolved** store and evidence paths with a tag saying which flag chose them, because this argument parser silently drops a value-taking flag whose value is missing, and an unnoticed `--evidence-dir` at the end of a line would otherwise audit the local store and report “0 captured hours, 0 DEFECT, exit 0” — which reads as clean rather than as not-run.

Exit codes

Verb	0	1
fetch	backfill completed	—
replay	at least one row written	no rows written
validate	at least one joined pair	no joined pairs, or <code>--live</code> missing
report	markdown written over ≥ 1 row	CSV missing, <code>--csv</code> missing, or the forward-OHLC fetch failed so no markdown was written
coverage	always, unless <code>--strict</code>	<code>--strict</code> and at least one DEFECT hour

Bad or missing arguments, an unknown subcommand, and any unhandled exception all exit 1.

Interpretation

`fetch` → `replay` → `validate` is the sequence that earns trust in a backtest: the replay is only as good as its agreement with rows the engine actually logged, and `validate` is the number that says so. Treat a low verdict-agreement figure as a reason to distrust the replay, not the engine.

coverage answers a different question — not “is the engine right?” but “is the tape complete?” — and it is the check to run **before** any study that leans on the raw-trade store, because a DEFECT hour is silent everywhere else.